WILD COMMANDER IMPROVED V1.11I 2026-10-01

Руководство программиста

Полный API, панельный менеджер, плагины, драйверы, WildDOS/CORE32 и карта памяти

- **Публичный API 0–86**
- **Все вызовы WildDOS**
- **Готовые примеры ASM**
- **Регистры, флаги, структуры**

Документация по текущим исходникам репозитория
Сборка PDF: 11.10.2026

Оглавление

Wild Commander Improved: документация для разработчика

- С чего начать
- Граница стабильности
- Источники истины
- Обозначения

Панельный менеджер: устройство и программное использование

- 1. Карта подсистемы
- 2. Две панели — два файловых контекста
- 3. Структура панели, 36 байт
- 4. Каталожные страницы и индекс
- 5. Путь и устройство панели
- 6. Переключение панелей
- 7. Навигация и главный цикл
- 8. Как плагину работать с панелью
- 9. Как добавить команду в сам менеджер
- 10. Типичные ошибки
- 11. Исходники по назначению

Плагины и полный публичный API Wild Commander

- 1. Главный контракт
- 2. Как WC запускает плагин
- 3. Формат .WMF
- 4. Резидентные адреса и страницы
- 5. Окно SOW
- 6. Полная таблица API 0–86
- 7. Готовые рецепты
- 8. Параметры плагина в wc.ini
- 9. Ограничения, которые должен учитывать плагин
- 10. Где смотреть рабочие примеры

Драйверы накопителей: ABI и разработка

- 1. Место драйвера в стеке
- 2. Физическая модель
- 3. Таблица входов #3800
- 4. Общая область состояния
- 5. Контракт ошибки ABT
- 6. Позиционирование
- 7. Многоблочный обмен и буфер
- 8. Встроенные драйверы
- 9. Выбор драйвера через DOS_SWP
- 10. Каркас нового драйвера
- 11. Подключение нового драйвера к WC
- 12. Диагностика по слоям
- 13. Файлы-источники

Уровень WildDOS/CORE32: файловое API и внутренняя архитектура

- 1. Четыре уровня вызова
- 2. Установка при старте
- 3. Монтирование тома
- 4. Рабочая память CORE32
- 5. FAT32 ENTRY и имена
- 6. Потоковая модель
- 7. Публичные файловые сценарии плагина
- 8. Внутренняя таблица #4000
- 9. API 76 APPEND
- 10. API 77 FILEX
- 11. Согласованность FAT32
- 12. Нереентерабельные глобальные объекты
- 13. Как расширять DOS API
- 14. Исходники

Wild Commander Improved: карта использования памяти

- 1. Карта за тридцать секунд
- 2. Термины: окно, страница и смещение
- 3. Обычная раскладка окон
- 4. Детальная карта логических 64 КиБ
- 5. Карта физических страниц #00..#FF
- 6. Пул плагинов
- 7. Где размещать буфер: таблица решений
- 8. Границы и арифметика без переполнения
- 9. Стек и прерывания
- 10. Частые ошибки памяти
- 11. Контрольный список автора плагина
- 12. Источники в проекте

Wild Commander Improved: документация для разработчика

Эта папка описывает текущую архитектуру и программные интерфейсы Wild Commander Improved. Документация рассчитана на автора плагина, драйвера или изменения самого менеджера, а не только на пользователя готовой сборки.

С чего начать

Задача	Документ
Понять две панели, их состояния, каталоги, курсор и обновление	Панельный менеджер
Написать или перенести `.WMF`, вывести окно, читать клавиатуру и вызвать API	Плагины и публичный API
Реализовать низкоуровневый блочный драйвер накопителя	Драйверы
Работать с FAT32, потоками, APPEND и FILEX	Уровень WildDOS/CORE32
Выбрать безопасные адреса и физические страницы	Карта использования памяти

Единая печатная версия всех пяти руководств: Wild Commander Improved — руководство программиста (PDF). Она содержит общее оглавление и собирается скриптом `tools/build_docs_pdf.py`.

Если нужна одна практическая точка входа, обычный плагин вызывает функции так:

```
WLD     EQU #6006

        ; Пример API без дополнительного байта в A.
        LD      HL,Text
        LD      DE,#0101
        LD      A,11                ; API 11, TXTPR
        CALL    WLD

        ; Пример API, которому нужен собственный A.
        ; Значение передаётся через альтернативный A'.
        LD      A,#FE               ; основной текстовый экран
        EX      AF,AF'
        XOR     A                   ; API 0, MNGC_PL
        CALL    WLD
```

Номера API в тексте без префикса записаны десятичными. Префикс `#` означает шестнадцатеричное число: API `78` — это `#4E`, а не `#78`. Это особенно важно для старых C-заголовков, где часть поздних номеров записана ошибочно.

Граница стабильности

В проекте есть три разных уровня входов. Они похожи внешне, но не взаимозаменяемы.

Уровень	Вход	Для кого	Статус
Публичный API плагинов	`A=0..86`, `CALL #6006`	Любой `.WMF`	Основной совместимый интерфейс
Внутренний API CORE32	`CALL #4000+offset`	Код WC и тесно связанный системный код	Требует файловой страницы в окне `#0000`; раскладка состояния внутренняя
Приватный шлюз CORE32_EXT	`CALL #5123`, затем байт ID	Только CORE32	Деталь реализации Improved; плагинам запрещён

Практическое правило: код, который можно оформить обычным `.WMF`, должен использовать `#6006`. Прямой вызов `#40xx` не является ускоренной версией того же API: он обходит обёртки страниц, состояние панели и восстановление памяти.

Источники истины

Документы сверены с текущими исходниками. При будущих изменениях числовой ABI нужно проверять в следующем порядке:

1. `source/MD20.ASM` — диспетчер и фактическая таблица публичного API `0..86`.
2. `source/CORE32_FILEX_API.ASM` — машинно-читаемые константы API 77.
3. `source/KERNEL.ASM` и `source/CORE32.ASM` — внутренняя таблица `#4000`, память файлового ядра и фактическое поведение.
4. `source/WCVLS.ASM`, `source/WCVW.ASM` и `source/WCINI.ASM` — структуры панелей, загрузчик плагинов и конфигурация.
5. `source/PHYSICAL_MEMORY_MAP.ASM` — обязательный контракт верхних физических страниц.

`source/plugins/READ_ME.TXT` остаётся важным историческим описанием плагинной системы. Старые копии `wc_api.h` и `wc_h.h` полезны как примеры интеграции с C, но не считаются точным числовым ABI.

Обозначения

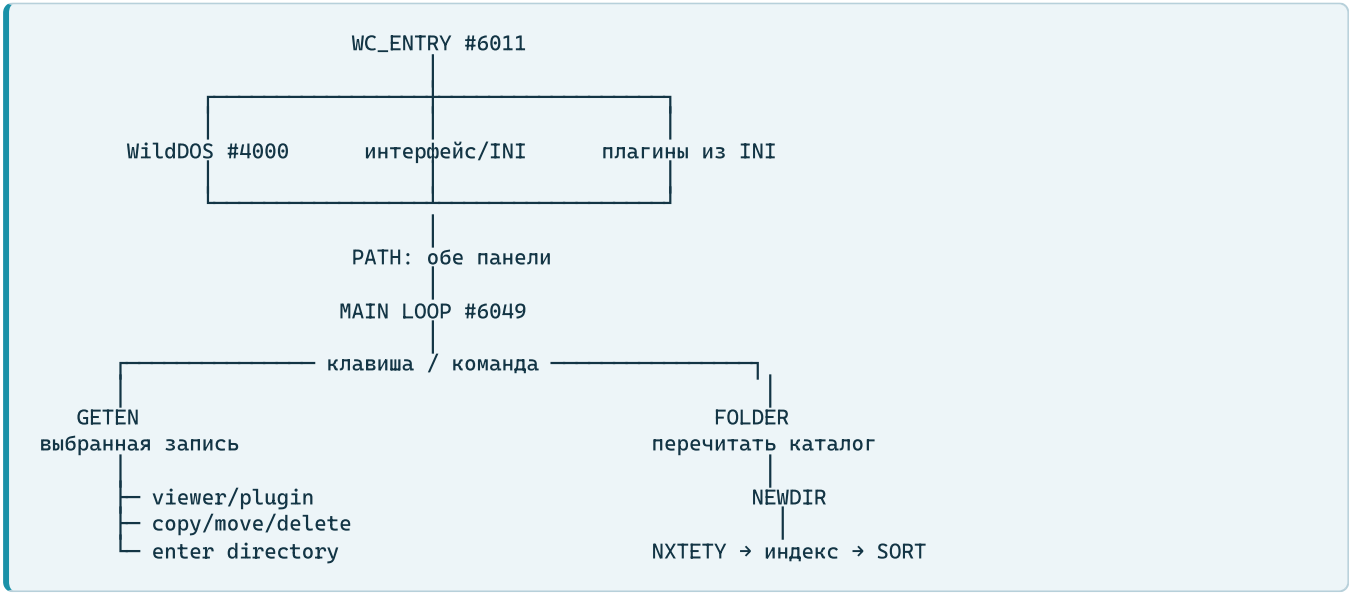
- `#1234` — шестнадцатеричное значение в синтаксисе используемых ASM.
- `LE16` / `LE32` — little-endian слово/двойное слово.
- `[start,end)` — полуинтервал: `end` уже не входит в диапазон.
- `Z` / `NZ`, `CF` — флаги Z80 непосредственно после возврата из функции.
- `A'`, `BC'`, `DE'`, `HL'` — альтернативный набор регистров.
- «страница» без уточнения — физическая страница TS-Conf размером 16 КиБ.
- «окно» — одно из четырёх логических окон Z80: `#0000`, `#4000`, `#8000` или `#C000`.

Не полагайтесь на незадокументированные сохранённые регистры. Диспетчер `#6006` как минимум изменяет альтернативный `BC`, а файловые расширения имеют собственные наборы разрушаемых регистров. Результирующие флаги нужно проверять сразу, до `OR`, `CP`, `INC` и других команд, которые меняют `F`.

Панельный менеджер: устройство и программное использование

Панельный менеджер связывает интерфейс, текущий каталог, драйвер и состояние CORE32. Для плагина панель представлена указателем `IX`; для кода самого WC — это две 36-байтовые структуры и два независимых файловых контекста.

1. Карта подсистемы



Основные точки текущего образа:

Адрес	Имя	Назначение
`#6011`	`WC_ENTRY`	Одноразовый старт, установка CORE32, INI и панелей
`#6049`	`WC_MAIN_LOOP`	Кадровый цикл и диспетчер клавиш
`#6149`	`Xpan`	Служебно заменить `IX` на другую панель
`#6160`	`XPAN`	Полное пользовательское переключение по Tab
`#62EB`	`PANELI`	Сброс вычисляемого состояния панели
`#62FE`	`GETEN`	Получить объект под курсором
`#63FE`	`FOLDER`	Перестроить активную панель
`#6407`	`NEWDIR`	Перечислить каталог и построить индекс
`#65BB`	`WLIT`	Нарисовать видимую часть списка
`#6715`	`I_N_T`	IM2: таймер, экран, клавиатура, часы
`#6868`	`DRACT`	Подключить файловый контекст и драйвер панели

Эти адреса — внутренняя раскладка текущего `boot.$C`, не публичные входы для `.WMF`. Плагин пользуется `#6006` и входным `IX`.

2. Две панели — два файловых контекста

```
                активная панель определяется IX

      LEFT                                      RIGHT
WINDOW = #7EA0                            WINDO2 = #7EC4
      │                                         │
      ├─ устройство 0..6                        ├─ устройство 0..6
      ├─ DOS/driver page #F0                    ├─ DOS/driver page #F1
      ├─ catalog pages #0B..#13                 ├─ catalog pages #14..#1C
      ├─ screen page #1E                        ├─ screen page #1F
      └─ path at #F7:#1000                      └─ path at #F7:#2000
```

Страница `#F0` или `#F1` отображается в логическое окно `#0000..#3FFF`. В ней находятся:

- буферы перечисления `#0000..#1FFF`;
- FAT-буфер `#3000` и секторные буферы `#3200/#3400`;
- текущий распакованный драйвер по `#3800`;
- поток, BPB, FAT32 ENTRY и текущий каталог.

Поэтому панели могут показывать разные устройства и каталоги: переключение меняет не только `IX`, но и рабочую физическую страницу DOS. Глобальный код CORE32 остаётся по `#4000`, а его данные меняются вместе с панельной страницей.

3. Структура панели, 36 байт

Структуры `WINDOW` и `WINDO2` имеют одинаковый формат. Offset дан одновременно в hex и decimal.

Offset	Размер	Поле	Назначение
`#00 / 0`	1	`style`	Тип окна и флаги рамки
`#01 / 1`	1	`cursor_mask`	Маска цвета курсора
`#02 / 2`	1	`x`	X панели: 0 слева, 40 справа
`#03 / 3`	1	`y`	Y панели
`#04 / 4`	1	`width`	Ширина
`#05 / 5`	1	`height`	Высота, меняется по текстовому режиму
`#06 / 6`	1	`color`	Цвет paper+ink
`#07 / 7`	1	reserved	Ноль
`#08 / 8`	2	`restore_buffer`	Для панелей `#FFFF`: фон отдельно не сохраняется
`#0A / 10`	1	`separator_1`	Первая горизонтальная линия
`#0B / 11`	1	`separator_2`	Вторая линия
`#0C / 12`	1	`cursor`	Видимая позиция курсора, от 1

Offset	Размер	Поле	Назначение
`#0D / 13`	1	`cursor_limit`	Нижняя граница/число доступных видимых строк
`#0E / 14`	1	`cursor_color`	Цвет активной строки
`#0F / 15`	1	`under_cursor`	Исходный цвет под курсором
`#10 / 16`	2	`list_top`	Номер первой видимой записи, от 0
`#12 / 18`	2	`entry_count`	Общее число записей каталога
`#14 / 20`	1	`at_end`	Внутренний признак конца списка
`#15 / 21`	1	`at_start`	Внутренний признак начала списка
`#16 / 22`	1	`screen_page`	`#1E` слева, `#1F` справа
`#17 / 23`	2	`saved_offset`	Сохранённое экранное/страничное смещение
`#19 / 25`	1	`catalog_base`	0 слева, 9 справа
`#1A / 26`	1	`catalog_page`	Текущая страница записей `0..8`
`#1B / 27`	1	`dos_page`	0 слева, 1 справа; физически `#F0+value`
`#1C / 28`	1	`activated`	Файловый контекст/драйвер уже активирован
`#1D / 29`	1	`device`	Номер устройства `0..6`
`#1E / 30`	2	`path_begin`	Начало строки пути на странице `#F7`
`#20 / 32`	2	`path_cursor`	Место добавления следующего компонента пути
`#22 / 34`	1	`restoring`	Защита во время восстановления пути
`#23 / 35`	1	`sort_mode`	0 off, 1 name, 2 extension, 3 size

Первые 16 байт совместимы с общей структурой окна SOW. Поля после `#10` — расширение именно панели. Они являются внутренним ABI: обычный плагин может читать их для диагностики, но должен менять панель только через публичные операции и код возврата.

3.1 Вычисление выбранного объекта

```
absolute_index = panel.list_top + panel.cursor − 1
               = word(IX+#10) + byte(IX+#0C) − 1
```

`GETEN` находит четырёхбайтовый элемент индекса, подключает страницу исходной записи и заполняет два резидентных кэша:

Адрес	Размер	Содержимое
`#7F28`	4	Первый кластер объекта (`BRRR`)
`#7F2C`	4	Размер файла (`ARRR`)
`#7F6B`	32	Текущая FAT32 ENTRY/нормализованная запись

После `GETEN` файловый просмотрщик получает размер из `ARRR`, имя из записи и сохранённый кластер для API 50. Эти области глобальны: любой следующий выбор или файловый вызов может их заменить.

4. Каталожные страницы и индекс

Один каталог хранится не как массив одинаковых 32-байтовых FAT ENTRY. CORE32 выдаёт нормализованные записи переменной длины; панель отдельно строит компактный индекс.

```
Левая панель                              Правая панель

#0B  index window                        #14  index window
#0C  entry data page 1                   #15  entry data page 1
 ...                                      ...
#13  entry data page 8                   #1C  entry data page 8

при чтении index отображается в #8000
при чтении записи data page отображается в #C000
```

Четырёхбайтовый элемент индекса:

```
+0  относительная data page
+1  адрес записи, low
+2  адрес записи, high
+3  длина/служебное значение записи для внутренних проходов
```

`NEWDIR` выполняет следующий конвейер:

```
PPATH
  │  сообщает CORE32 текущий кластер каталога
  ▼
GLSTCAT #401E
  │
  ▼
NXTETY #4057 ──► запись в #C000 ──► 4-byte index в #8000
  ▲                                        │
  └──────────── до конца каталога ─────────┘
                                           │
                                  SORT → WLIT
```

На панель выделено девять физических страниц. При исчерпании буфера построение останавливается; публичный API отмеченных объектов документирует номера `1..2047`, и код плагина не должен предполагать больший доступный индекс.

Нельзя сохранять адрес записи из `#C000` и использовать его после следующего `GETEN`, сортировки или файлового вызова: это адрес в переключаемой странице. Копируйте требуемые поля в память плагина.

Элемент списка в `#C000`: `+0` — отметка, `+1..+3` — расширение для сортировки, `+4..+7` — первый кластер, `+8..+11` — размер, `+12..+13` — время изменения, `+14..+15` — дата изменения, `+16` — признак (`0` — каталог), `+17` — имя с нулём. Время и дата идут подряд, как в записи FAT, и вместе дают 32-битный ключ сортировки по дате.

Сортировка переключается Ctrl+F3 (по имени), Ctrl+F4 (по расширению), Ctrl+F5 (по дате изменения, новые сверху) и Ctrl+F6 (по размеру); повторное нажатие выключает её. В `wc.ini` это `CSORT` 1..4. Каталоги во всех режимах идут отдельно от файлов. Сортировка по размеру до 2026-09-24 сравнивала младший байт размера с байтами кластера, и 256 байт стояли раньше 100 — исправлено.

Строка информации под панелью (`INFO=1`) включена по умолчанию: и в поставляемом `wc.ini`, и в шаблоне, который создаёт WC Setup. Она занимает всю ширину панели без рамки. В текстовых режимах на 80 колонок это 38 знаков: имя, размер справа в десяти знаках (с пробелами между тысячами, если в числе не больше восьми цифр) или `<DIR>`, дата изменения `ДД.ММ.ГГ` и время `ЧЧ:ММ`. В режиме 90 колонок — 43 знака: размер в тринадцати знаках и всегда с пробелами между тысячами, дата `ДД.ММ.ГГГГ`. У записи без времени (нули) дата не показывается. Хвост строки собирает функция расширения WildDOS `STATUS_TAIL`; ширину она берёт из окна панели (байт +4: 40 или 45), время и дату элемента `WCFX.STATUS` кладёт ей в `LOBU+44..+47`, за пределы строки.

5. Путь и устройство панели

Строки путей хранятся на физической странице `#F7`:

```
#1000  путь левой панели
#2000  путь правой панели

формат: "1:\DIR\SUBDIR\", #FF
```

Ключевые операции:

Процедура	Действие
`RPATH`	Создать корень `<device>:\`, сбросить файловое состояние
`MPATH`	Добавить каталог или обработать `..`
`PPATH`	Вывести/передать текущий путь и подключить DOS-контекст
`PATH/SIMUL`	Восстановить сохранённые пути обеих панелей из `wc.ini`

Номера устройств:

`device`	Драйвер
`0`	IDE Nemo master
`1`	SD Z-Controller, первый интерфейс
`2`	SD через NGS
`3`	IDE SMUC master
`4`	IDE SMUC slave
`5`	IDE Nemo slave
`6`	SD Z-Controller, второй интерфейс

`DRACT` сначала подключает `#F0 + dos_page` в окно `#0000`. Если панель ещё не активирована, она получает корневой путь и вызывает внутренний `DOS_SWP` с номером устройства. Инициализация накопителя и поиск FAT32 выполняются до первого построения каталога.

6. Переключение панелей

Есть две похожие, но разные процедуры:

```
Xpan                          XPAN
────                          ────
только меняет IX              гасит старый курсор
ставит status dirty           сохраняет состояние UI
для внутренней операции       меняет активную отметку/страницу
не рисует интерфейс           рисует новую активную панель
```

Используйте `Xpan`, когда команда должна ненадолго обратиться к противоположной панели, например получить каталог назначения. Обязательно вызовите его второй раз перед возвратом либо сохраните/восстановите исходный `IX`.

В плагине не вызывайте ни одну из них по внутреннему адресу. Входной `IX` уже указывает на активную панель, а переключение пользователя должно остаться за менеджером.

7. Навигация и главный цикл

Главный цикл синхронизирован с кадром через `EI/HALT`. Порядок обработки важен:

```
cursor
  → Alt/Ctrl
  → arrows, PgUp/PgDn, Home/End
  → Delete/Insert/Space, Backspace
  → status, Shift
  → F1, F3, F10, Tab
  → F2, F4, F5, F6, F7, F8, F9
  → Enter
  → печатный символ / '*'
```

Состояние прокрутки образуют три значения:

```
word (IX+#10)   первая видимая запись
byte (IX+#0C)   строка курсора от 1
word (IX+#12)   всего записей
```

С 2026-10-11 первый вызов цикла — `WDOS_EXT.MEDIA_IDLE` вместо `CURSOR`: если загрузочную карту SD-ZC вынимали и вернули (docs/04, 8.5), каждая панель на ней перемонтируется, как при выборе диска (`(IX+#1C)=0`, DRACT, ORIGA — корень), и перерисовывается строка состояния второй панели; затем `CURSOR`, как прежде.

Home/End и PgUp/PgDn меняют их, затем `WLIT` перерисовывает видимую область. Флаги `+20/+21` не являются заменой этой арифметике; это кэш внутреннего отрисовщика.

7.1 Файловые операции панелей

- F5 копирует текущий объект или отмеченные (Insert, пробел, «*» по маске) в каталог другой панели; каталоги — рекурсивно. Существующее имя не перезаписывается: «Name already exists!», диск не меняется. MKFILE сразу выделяет файлу цепочку нужной длины, данные идут кусками по 16 КиБ. Копия получает атрибут 0 (R/H/S/A источника не переносятся — так было и в исходном WC).

- Ход F5 и F8 (с 2026-09-26) — одно окно на всю операцию, цветом прежней полосы (PBWND, заголовок «COPY», у F8 — «DELETE»; с 2026-09-28 окно 42×7 по центру экрана в любом текстовом режиме — X/Y сбрасываются в «центр» перед каждым показом, прежде 66×5). После Enter диалог «Copy N files/dirs?» закрывается, и сразу, пока ищутся файлы и читается FAT, в строке 2 — «Copying started...»; с первой порции данных — «Copying имя» (по центру, до 40 знаков с префиксом); строка 3 — `<n of M>`, строка 5 — полоса по всей операции (40 клеток: доля в 64-х·40/64); строки 1 и 4 пустые. С 2026-09-27 n и M — файлы операции: до первого действия WC считает их (WCINI.CP_COUNT: отмеченный файл — один, каталог — все файлы его дерева; обход только на чтение, как у копии, с той же проверкой подкаталога; каталог панели — как был), n растёт на каждом начатом файле — и в каталогах. Прежде там был номер объекта панели: у каталога «0001 of 0001» на всё дерево. Больше 9999 — 9999; отказ чтения или испорченное дерево при счёте — «of ????», ошибку покажет сама операция. Каталог, где записей больше 65535 (стандартный FAT32 столько не допускает), тоже «of ????»: 16-битный индекс перечисления переполнился бы, и счёт после подъёма из подкаталога шёл бы по кругу (у самого обхода F5 — прежний предел). Счёт и номер — не больше 65535; дальше полоса стоит полной. M файлов делят полосу поровну: файл заполняет свою долю по уже переданным байтам (перед каждой порцией — по сумме прежних порций, без текущей; до конца файла доля не больше 63/64, у файла короче 64 байт до передачи — 0, полная — по концу файла: при отказе полоса не бывает полной раньше времени); в конце успешной операции полоса доходит до 64 перед закрытием окна (у каталогов без файлов). Прежде внутри каталога шли ступени 64·j/(j+8): число файлов дерева WC не знал. Полоса только растёт — и у файла размером около 4 ГиБ, где счётчик байтов упирается в #FFFFFFFF. Окно не открывается и не закрывается на каждом файле, сообщения об ошибках и «Abort operation?» открываются поверх него. Прежде диалог с кнопками [OK]/[CANCEL] стоял, пока ищутся файлы и читается FAT (у большого файла — секунды без перемен), а окно полосы мигало поверх него на каждом файле, начиная полосу с нуля. F8 (удаление) — то же окно: заголовок «DELETE», «Deleting started...», затем «Deleting имя»; прежде после Enter в диалоге стояло «Deleting is in Progress!». Счёт и доли — те же, по файлам: удалённый файл закрывает свою долю, и в каталоге, и отмеченный (имя файла — до его удаления, у первого файла — после: до первого удалённого в строке «Deleting started...»). В конце F8 полоса не дорисовывается до 64: «DIR is not empty!» удаление не прерывает. Отказ удаления отмеченного файла или каталога (и поиска каталога перед удалением) — «UNKNOWN ERROR!!», F8 останавливается (прежде отказ удаления каталога не проверялся, а файла — останавливал F8 молча). Внутри удаляемого дерева у файла (WCINI.DL_INNER, с 2026-09-27) «File not found» — только для имени, которого действительно нет; отказ чтения при его поиске, отказ записи каталога и отказ освобождения его цепочки — «UNKNOWN ERROR!!». У подкаталога так же (с 2026-09-27, WCINI.TREE_NOT_FOUND и DL_DIRDEL): «DIR not found (src)» (поиск перед входом, и в F5) и «DIR not found (dst)» (удаление опустевшего) — только для имени, которого нет; отказ чтения, записи или освобождения цепочки — «UNKNOWN ERROR!!». F8 в любом случае останавливается. Если перечисление каталога внутри дерева (файлы, подкаталоги, проверка пустоты перед удалением) кончилось на отказе чтения — «UNKNOWN ERROR!!», F8 останавливается. Прежде такой конец принимался за настоящий: F8 удалял подкаталог с непройденными файлами, их цепочки терялись без сообщения; проверка пустоты говорила «DIR is not empty!» либо удаляла каталог. F5 по дереву так же: сообщение и остановка, прежде копия дерева молча оставалась неполной. Недопустимый первый кластер каталога (1, `#0FFFFFF0..#0FFFFFF7`) — тоже отказ: прежде ядро теряло эту ошибку (NXTINI), и F5 такого каталога давал пустую копию без сообщения. Перед входом в каждый каталог дерева (с 2026-09-27) проверяется, что он настоящий: это не корень, его цепочка FAT цела (все кластеры в области данных тома, без свободных и недопустимых ссылок, не длиннее 4096 секторов — иначе замкнутая цепочка перечислялась бы без конца), первый сектор начинается «.» на этот же кластер и «..» на каталог, откуда вошли (у корня «..» — 0 или номер корневого кластера). Иначе запись испорчена — отказ, «UNKNOWN ERROR!!» (WCINI.DL_ENTER/ CP_ENTER, ID_TREE_CHILD

расширения). Прежде обход входил куда укажет запись: кластер 0 ядро понимает как корень (так задумано для «..»), и F8 одного такого каталога удалял неотмеченные файлы корня (на стенде — все 22, с boot.$c), F5 копировал корень; кластер корня или чужого каталога — по корню или по чужому дереву. Подкаталог, указывающий на сам текущий каталог, — тоже отказ; с 2026-09-27 — и подкаталог, в цепочке которого есть кластер корня (хвост, ушедший в корень, вёл F8 дерева по именам корня). Общие цепочки — отдельным пунктом ниже. Замкнутую или уходящую за том цепочку любого каталога ядро с 2026-09-27 ловит до чтения: панель показывает его пустым, поиск и создание отказывают — прежде панель, поиск и создание в таком каталоге висели без конца (исходный WC — так же). Каталог не растёт дальше 4096 секторов (65536 записей — предел FAT32): полный — «Not Enough Space!» при создании. Нулевая ссылка посреди цепочки файла (свободный кластер) — порча, поток кончается ошибкой; прежде ядро шло по ней в корневой каталог — F5 молча копировал байты корня вместо конца файла. Кластер за концом области данных тома ядро с 2026-09-27 считает недопустимой ссылкой везде, где идёт по цепочке (GIPAG, переходы потока, DLSG), — прежде испорченная цепочка вела чтение и запись за границу тома. Кластеры корня DLSG с 2026-09-27 тоже не освобождает (ID_CLASSIFY_DATA; таблица корня — в пункте об общих цепочках): запись файла на кластере корня (порча) F8 — и выбранную, и внутри дерева — удаляет с «UNKNOWN ERROR!!», корень цел; прежде F8 освобождал FAT корня, и том терял все имена (первый кластер) или хвост корня с его именами (второй и следующие). Отличает отказ признак `ABT` страницы, где шло перечисление (WCINI.DL_NEXT, CP_NEXT, DL_UP, DL_EMPTY, DL_NED); он живёт до следующего обмена с носителем. Поэтому однократный отказ при чтении наперёд, после которого обход сам перечитывает каталог (после каждого удалённого подкаталога — заново), может пройти без сообщения: итог тогда тот же, что без отказа. Удаление и копирование не атомарны: при отказе носителя посреди шага (запись файла уже убрана, а освобождение его цепочки отказало; новый кластер каталога назначения выделен, но не присоединён) остаются потерянные кластеры — всегда с сообщением и остановкой. Это политика ядра: лучше потерянная цепочка (её найдёт chkdsk), чем запись, указывающая на свободные кластеры. Код — WCINI.CP_*, DL_GO, DL_INNER и PBPR (WCFX), тест `test_copy_window.py`.

- Общие цепочки — две записи на одних кластерах уже испорченного тома: F8 одной освобождал данные другой, F8 каталога шёл по чужому дереву. С 2026-09-27 (частичная защита) ловятся два случая. Корень: ни один кластер его цепочки (не только первый) не бывает в освобождаемой цепочке (DLSG), в цепочке источника F5 и в цепочке подкаталога — таблицу кластеров корня (ROOT_BUILD расширения) строит каждая такая проверка заново, одно чтение сектора FAT на корень в один кластер. Записи того же каталога: F8 до первого удаления (WCINI.DL_ALIAS, ID_ALIAS_ADD и ID_ALIAS_SCAN) сверяет отмеченные объекты с записями их каталога на носителе, не с панелью: на первом кластере отмеченного не должна начинаться ещё одна запись — ни неотмеченная, ни отмеченная (две записи на одной цепочке — порча: удалить их по очереди нельзя, первое удаление освобождает цепочку, и второй каталог остался бы записью на свободном кластере), а цепочка отмеченного каталога не должна проходить через первый кластер ни одной записи, в том числе отмеченной, — и через «.» и «..» самого каталога панели. Нарушение или отказ чтения — «UNKNOWN ERROR!!», F8 не удаляет ничего; такой том — для chkdsk. Проход по каталогу — порциями по 16 секторов (между ними кадр IM2); больше ~250 разных кластеров в выборе (или длинный хвост отмеченного каталога) — по частям, каталог читается раз на часть, хвост каталога продолжается с того же места. Цена — проход по каталогу панели на каждую часть: обычно это доли секунды, но отмеченный каталог в тысячи кластеров внутри каталога в тысячи записей — минуты (16 частей по 16384 записи — около 5 минут при 14 МГц, между порциями часы и клавиатура обслуживаются). Чего защита не ловит: общие кластеры с записями других каталогов (файл внутри удаляемого дерева на кластерах объекта из другого каталога), слияние цепочек не на первом кластере (хвосты двух файлов), корень длиннее 768 кластеров (бывает только при кластере меньше 4 КиБ: больше 12288 записей при 512 байтах; дальние кластеры такого корня не в таблице).

Это находит только проверка всего тома (chkdsk): при каждом F8 это минуты на заполненной карте, и её не делают ни DOS, ни Windows. F5 общие цепочки не портит (источник не меняется), но копирует чужие данные — кроме корня, это не ловится. Проверено стендом (`alias_look.py`): прежняя сборка удаляла файл невыбранного соседа, освобождала данные файла-двойника, копировала хвост корня и падала после F8 файла в хвосте корня; новая останавливает каждый случай сообщением, исправные каталог и файл удаляет, как прежде.

- Перед копированием файла (F5, и в дереве) проверяется его цепочка FAT на всю длину данных (с 2026-09-27; WCINI.CP_SOURCE, ID_FILE_CHAIN): кластеры заняты, ссылки допустимы, в пределах тома и не на кластер корня, цепочка не короче размера. Иначе — «UNKNOWN ERROR!!», копия не создаётся. Прежде кластер источника, помеченный в FAT свободным, MKFILE тут же отдавал копии: источник и копия делили кластер, часть копии — чужие байты, и всё без сообщения; файл на кластере корня давал копию сектора корня. Проверяются только нужные размеру кластеры и запись FAT последнего из них: хвост цепочки за размером не обходится, цикл внутри нужной длины не ловится (чтение копии всё равно кончается на размере). Обход стоит ~1400 тактов на кластер при сплошной цепочке и ~7000 при разбросанной (сектор FAT на кластер; с 2026-09-27 — со сверкой с таблицей корня): у файла в 1 ГиБ при кластере 4 КиБ (262144 кластера) — порядка 25 с при 14 МГц до начала копирования. Внутри шлюза расширения прерывания выключены, поэтому обход идёт порциями до 1024 кластеров (первая короче, если длина не кратна 1024), а между ними WCFX.FC_LOOP отдаёт кадр обработчику IM2 (EI:HALT): часы, таймер и клавиши обслуживаются. Выделение цепочки копии (MKSG, ~4900 тактов на кластер) с того же дня идёт вне шлюза, при прерываниях, как в исходном WC: с 2026-07-16 его звал ALLOCATE_FILE расширения, и файл в гигабайт выделялся при DI минуту и дольше. Поиск свободного кластера для роста каталога (DIR_GROW), F7 (MKDIR) и APPEND с 2026-09-27 тоже идёт вне шлюза: расширение зовёт MKSG и SRHFCL через резидентный блок в странице ядра (CORE_MKSG, CORE_SRHFCL), который на время поиска возвращает окну #C000 страницу вызывающего и его прерывания, а после — снова #E8 и DI. Прежде на почти полном томе с далёкой подсказкой это было DI надолго: 65536 проверенных записей FAT ≈ 11 с при 14 МГц, миллион — около 3 минут. С 2026-10-11 сектор FAT проверяется одним вызовом шлюза (SCAN_FREE_SECTOR, до 128 записей плотным циклом), а не вызовом на каждую запись (~2330 тактов, CHECK_FREE_SCAN_LIMIT): полный обход FAT в 10 с лишним раз быстрее; прерывания обслуживаются между секторами.
- Отказ чтения FAT при поиске свободного места — код #FF у MKFILE и MKDIR: F5 файла и F7 пишут «UNKNOWN ERROR!!»; прежде «Not Enough Space!», хотя место было (код 16 у MKSG один на оба случая; различает MKSG_CHECKED расширения). F5 дерева на любой отказ MKdir каталога назначения пишет свой текст — «Make DIR failed (dst)».
- Отказ чтения или записи посреди копирования (LOAD512/SAVE512, обёртки WDOS.COPY_LOAD в ядре и WCFX.COPY_SAVE, общий отказ COPY_FAIL в MD20) прерывает F5: «UNKNOWN ERROR!!», недописанная копия удаляется (DELFL; если откажет и удаление, копия может остаться в каталоге назначения под своим именем либо без имени, потерянными кластерами, — отказ удаления не проверяется). Прежде ошибки не проверялись: после отказа чтения в копию уходило прежнее содержимое буфера, а F5 сообщал успех. Условие — драйвер, который сообщает ошибки (`ABT`): штатные драйверы `boot.$C` этого не делают — SD-ZC при ошибке чтения ждёт без предела, а отказ записи проходит незамеченным (см. docs/04, раздел о `#400C RDD`).
- Каталог нельзя скопировать в него самого или в его подкаталог: «Can't copy DIR into itself!», диск не меняется. До 2026-09-25 такая копия уходила в бесконечную рекурсию TREE/SUB1/TREE/SUB1/… до конца места, а стек индексов DIRCOPY (2 байта на уровень в странице окна #C000) через 8192 уровня вышел бы в страницу ядра. Проверка — DIRCOPY_GUARD (WCFX) и WCINI.DIRCOPY_INSIDE: если обе панели на одном устройстве, от каталога назначения поднимаемся по «..» до корня;

встретился исходный каталог (его первый кластер — BRRR, записан GETen) — отказ. «..» не нашлась — это корень, только если поиск прошёл без ошибки чтения и текущий каталог действительно корневой; иначе тоже отказ. Отказ, как любая ошибка F5, прерывает и всю отмеченную группу.

- Глубина рекурсивных F5 и F8 ограничена местом стеков обхода в странице окна #C000: за 256 байт до её конца — «DIR name too long/END of buffer» (WDOS.DCIN_TAIL, WDOS.RECBUF_DEPTH). Прежде очень глубокое дерево или зацикленный каталог испорченного тома выводили стек за #FFFF, в страницу потока.
- F6 — переименование на месте (перенос в другой каталог не делается); атрибуты записи сохраняются, если сектор новой записи удалось найти, перечитать и записать — при сбое на этом шаге переименование всё равно состоялось, а атрибут может остаться урезанным до бита каталога. Смена одного регистра имени («case.txt» → «CASE.TXT») отклоняется: «Name already exists!», диск не меняется.
- F7 — каталог. Причина отказа называется по коду MKDIR (FEEM): «Not Enough Space!», «Name already exists!», «Long Name is not valid!», при отказе носителя (в том числе записи FAT при выделении кластера) — «UNKNOWN ERROR!!» (с 2026-09-26; прежде сырой код драйвера давал «Long Name is not valid!»). Прежде на любую неудачу, и на нехватку места тоже, выводилось «DIR with same name already exists!». Тело нового каталога («.», «..», обнулённый хвост кластера) пишется до записи в родителе; при отказе цепочка освобождается (если откажет и запись FAT, могут остаться потерянные кластеры), и каталог с мусором свободного кластера не появляется. С 2026-09-26 так же и при продлении заполненного родительского каталога (любая новая запись — F5, F6, F7, UNZIP): новый кластер сначала обнуляется, потом вписывается в цепочку каталога, а отказ чтения каталога прекращает операцию, а не принимается за конец каталога.
- F8 и Delete — удаление, каталоги — всем деревом (DIRTREM). После удаления перечитываются обе панели (WCINI.DEL_REFRESH). Если соседняя панель стояла внутри удалённого дерева (первый кластер её каталога теперь свободен в FAT, не начинается записью «.» этого каталога, вторая запись не «..» каталога либо родитель из неё не ссылается на него живой записью каталога — кластер ушёл под файл, возможно ещё не писавший данных), она уходит в корень своего устройства; если FAT или сектор не прочитать — тоже (проверка — ID_DIR_ALIVE расширения; обход родителя — не больше 4096 секторов, предел каталога FAT32, чтобы зацикленная цепочка испорченного тома не вешала WC). Прежде она оставалась в освобождённом кластере: показывала удалённые файлы, а F7 и F5 в ней писали запись в свободное место (новый объект терялся) или поверх файла, которому этот кластер уже отдан. Та же проверка соседней панели — после плагина с кодом возврата 3 (PLUGIN_REFRESH): плагин мог удалить каталог через API 75 и тут же создать файл на его кластере. Проверка ловит кластер, отданный файлу, но целостности тома не доказывает: у «..» сверяются два байта имени и бит каталога, сам родитель (занятость в FAT, путь от корня) не проверяется.
- На диске без свободных кластеров отказ приходит не сразу: поиск места (SRHFCL) проходит всю FAT и на каждый кластер вызывает шлюз расширения. На томе 48 МБ (96 760 кластеров) это около минуты на стенде, на больших картах дольше. Редактор всё это время показывает «Saving...», затем «Save ERROR! Original file preserved.»; данные не меняются.
- До 2026-09-25 F8 по каталогу с подкаталогами обрывался на «DIR not found (src)»: стек имён DIRTREM лежит в странице окна #C000, а поиск ядра заходит в расширение, и шлюз на это время ставит туда свою страницу. Имя подкаталога теперь копируется в LOBU+768 (переходники DIRTREM_SUBDIR и DIRTREM_DELETE в MD20), и дерево удаляется целиком.
- Две панели на разных устройствах — два тома разного размера. Граница области данных у каждого своя (страница потока, поле RECCAT). До 2026-09-25 она была одна на все тома, в странице расширения: после монтирования большего тома аллокатор меньшего мог выдать кластер за концом его раздела.

- Обычные операции (MKFILE, MKDIR, DELFL, F5, F8) FSInfo не пишут (с 2026-08-24: эта запись зависала на реальной SD-ZC), поэтому счётчик свободных кластеров в FSInfo после них устаревает. Пишет его безопасная RFRH (#4030, RFRH_SAFE расширения): её зовут APPEND, когда заводит файлу новую цепочку, FILEX после освобождения цепочки (хвоста при усечении SET_EOF32, старого назначения при MOVE REPLACE) и плагин, вызвавший #4030 сам. Счётчик тогда — «неизвестно» (#FFFFFFFF), Next_Free — текущая подсказка аллокатора (FSTFRC, негодная — #FFFFFFFF). Данные это не портит; chkdsk и fsck.fat счётчик поправят, Windows до этого может показывать неверное свободное место.

Все операции проверены на стенде сверкой двух одинаковых образов (`Build/plugin-ondemand-20260924/fileops`): каждый изменённый сектор приписывается объекту операции, чужие данные — файлы, записи каталогов, цепочки, служебные сектора — не меняются; после каждой операции том проверяется на уровне chkdsk (копии FAT, потерянные и общие кластеры, LFN, имена 8.3, «.»/«..», FSInfo) и независимым чекером проекта ChkDsk. Образы — с кластером 512 байт без MBR и с MBR и кластерами 4 и 32 КиБ.

8. Как плагину работать с панелью

8.1 Получить файл, на котором запущен плагин

При вызове по расширению WC уже:

1. заполнил текущую ENTRY;
2. передал размер в `DE:HL` и имя по `BC`;
3. спозиционировал файловый поток на начало.

Можно сразу вызывать API 48 `LOAD512`. Если перед чтением выполнялся другой файловый вызов, восстановите позицию API 50 `GIPAGPL`.

8.2 Найти объект по имени в активном каталоге

```
        LD      HL,Name
        LD      A,59                    ; FENTRY
        CALL    #6006
        JR      Z,.not_found
        ; DE:HL = размер. Для чтения затем API 62 GFILE.

Name:   DB      #00,"FILE.BIN",0
```

8.3 Обработать отмеченные файлы

API 54 не требует разбирать внутренний каталог панели:

```
HL=0       → NZ, BC=число отмеченных; Z, если отмеченных нет
HL=1..2047 → DE получает name,0; Z = имя получено
```

На каждом успешном объекте API также позиционирует поток в начало файла, если bit 7 регистра H не был установлен. `IX` должен оставаться адресом панели, полученным при входе.

8.4 Изменить каталог и обновить экран

После MKFILE, MKDIR, DELFL, RENAME, APPEND или FILEX завершите плагин с `A=3`:

```
        LD      A,3                 ; WC сам выполнит FOLDER/NEWDIR/SORT
        RET
```

Это единственный безопасный публичный способ согласовать число записей, индекс, сортировку, курсор и строку статуса. Прямое увеличение `(IX+#12)` не создаёт запись и повреждает представление панели.

8.5 Перебирать каталог отдельно от панели

Клонируйте поток API 57, `D=#FE`, затем сбросьте перечислитель API 56, `A'=1`, и вызывайте API 58. Так переход в подкаталог или FENTRY не меняет живой поток, которым менеджер продолжит пользоваться после возврата.

9. Как добавить команду в сам менеджер

Для изменения `MD20/WCFX/WCVW` используйте такой порядок:

```
клавиша в MAIN
    ├─→ GETEN, если нужен выбранный объект
    ├─→ DRACT/файловая операция в контексте активной панели
    ├─→ Хран дважды, если нужен контекст второй панели
    ├─→ FOLDER для каждой изменённой панели
    └─→ пометить STUS и вернуться в кадровый цикл
```

Правила внутренней команды:

- сохраняйте `IX`, если временно меняете панель;
- не держите указатели в `#8000/#C000` через вызов, который меняет страницу;
- считайте `#7F28/#7F2C/#7F6B` одноэлементным глобальным кэшем;
- перед прямым `#40xx` подключайте правильную страницу `#F0/#F1` и на каждом выходе восстанавливайте прежнее отображение;
- после изменения каталога перестраивайте индекс, а не редактируйте его частично;
- учитывайте, что F5/F6 используют обе панели и два разных DOS-контекста.
- при F5 храните оставшееся логическое число байт из `FENTRY` и передавайте последнему `LOAD512/SAVE512` только `ceil(bytes_left/512)` секторов; FAT EOC ограничивает выделенную цепочку, но не обозначает логический конец файла.

10. Типичные ошибки

Ошибка	Почему проявляется	Правильное действие
Сохранён указатель на запись в `#C000`	Следующий GETEN подключил другую страницу	Скопировать имя/метаданные в страницу плагина
Плагин изменил `(IX+#12)`	Изменилось число, но не индекс и каталог	Выполнить файловую API и вернуть `A=3`
После FENTRY сразу вызван LOAD512	FENTRY только нашёл запись	Вызвать API 62 `GFILE`
F5 ждёт EOC для маленького файла	Выделенный кластер длиннее логического файла	Ограничить последний обмен размером из FENTRY
GDIR выполнен в исходном потоке панели	Активный каталог панели заменён изнутри плагина	Сначала клонировать STREAM `D=#FE`
Служебный Xpan вызван один раз	`IX` остался на другой панели	Вызвать второй раз или восстановить IX
Один файловый адрес используется в обеих панелях	`#0000` отображает разные физические страницы	Копировать данные через постоянное окно/явный буфер

11. Исходники по назначению

Файл	Что искать
`source/MD20.ASM`	MAIN, навигация, GETEN, FOLDER, NEWDIR, DRACT, публичные обёртки
`source/WCVLS.ASM`	Две 36-байтовые структуры и глобальные переменные менеджера
`source/WCFX.ASM`	F2/F4/F5/F6/F7/F8, отметки, статус и файловые диалоги
`source/WCVW.ASM`	Просмотрщики, выбор плагина и передача текущего объекта
`source/WCINI.ASM`	Пути, устройства, сортировка и восстановление панелей
`source/KERNEL.ASM`	Состояние DOS, которое физически принадлежит каждой панели

Плагины и полный публичный API Wild Commander

Этот документ отвечает на практический вопрос: как написать `.WMF`-плагин и как вызвать каждую публичную функцию Wild Commander. Он описывает всю таблицу API `0..86`, а не только новые API 76 и 77.

1. Главный контракт

Публичная точка входа одна:

```
WLD     EQU #6006

        LD      A,номер_API
        CALL    WLD
```

Номер API всегда передаётся в основном `A`. Если целевой функции одновременно нужно собственное входное значение `A`, это значение заранее кладётся в `A'`:

```
        LD      A,значение_для_функции
        EX      AF,AF'
        LD      A,номер_API
        CALL    WLD
```

Диспетчер `#6006` выбирает двухбайтовый адрес из таблицы, выполняет `EX AF,AF'` и передаёт управление функции через `RET`. Поэтому переданное в `A'` значение становится обычным `A` уже внутри функции. Возвращаемый обычный `A` и флаги приходят вызывающему коду без обратного обмена.

Удобные макросы:

```
WLD     EQU #6006

        MACRO WC_CALL api
        LD      A,api
        CALL    WLD
        ENDM

        ; Перед вызовом значение функции должно находиться в A.
        MACRO WC_CALL_A api
        EX      AF,AF'
        LD      A,api
        CALL    WLD
        ENDM
```

Номера без `#` в этом руководстве десятичные. Например, API `78` имеет машинный номер `#4E`. Запись `LD A,#78` вызовет совсем другой, несуществующий элемент за таблицей. Это одна из ошибок старых C-заголовков.

Диспетчер как минимум разрушает альтернативный `BC`. Сохранять остальные регистры следует только там, где это прямо обещано конкретной функцией. Флаги результата проверяются сразу:

```
        LD      A,59                ; FENTRY
        CALL    WLD
        JR      Z,.not_found        ; до OR/CP/INC и других команд
```

2. Как WC запускает плагин

WC отображает выбранную страницу плагина в окно `#8000..#BFFF` и выполняет:

```
        CALL    #8000
```

На входе:

Регистр	Значение
`A`	Тип запуска: `#00`, `#01`, `#02`, `#03`, `#04`, `#06`, `#12` или `#14`
`DE:HL`	32-битный размер выбранного файла, `DE` — старшее слово
`BC`	Адрес нуль-терминированного имени при типах `#00`, `#04`, `#14`
`IX`	Адрес 36-байтовой структуры активной панели
`A'`	Для `#00` и `#04`: индекс `0..31` у WMF `#02..#10`, код обработчика `0..255` у WMF `#11` — по записи, совпавшей с расширением файла; `#04` из меню без совпадения (универсальный просмотрщик, `+33` bit 1) — `#FF`
`DE'`	Число объектов панели, включая `..`, для `#00/#04/#12/#14`
`HL'`	Номер переданного объекта от 1, для `#00/#04/#12/#14`

Типы запуска:

Код	Причина
`#00`	Файл передан по совпавшему расширению
`#01`	Немедленный запуск после ручной загрузки плагина по Enter
`#02`	Таймер экранной заставки
`#03`	Меню плагинов F10
`#04`	Меню выбора просмотрщика Shift+F3/Shift+Enter
`#06`	Одноразовый автозапуск из `[PLUGINS]` при старте WC
`#12`	Команда F2
`#14`	Команда F4

Если плагину понадобится исходный `IX`, сохраните его до первого вызова кода, который использует IX:

```
PluginEntry:
        PUSH    IX
        ; ... работа плагина ...
        POP     IX
        XOR     A
        RET
```

Значение `A` при возврате:

`A`	Действие WC
`0`	Обычный выход
`1`	Файл не распознан; попробовать следующий обработчик, только для `#00`
`2`	Передать следующий файл, только для `#00`
`3`	Перечитать обе панели, сохранив активную; недоступно для `#01/#02`
`4`	Передать предыдущий файл, только для `#00`

Следующий (предыдущий) файл WC открывает как по F3: тем плагином, что его принимает, — и цепочка идёт дальше. Файл, который никто не открывает, её завершает: WC возвращается в панель с курсором на нём (с 2026-10-11; прежде на таком файле открывалось меню Shift+F3).

При входе может быть включён любой видеорежим. API 15 `GEDPL` переводит WC в основной текстовый режим, но уничтожает `#1000..#1447` текущей страницы окна `#0000`. Для файлового просмотрщика сначала определите формат, затем вызывайте `GEDPL`; так неизвестный файл можно вернуть следующему обработчику без мигания экрана.

2.1 Тип плагина и причина запуска — не одно и то же

Байт заголовка `.WMF +197` задаёт **условия регистрации плагина**. Регистр `A` при `CALL #8000` сообщает **фактическую причину конкретного запуска**. У составных типов эти числа различаются: например, плагин типа `#15` получает `A=#03` из F10, `A=#04` из меню просмотрщиков и `A=#14` по F4.

Полная матрица типов:

`.WMF +197`	Когда WC вызывает плагин	Фактический `A` на входе
`#00`	Только при совпадении расширения	`#00`
`#01`	Сразу после ручной загрузки самого `.WMF` по Enter	`#01`
`#02`	По таймеру заставки или из F10	`#02` / `#03`
`#03`	Из меню плагинов F10	`#03`
`#04`	Из меню просмотрщиков Shift+F3/Shift+Enter	`#04`
`#05`	Из F10 или из меню просмотрщиков	`#03` / `#04`
`#06`	Один раз при старте, если строка находится в `[PLUGINS]`; с `+33` bit 0 — ещё и сразу после своей загрузки	`#06` / `#07`
`#11`	Из F10 или по F2	`#03` / `#12`
`#12`	По F2	`#12`
`#13`	Из F10 или по F4	`#03` / `#14`
`#14`	По F4	`#14`
`#15`	Из F10, меню просмотрщиков или по F4	`#03` / `#04` / `#14`

Блок расширений `+64..+159` действует независимо от `+197`. Если в нём есть расширение, любой из перечисленных типов дополнительно может получить файл с `A=#00`. Чтобы плагин никогда не запускался по расширению, заполните все 96 байт блока нулями.

Меню Shift+F3/Shift+Enter (с 2026-10-11) строится под выбранный файл. В нём:

- любой плагин, чей список расширений (`+64`, у WMF `#11` — записи метаданных) содержит расширение файла: типы `#04/#05/#15` вызываются с `A=#04`, остальные — с `A=#00`, как по Enter; в `A'` — индекс или код совпавшей записи;
- просмотрщик `#04/#05/#15` с `+33` bit 1 — «открывает любой файл» (текстовый/HEX-просмотрщик, редактор): `A=#04`, `A'=#FF`.

Подпись пункта — `+204`, а если она пуста — имя `+165`. Не больше восьми пунктов, в порядке таблицы заголовков. Прежде меню показывало все `#04/#05/#15` для любого файла (плеер трекерной музыки — и для `.MID`), плагин по расширению (`#00`) — никогда, а выбранный получал `A'` от прошлого файла.

2.2 Жизненный цикл автозапуска `#06`

```
wc.ini: [PLUGINS]
        │
        ▼
загрузчик выделяет страницы и сохраняет 512-байтовый заголовок
        │ тип #06 пока только зарегистрирован
        ▼
GEDZ → DEF: интерфейс WC уже построен
        │
        ▼
AX_RUN просматривает заголовки в порядке [PLUGINS]
        │
        ├─ подключает entry-page
        ├─ CALL #8000, A=#06
        ├─ возвращает прежнюю страницу окна #C000
        └─ удаляет этот заголовок и сдвигает следующие
        │
        ▼
PATH восстанавливает каталоги и позиции обеих панелей
```

Установка сразу при загрузке. Если в заголовке `#06` стоит `+33` bit 0, ядро (хвост `ALPL` в MD20, с 2026-09-25) вызывает плагин ещё и сразу после его загрузки из `[PLUGINS]`, с `A=#07`, — до разбора следующей строки. Так ставит свои перехваты `PLM.WMF`: остальные плагины он уже только регистрирует, не загружая. Потом AX_RUN вызывает такой плагин ещё раз, с `A=#06`, как любой `#06`, и удаляет его заголовок; второй вызов плагин обязан переносить (менеджер видит уже стоящий перехват и выходит). Ручная загрузка `.WMF` по Enter раннего вызова не делает.

Контракт `#06`:

- проход выполняется ровно один раз при старте; ручная загрузка такого `.WMF` позднее не запускает проход повторно (ранний вызов `A=#07` — отдельный, см. выше);
- несколько модулей `#06` выполняются по порядку строк `[PLUGINS]`;
- на входе гарантирован `A=#06`; файловые `BC`, `DE:HL` и альтернативные регистры для этого типа использовать нельзя;
- код возврата плагина не трактуется как переход по файлам или обновление панели: после `RET` заголовок в любом случае удаляется;
- выделенные физические страницы не перемещаются и не выдаются другим плагинам до завершения сеанса WC;

- удалённый заголовок больше не участвует в F10, меню просмотрщиков, F2/F4 и маршрутизации расширений;
- окно `#C000` после всего прохода восстанавливается перед вызовом `PATH`.

Тип предназначен для резидентного установщика или провайдера. `FILEX.WMF` публикует страницу провайдера в шлюзе API 77, поэтому стоит первой активной строкой `[PLUGINS]`. После удаления заголовка FILEX не виден пользователю, но его зарезервированная страница продолжает обслуживать вызовы.

С 2026-10-10 `FILEX.WMF` — ещё и обязательная часть расширения ядра: в нём хранится оверлей страницы `#E8` — код создания, переименования и удаления записей, APPEND, псевдонимов, часов и строки состояния, которому нет места в `boot.$C` (docs/05, 5.5). FILEX заявляет раннюю установку (`+33` bit 0) и ставит оверлей сразу после своей загрузки, сверив штамп сборки. Если оверлея нет, AX_RUN до первого плагина `#06` выводит окно и останавливает машину (`DI:HALT`):

```
Core extension plugin: FILEX.WMF
was not found in folder /WC.          - FILEX нет или он старый
in folder /WC doesn't match boot.$C.  - FILEX от другой сборки
Commander cannot work. HALT.
```

`boot.$C` и `FILEX.WMF` обновляются только вместе: штамп сборки у них общий. WC Setup не даёт снять FILEX с отметки.

Ранний плагин (`+33` bit 0), стоящий в `[PLUGINS]` выше FILEX, в вызове `A=#07` не должен создавать, переименовывать и удалять файлы: оверлея ещё нет, и вход в его код останавливает машину (на месте оверлея — `HALT`), хотя и без записи на диск. Чтение каталогов и файлов идёт загрузочной частью и работает. `PLM.WMF` в раннем вызове файлов не пишет и может стоять выше FILEX.

3. Формат `.WMF`

Первые 512 байт файла — заголовок. У WMF `#02..#10` за ним сразу лежат блоки кода. У WMF `#11` между заголовком и кодом находится список расширений переменной длины (раздел 3.1.1). Блоки кода загружаются в выделенные плагину 16-КиБ страницы.

3.1 Поля заголовка

Offset	Длина	Поле
`+0`	16	Нули/резерв
`+16`	16	Строго `WildCommanderMDL`
`+32`	1	Версия формата; загрузчик принимает `#02..#11`
`+33`	1	bit 0 — только для `#06`: вызвать сразу после загрузки из `[PLUGINS]` с `A=#07` (см. 2.2); bit 1 — только для `#04/#05/#15`: в меню Shift+F3 для любого файла (см. 2.1); остальные биты — резерв, ноль
`+34`	1	Число резервируемых 16-КиБ страниц
`+35`	1	Относительная страница, отображаемая в `#8000` перед `CALL #8000`
`+36`	12	Шесть пар `[относительная_страница, число_секторов_по_512]`

Offset	Длина	Поле
`+48`	15	У `#02..#10` резерв; у `#11` описание списка, см. 3.1.1
`+63`	1	Маршрутизация расширений по Enter/F3
`+64`	96	У `#02..#10`: 32 расширения по три заглавных байта; у `#11`: нули, не используется
`+160`	1	Ноль
`+161`	4	Максимальный размер принимаемого файла, LE32
`+165`	32	Имя плагина, дополненное пробелами
`+197`	1	Условие запуска
`+198`	6	Подпись F2 или F4 для подходящих типов
`+204`	24	Подпись в меню просмотрщиков; пустая (первый байт — пробел или ноль) — в меню имя `+165`
`+228`	32	Резерв, нули
`+260`	252	Схема параметров из `wc.ini`

Биты `+63`:

- bit 0 = 0: запуск по Enter и F3;
- bit 0 = 1, bit 1 = 0: только Enter;
- bit 0 = 1, bit 1 = 1: только F3.

Так байт `+63` проверяет и ядро — для всех версий формата. До WMF `#11` ядро смотрело на весь байт: только ноль давал «Enter и F3», и значение `#02` означало «только F3».

Условия `+197`:

Код	Запуск
`#00`	Только по расширению
`#01`	Сразу после ручной загрузки
`#02`	Таймер и F10
`#03`	F10
`#04`	Меню просмотрщиков
`#05`	F10 и меню просмотрщиков
`#06`	Один раз при старте из `[PLUGINS]`
`#11`	F10 и F2
`#12`	F2
`#13`	F10 и F4
`#14`	F4
`#15`	F10, меню просмотрщиков и F4

Наличие расширений действует независимо от `+197`: даже меню-плагин можно одновременно зарегистрировать как файловый обработчик.

3.1.1 WMF #11: переменный список расширений

Версия формата `+32=#11` не связана с типом запуска `+197`. Старые плагины пересобирать не требуется. Новый формат требует обновлённого `boot.$C` и, если используется загрузка по требованию, обновлённого `WC/PLM.WMF`: прежний менеджер не пропускает секторы списка и загрузил бы их вместо кода, поэтому оба файла обновляются вместе. Старое ядро отвергает неизвестную версию.

Offset	Длина	Значение для `#11`
`+48`	2	LE16: значащие байты списка, без секторной добивки
`+50`	2	LE16: число записей
`+52`	2	В файле нули; внутренний указатель в зарегистрированной копии RAM
`+54`	1	Схема записи: `1`
`+55`	8	Резерв, нули (упаковщик обнуляет)

Список начинается с файлового смещения 512. Каждая запись содержит:

```
длина имени: u8 (1..255), код обработчика: u8 (0..255), имя: length байт
```

Имя записывается без точки, допускает печатный ASCII `33..126`, кроме `. / \ * ?`. Сравнение не зависит от регистра ASCII. Используется часть полного имени после последней точки, поэтому расширения длиннее трёх букв поддерживаются; так же сравниваются и файлы только с коротким именем 8.3. Совпадение точное, без масок; при дубликате побеждает первая запись. Упаковщик отвергает дубликаты, чтобы не создавать неоднозначный файл. Файл без точки в имени списком `#11` не обрабатывается: у старых WMF его ловила запись из трёх пробелов, в `#11` такой записи нет, и упаковщик её не переносит (отказ с ошибкой).

Список дополняется нулями до границы сектора (кратно 512 байтам). Код начинается с `512 + ceil(metadata_bytes / 512) * 512`. Метаданные не входят в число страниц `+34` и пары блоков `+36`. Поля имени, типов, максимального размера и параметров `+161..+511` сохраняют прежний смысл. Пустой список: обе длины равны нулю, между заголовком и кодом нет дополнительных секторов.

В `A'` приходит **код обработчика из записи**, а не её порядковый номер. Любое количество псевдонимов может ссылаться на один обработчик. Например, записи `TXT/0`, `TEXT/0`, `LOG/0` передают `A'=0`. Это позволяет иметь больше 256 расширений без изменения регистрового интерфейса плагина. Значения кодов определяет сам плагин; добавление ассоциации не добавляет декодер файла.

Прежнего предела в 32 расширения на плагин нет. Текущая реализация выделяет общий пул метаданных 16 КиБ (страница `#E9`) для всех плагинов. Каждый список занимает непрерывное целое число секторов. Например, 1000 пятибуквенных расширений занимают 7000 значащих байт и 7168 байт пула. Вместимость зависит от длины имён, других списков и непрерывного свободного места. Общая таблица остаётся на 32 плагина.

Список полностью проверяется перед регистрацией. Неверная схема, длины, записи, обрыв чтения или отсутствие места дают ошибку без добавления полузагруженного заголовка. WC показывает окно «Invalid WMF metadata» (неверное описание или запись), «WMF metadata memory full» (нет места в пуле) или

«WMF metadata read error» и ждёт клавишу, как при других отказах загрузки плагина; следующие строки `[PLUGINS]` загружаются дальше. Код нерезидентного плагина по-прежнему загружается по требованию. Выгрузка кода сохраняет ассоциации; удаление заголовка позволяет повторно использовать его место при следующей регистрации. Порядок плагинов и флаги Enter/F3 не меняются.

Утилита сохраняет код и параметры существующего WMF:

```
python tools/wmf_metadata.py Build/VIEWER.WMF Build/VIEWER11.WMF
python tools/wmf_metadata.py Build/VIEWER.WMF Build/VIEWER11.WMF --extensions extensions.json
```

Без JSON переносятся прежние расширения с прежними индексами обработчиков. JSON — массив вида `[{"extension":"TXT","handler":0},{"extension":"TEXT","handler":0}]`. Выходной путь должен отличаться от входного. Исходные константы формата: `source/WMF_FORMAT.ASM`; машинные регрессии: `tests/core32_unreal/test_wmf_dynamic.py`.

3.2 Минимальный одностраничный заголовок

```
        DEVICE ZXSPECTRUM128
        ORG     0

PluginFile:
        DS      16,0
        DB      "WildCommanderMDL"
        DB      #10,0                   ; версия, резерв
        DB      1,0                     ; одна страница, entry page 0
        DB      0,(PluginEnd-PluginCode+511)/512
        DS      5*2,0                   ; ещё пять пустых описателей
        DS      15,0
        DB      0                       ; Enter и F3
        DB      "TXT"                   ; расширение 0
        DS      31*3,0
        DB      0
        DD      #FFFFFFFF               ; без ограничения размера
PluginName:
        DB      "Example viewer"
        DS      32-($-PluginName)," "
        DB      #00                     ; обработчик расширения
        DS      6,0
        DS      24,0
        DS      32,0
        DS      252,0
        ASSERT  $ == 512

        DISP    #8000
PluginCode:
        ; Здесь находится точка CALL #8000.
        XOR     A
        RET
PluginEnd:
        ENT

        SAVEBIN "EXAMPLE.WMF",PluginFile,PluginEnd-PluginFile
```

Каждая непустая пара в `+36` загружает очередной блок файла в указанную относительную страницу, начиная с `#C000`. Для обычной полной страницы размер равен 32 секторам. Не допускайте перехода одного блока через `#FFFF`.

3.3 Распределение страниц

Заголовки находятся на физической странице `#60`, код и данные плагинов — в пуле `#61..#9F`. Поле `+34` резервирует страницы на всё время, пока плагин загружен; не загруженные из файла страницы можно использовать как постоянные буферы. Сумма страниц всех загруженных плагинов должна быть меньше 63, то есть фактический предел загрузчика — 62 страницы (992 КиБ). С `PLM.WMF` при старте загружаются только строки выше менеджера и резиденты `#06`; остальные лишь регистрируются, поэтому предел касается каждого плагина по отдельности, а не суммы всего набора.

Разбор `[PLUGINS]`: ядро вырезает комментарии от `;` до конца строки, пустые строки пропускает; секцию кончает только следующий заголовок `[` или конец файла. До 2026-09-25 секцию кончала первая пустая строка — в том числе строка из одного комментария, как `;CLOCK.WMF` у выключенного в WC Setup плагина, — и всё ниже неё молча не грузилось.

API 0/78/79 принимает относительный номер внутри выделенного блоку. Плагину не нужно знать фактическую физическую страницу.

Тип `#06` выполняется после инициализации интерфейса, но до восстановления путей панелей. Его 512-байтовый заголовок после вызова удаляется из списка меню; выделенные страницы остаются. Это позволяет установить резидентный провайдер, как делает `FILEX.WMF`.

3.4 Менеджер загрузки `PLM.WMF`: плагины живут на диске

Поставляемая сборка включает `PLM.WMF` — резидент `#06` в одну страницу. Он стоит второй строкой `[PLUGINS]`, сразу за обязательным `FILEX.WMF`, и меняет момент, когда плагин попадает в память:

- менеджер ставится сразу после своей загрузки (`+33` bit 0, вызов `A=#07`, раздел 2.2), и каждую следующую строку `[PLUGINS]` ядро только регистрирует: заголовок ложится в таблицу, первый кластер `.WMF` (поля `+1..+4`) берётся из поиска файла, `+0 = #FF` — не загружен. Код при старте не читается вовсе: со штатным набором (51 страница) WC в Unreal выходит в панели за 1,9 с вместо 3,1 с, в пуле 2 страницы, а сумма страниц набора больше не ограничена пулом — каждый плагин лишь должен помещаться в него сам;
- строки выше менеджера ядро загружает как прежде, и они остаются в пуле неподвижными; резидент `#06` ниже менеджера тоже загружается сразу и тоже становится неподвижным (после AX_RUN его заголовка в таблице уже нет, и уплотнение иначе заняло бы его страницы);
- со старым ядром, которое зовёт `#06` только из AX_RUN, менеджер устанавливается по-прежнему: все нерезидентные записи помечаются выгруженными, ватерлиния возвращается к вершине резидентов, а кластеры находятся по именам из `wc.ini`;
- при запуске плагин читается с диска в выделенные страницы и остаётся в пуле; повторный запуск идёт уже без чтения;
- если места не хватает, вытесняется самый давно запускавшийся нерезидент, после чего пул уплотняется переносом страниц через DMA, а базы в заголовках исправляются;
- плагин, который не помещается в пул целиком, просто не запускается: менеджер возвращает управление, ничего не испортив;
- `.WMF` читается в потоке каталога WC (страница `#F2`, в ней и запомнены кластеры), а прежний поток окна `#0000` потом возвращается: панель может стоять на другом диске или разделе, и её поток — в том числе файл, уже подготовленный WC для просмотрщика, — загрузка не трогает;
- ошибка чтения `.WMF` отменяет запуск: полузагруженные страницы не исполняются, запись снова «не загружена», страницы возвращаются пулу;

- шлюз запуска отдаёт плагину регистры так, как их подготовил WC: `DE` — старшее слово размера файла, `IX` — описатель панели. До 2026-09-25 менеджер портил `DE`, и редактор F4 на любой файл отвечал «Read ERROR»;
- после регистрации (каждая строка `[PLUGINS]` при старте, ручная загрузка `.WMF` по Enter) окно `#8000` возвращается коммандеру: перехват `WALK` лежит в его странице. До 2026-09-25 возврат попадал в код менеджера — ручная загрузка `.WMF` при установленном менеджере ломала WC.

Сам менеджер не меняет ни байта в `boot.$C` (ядро лишь зовёт его раньше, см. 2.2): он переписывает в RAM четыре известных куска кода WC — общий пролог запуска `L6011_241` (`#70B2`), цикл загрузки блоков `WALK` (`#97A4`), меню плагинов `MNU` (`#9908`) и пересчёт выбранной строки в запись (`#99AB`). Каждый кусок правится только после побайтовой сверки с эталоном, поэтому на чужом или более старом runtime менеджер молча не ставится, и WC работает по-прежнему.

Что это меняет для автора плагина:

- физическая страница нерезидента непостоянна. Между запусками его образ переносится уплотнением или выгружается совсем. Относительные номера (API 0/78/79) по-прежнему верны, а вычисленный когда-то физический номер `#61 + base` кэшировать нельзя;
- страницы, не загружаемые из файла, остаются зарезервированными, но переживают только пребывание в пуле. После вытеснения плагин читается с диска заново, и такой «постоянный» буфер содержит уже не те данные. Состояние, которое обязано жить между запусками, нужно хранить в файле;
- резидент `#06` не двигают и не вытесняют: он получает страницу один раз при старте, как и раньше;
- меню F10 показывает то же, что WC Setup относит к группам «F10» и «saver»: всё, что заявляет в `+197` запуск из F10, — заставки (`#02`) и типы `#03`, `#05`, `#11`, `#13`, `#15`, в порядке строк `[PLUGINS]` (с 2026-10-11; прежде — по алфавиту). Порядок задаёт WC Setup: F5/F6 двигают плагин в его группе, S — группу по алфавиту; новый порядок — после перезапуска WC. Ядро запускает их из меню с `A=#03` (`PSEXE`). Свои расширения (`+64`) не мешают: TR-DOS Mounter, TRDump, TRDispatcher и TXT Editor из F10 открывают свой интерфейс — подключение дисков, снятие образа дискеты, новый TRD, новый текст. Просмотрщик, которому без файла нечего показать, F10 не заявляет: FT812 Viewer — `#00`, Text/HEX Viewer и TXT/HEX VDAC2 — `#04`. Чтобы отключить лишний плагин, снимите его отметку в WC Setup (или уберите строку из `[PLUGINS]`): это переживёт перезапуск, в отличие от прежнего удаления плагина по `DEL`;
- меню вмещает все 32 записи таблицы WC (до 2026-09-25 с 17-го пункта имена затирали таблицу строк, и Enter мог взять чужую запись; полная таблица из 32 записей не показывалась вовсе). Перед запуском выбранного плагина окно меню убирается своим описателем — раньше после заставки оно оставалось на экране;
- окно меню больше не растёт по числу плагинов: его высота берётся из высоты текстового экрана (25, 30 или 36 строк), а список прокручивается стрелками, `PgUp` / `PgDn` и `Home` / `End`. Справа идёт полоса прокрутки, а в подвале окна вместо прежней подсказки про `DEL` — сколько памяти пула ещё свободно (`Free: 976K`). Удаление плагина из меню убрано: оно освобождало память на один сеанс, а памяти теперь и так хватает;
- связь «запись — файл» держится на первом кластере. Если `.WMF` переименовать или переписать при запущенном WC, плагин перестанет запускаться до следующего старта.

3.5 Настройки WC — обычный плагин `SETUP.WMF`

Диалога настроек по F9 в ядре больше нет (клавиша F9 свободна), и ядро само `wc.ini` не пишет: только читает при старте. Настройки правит `SETUP.WMF` — обычный плагин меню F10 «WC Setup» (тип `#03`, одна страница), третьей строкой `[PLUGINS]`, сразу за `PLM.WMF`.

- Если `wc.ini` нет, ядро вместо прежнего «WC.INI not found!» и старта без плагинов говорит «No settings: press F10 and run WC Setup.» и берёт встроенный минимум `FILEX.WMF`, `PLM.WMF`, `SETUP.WMF`. «WC Setup» в F10 открывает шаблон с заводскими значениями и сообщением «No wc.ini yet: F2 creates it».
- Первый экран — параметры `[SETUP]` и обеих панелей (`[LPANEL]`, `[RPANEL]`: устройство, строка информации, сортировка). Пробел или Enter перебирает значение, цифра правится прямо в тексте файла, поэтому комментарии и порядок строк не страдают. Ключ ищется только в своей секции. С 2026-10-11 экран оформлен по центру: колонка группы (Commander, Left panel, Right panel — название только в первой строке группы), параметр с точками до значения, поля слева и справа поровну; подсказка клавиш — по центру нижней рамки.
- Tab — второй экран: все `.WMF` каталога `WC` (до 250 файлов; включить можно не больше 32 — столько записей в таблице WC) по группам, внутри группы — в порядке строк `[PLUGINS]` (с 2026-10-11; без строки в файле — в конце группы, по названию). Название группы — только в первой её строке, между группами — тонкая линия, курсор её перешагивает; названия — до 31 знака, флаги длиннее колонки обрываются знаком `>`. Группы: Core — обязательные (по имени файла); Autostart — autoexec (`#06`); F10 menu — всё, что есть в меню F10 (`#03`, `#05`, `#11`, `#13`, `#15`), в том числе со своими расширениями (TR-DOS Mounter, TRDump, TRDispatcher, TXT Editor); Savers — заставки (`#02`); Files — только по файлу: по расширению (`#00`), меню просмотрщиков (`#04`), F2 (`#12`), F4 (`#14`). F5/F6 двигают плагин под курсором вверх/вниз в его группе — меняются местами строки `[PLUGINS]` (за границу группы не выходит; обязательные, автозапуск и заставки не двигаются); S — разовая сортировка групп F10 menu и Files по алфавиту названий. Порядок строк — это порядок меню F10 и обработчиков файлов после перезапуска WC. Плагин, имя которого стоит в `[PLUGINS]` не одной строкой (включённые и выключенные — в любом сочетании), SETUP при открытии исправляет: остаётся строка, флаги которой он показывает (первая включённая, иначе первая выключенная), прочие убираются вместе с переводом строки; в окне — «Duplicate lines in [PLUGINS] removed», текст правленый («*»: F2 запишет, Esc спросит). Иначе F5/F6 и S, переставив одну из строк, связали бы плагин с другой, с её флагами. Пустые строки, комментарии «; …» и строки с пробелом в начале не трогаются. Текст `wc.ini` — до 3871 байта (длиннее — «wc.ini is too long to edit»; поставляемый — 1687). До 2026-10-11 список вмещал 40 файлов и перебор каталога молча кончался на сороковом: у пользователя их 41, и последний в каталоге (недавно записанный) в WC Setup не попадал. Теперь у плагина две страницы (`+34` = 2): во второй, без данных из файла, — список и новый текст `wc.ini`; она подключена к `#C000` всё время работы, а после каждого вызова WC — снова (вызовы окон и файлов перекладывают это окно; тень `#6003` тоже её, по ней окно возвращает обработчик прерываний). Записей — 250 (байтовые счётчики строк с линиями групп); если `.WMF` больше, окно плагинов говорит «Not all .WMF fit in the list». FT812 Viewer с 2026-09-25 — тип `#00`, обычный просмотрщик: вызов из F10 лишь показывал окно с версией и убран; Text/HEX Viewer и TXT/HEX VDAC2 — `#04`: из F10 они только просили выбрать файл. Пробел ставит и снимает отметку «использовать»: снятый плагин остаётся строкой с `;` в начале вместе со своими флагами и вернётся с ними; новый дописывается сразу за последней строкой секции. Обязательные `FILEX.WMF` и `PLM.WMF` снять нельзя. Ядро такую строку пропускает и грузит всё, что ниже (раздел 3.3); до 2026-09-25 снятая отметка выключала и все плагины под ней — так у пользователя пропала заставка.

- Заставка у WC одна: отметка заставок — «(*)»/«( )», включённая выключает остальные (их строки получают `;`); снять можно и единственную.
- Enter на строке плагина правит его флаги прямо в строке списка штатным строковым редактором WC (API 9); подсказка после `;` в строке сохраняется. Поле вмещает 64 знака: флаги длиннее SETUP не правит и говорит «Flags are longer than 64 chars», такие правятся в самом `wc.ini`.
- F2 пишет файл через временный `wc.new`; прежний `wc.ini` сначала откладывается в `wc.old`, новый встаёт на имя `wc.ini`, и только потом `wc.old` удаляется. Хвосты прошлой неудачной записи (`wc.new`, `wc.old`) удаляются, только если они есть (с 2026-10-11; прежде DELFL звался всегда — лишний проход каталога), а неизменённый строкой версии текст не разбирается заново. Первая после загрузки запись прежде могла идти около минуты — поиск свободного кластера (docs/04, MKSG/SRHFCL); теперь — доли секунды. Если новый встать не смог, делается попытка вернуть `wc.old` на место и выводится «wc.ini was NOT saved» (если это переименование или возврат вернули «исход неизвестен» — «wc.ini: check the disk», см. ниже; обычно при двойном отказе остаются `wc.old` и `wc.new`, но при сбоях носителя внутри RENAME имена могут и пропасть — цепочки остаются потерянными кластерами, — или остаться две записи на одной цепочке; это возможно только с драйвером, сообщающим ошибки). Хвосты прошлой попытки (`wc.new`, `wc.old`) следующее сохранение удаляет, но хвост, чья запись начинается с того же кластера, что `wc.ini` (он делит с ней цепочку, и удаление освободило бы данные `wc.ini`), не трогает; не трогает и хвост, цепочку которого не удалось сверить из-за ошибки чтения каталога (с 2026-09-26; кластер — из TENTRY за FENTRY, размер для этого не годится: APPEND меняет размер лишь в одной из двух записей). Тогда сохранение отказывает с сообщением «wc.ini: check the disk» — нужна проверка тома. Его же SETUP выводит сразу, если какое-то из переименований сохранения вернуло «исход неизвестен» (A=#FF): на цепочке могут остаться две записи. Проверьте том до любой работы с `wc.ini`, `wc.new`, `wc.old`: если одну из двух записей переименовать, SETUP её уже не узнает, и удаление любой из двух (им самим при следующем сохранении или F8) освободит данные другой. «wc.ini saved» выводится только после успешного переименования и сверки длины. Первой строкой пишется версия WC из `source/VERSION.ASM` — по её префиксу `Wild Commander v1.1` ядро узнаёт свой файл.
- Если `wc.ini` не прочитался (ошибка носителя), WC Setup говорит «wc.ini could not be read» и F2 ничего не пишет: недостоверный текст не ляжет поверх исправного файла. Так же не пишется файл длиннее буфера. С 2026-09-27 так же и при ошибке чтения каталога `WC` (FENTRY: Z и CF=1): прежде она сходила за «файла нет» — шаблон, и F2 записал бы его поверх настоящего `wc.ini`.
- Правленый, но не записанный текст отмечен в заголовке окна: «*» перед названием, и всё название красное на белом, как «ERROR:» окна ошибки WC (с 2026-09-27). Признак ставят только настоящие правки текста — новое значение, вписанный ключ, отметка или флаги плагина, — и снимает только удачная запись: заголовок обычный уже под «wc.ini saved». Поле флагов при подтверждении убирает лишние пробелы (ведущие, хвостовые, второй пробел после имени): если они были, текст стал другим, и это тоже правка; флаги с одним пробелом после имени, подтверждённые без изменений, собирают тот же текст — признака нет. Значение, возвращённое перебором к прежнему, признак не снимает. Шаблон вместо ненайденного файла правкой не считается.
- Esc при правленом тексте спрашивает в том же окне «Save changed settings?» с кнопками WC `[ YES ]` / `[ NO ]` (выбрана Yes, стрелки — выбор). Yes — то же, что F2, но без «wc.ini saved»: записан — окно закрывается, не записан — сообщение об отказе, и список остаётся; No — выход без записи; Esc — назад к списку. Без правок Esc закрывает окно сразу.
- Ключ настройки считается только с начала строки, как его читает сам WC: закомментированное `;CPU_FREQ=0` и слова за `;` не в счёт. Если ключа в файле нет, при изменении он вписывается строкой сразу за заголовок своей секции, а если нет и секции — она дописывается в конец текста.

- Список длиннее окна прокручивается; справа на рамке — полоса прокрутки, как в меню F10. Полоса курсора переносится сразу при шаге, поэтому видна и при автоповторе стрелок. Окно не выше экрана без шести строк (строк списка — до высоты экрана минус 8): под заголовком WC остаются две строки, под окном — тень и ещё две (с 2026-10-11; прежде окно во всю высоту вставало вплотную под заголовок WC).
- Всё состояние плагина — текст `wc.ini` в памяти (до 3871 байта с 2026-10-11; прежде 2559, ещё раньше 2751): отметки, флаги и значения вписываются в него сразу, поэтому на экране всегда то, что запишет F2. Сообщения показываются в том же окне, по центру. Если на экране плагинов меньше двух строк (в каталоге `WC` нашёлся один `.WMF` или ни одного — каталог не читается), сообщение и вопрос показываются на экране настроек: под текстом нужна строка для подсказки или кнопок, а при пустом списке прежний счёт строк уходил за окно. На пустом списке пробел и Enter ничего не делают. После выхода WC перечитывает обе панели: у пересохранённого `wc.ini` новые кластер и дата.

4. Резидентные адреса и страницы

Адрес	Назначение
`#6000..#6003`	Тени физических страниц окон `#0000/#4000/#8000/#C000`
`#6004`	Флаг Esc менеджера плагина; это не `ABT` файлового ядра
`#6005`	Флаг Enter
`#6006`	Публичный диспетчер API
`#6009..#600A`	Кадровый таймер
`#600D`	Тип/версия видеоконфигурации
`#600E`	Высота текстового экрана: 25, 30 или 36
`#6020`	Переключить страницу плагина и выполнить `JP (HL)`
`#6028`	Переключить страницу, выполнить `CALL (HL)` и восстановить страницу

Вызов кода другой страницы:

```
        LD      A,2                     ; относительная страница плагина
        LD      HL,#8120                ; процедура собрана для окна #8000
        CALL    #6028
        ; #6028 сохраняет все регистры, кроме A и HL.
```

`#6020` — безвозвратный межстраничный переход. Возвращаться после него можно только по протоколу самого плагина. Для обычного вызова используйте `#6028`.

5. Окно SOW

API интерфейса принимает `IX` на структуру окна. Она должна находиться в постоянно доступной памяти плагина: `#8000..#BFFF` или `#0000..#3FFF`.

Offset	Размер	Назначение
`+0`	1	bit 7 — тень; bit 6 — курсор во всю ширину; bit 4 — инверсный заголовок; bits 3..0 — тип рамки/содержимого
`+1`	1	Маска цвета курсора
`+2/+3`	1+1	X/Y; `255` означает центрирование и заменяется вычисленным значением
`+4/+5`	1+1	Ширина/высота; `0` означает весь экран и заменяется значением
`+6`	1	Цвет paper+ink
`+7`	1	Обязательно ноль
`+8`	2	Буфер фона: `#FFFF` — не сохранять; `#0000` — менеджер назначит при выводе
`+10/+11`	1+1	Разделители, смещение от нижнего края; ноль отключает
`+12`	1	Позиция курсора от 1 для меню
`+13`	1	Нижний ограничитель меню
`+14/+15`	1+1	Цвет курсора и цвет под ним

Низкие четыре бита `+0`:

Тип	Содержимое
`0/3/6`	Обычное окно, три варианта рамки
`1/4/7`	Заголовки и текст без курсора
`2/5/8`	Заголовки и текст с курсором

Для типов `1/4/7` слова `+12`, `+14`, `+16` — указатели верхнего заголовка, нижнего заголовка и текста. Для `2/5/8` те же указатели лежат в `+16`, `+18`, `+20`, потому что `+12..+15` заняты курсором.

Правильная пара показа и удаления:

```
        LD      IX,Dialog
        WC_CALL 1                   ; PRWOW
        ; ... цикл диалога ...
        LD      IX,Dialog
        WC_CALL 2                   ; RRESB
```

Не копируйте SOW после `PRWOW`: центрированные координаты, размеры и адрес буфера уже могли быть изменены менеджером.

5.1 Разметка текста

API 11/12 и текстовые поля SOW понимают управляющие байты:

Последовательность	Значение
`#00`	Конец строки/абзаца
`#00,#00`	Разделитель сообщений для MEZZ
`#FE,low,high`	Ссылка на другую строку

Последовательность	Значение
`#0B,x`	Сдвиг вправо на `x`
`#0C,y`	Сдвиг вниз на `y` без сброса X/цвета
`#0D`	Новая строка
`#0E`	Центрировать следующую строку
`#09`	Поменять paper и ink местами
`#01..#08`	Цвет ink
две команды `#01..#08`	Последовательно задают paper и ink

5.2 Какие вызовы образуют интерфейс плагина

Интерфейс — не одна процедура, а небольшой набор слоёв:

```
API 15 GEDPL                        привести экран к основному TXT
      │
      ▼
API 1 PRWOW                        показать SOW и сохранить фон
      │
      ├─ API 3/4                    простой текст и его цвет
      ├─ API 11/12                  размеченный текст/сообщения
      ├─ API 5                      получить экранный адрес
      ├─ API 6/7                    нарисовать/стереть курсор
      ├─ API 8/9/10                 вопрос, редактор строки, HEX
      └─ API 16..47,85              клавиатура и ожидание
      │
      ▼
API 2 RRESB                        убрать окно и восстановить фон
```

Задача интерфейса	Вызовы
Перейти в интерфейс WC	API 15 `GEDPL`
Показать и закрыть окно	API 1 `PRWOW`, API 2 `RRESB`
Печатать и красить точный диапазон	API 3 `PRSRW`, затем API 4 `PRIAT`
Печатать строки с разметкой	API 11 `TXTPR`, API 12 `MEZZ`
Управлять курсором окна	API 5 `GADRW`, 6 `CURSOR`, 7 `CURSER`
Получить подтверждение/строку/HEX	API 8 `YN`, 9 `ISTR`, 10 `NORK`
Опросить клавиши	API 16–47 и 85
Переключить страницу плагина	API 0, 78, 79 или резидент `#6028`
Работать со своим видео	API 64–71, 80, 81
Узнать параметры `wc.ini`	API 83 `PRM_PL`
Восстановить стандартные цвета	API 84 `DEFPRM`
Получить текущую конфигурацию видео	API 86 `GVCONFIG`

Полный минимальный цикл модального окна:

```
WLD     EQU #6006

ShowDialog:
        LD      A,15                    ; GEDPL: основной текстовый интерфейс
        CALL    WLD

        LD      IX,Dialog
        LD      A,1                     ; PRWOW
        CALL    WLD

.wait:
        EI
        HALT                            ; один опрос на кадр

        LD      A,22                    ; Enter
        CALL    WLD
        JR      NZ,.accepted

        LD      A,23                    ; Esc
        CALL    WLD
        JR      Z,.wait
        XOR     A                       ; результат диалога: отмена
        JR      .close

.accepted:
        LD      A,1                     ; результат диалога: принято
.close:
        LD      (DialogResult),A
        LD      IX,Dialog
        LD      A,2                     ; RRESB
        CALL    WLD
        LD      A,(DialogResult)
        RET

DialogResult:
        DB      0

Dialog:
        DB      4+16+128,0              ; рамка 2, inverse title, shadow
        DB      255,255                 ; центрировать X/Y
        DB      42,7                    ; width/height
        DB      %01011111,0             ; цвет, обязательный ноль
        DW      #0000                   ; PRWOW назначит буфер фона
        DB      0,0                     ; без разделителей
        DW      DialogTitle
        DW      DialogBottom
        DW      DialogText

DialogTitle:
        DB      #0E,#09,"EXAMPLE",0
DialogBottom:
        DB      #0E,"ENTER - OK, ESC - CANCEL",0
DialogText:
        DB      #0B,2,"Plugin interface is active",0
```

`PRWOW` изменяет вычисляемые поля SOW, поэтому структура должна лежать в доступной до `RRESB` памяти. Не переключайте страницу `#8000`, содержащую `Dialog`, пока окно открыто. Для меню с курсором используйте тип SOW `2/5/8` и вызывайте API 6 каждый кадр, а API 7 перед окончательным удалением курсора.

6. Полная таблица API 0–86

6.1 Базовые страницы и интерфейс: 0–15

API dec/hex	Имя	Вход	Выход и замечания
`0/#00`	`MNGC_PL`	`A'` = страница плагина `0..63`; `#FE` = основной TXT; `#FF` = font	Отображает её в `#C000`; значения `>=64`, кроме специальных, игнорируются
`1/#01`	`PRWOW`	`IX` → SOW	Рисует окно, включает TXT в `#C000` и оставляет его включённым
`2/#02`	`RRESB`	`IX` → ранее показанная SOW	Восстанавливает фон, TXT остаётся в `#C000`
`3/#03`	`PRSRW`	`IX`, `HL` → байты, `D=Y`, `E=X`, `BC=длина`	Печатает строку; SOW и текст только в постоянных окнах
`4/#04`	`PRIAT`	Сразу после PRSRW; `A'` = цвет	Окрашивает только что выведенный диапазон
`5/#05`	`GADRW`	`IX`, `D=Y`, `E=X`	`HL` = экранный адрес внутри окна
`6/#06`	`CURSOR`	`IX` → SOW	Выполняет фазу мигания/показа курсора
`7/#07`	`CURSER`	`IX` → SOW	Стирает курсор, восстанавливая фон
`8/#08`	`YN`	`A'`: `1` OK/Cancel, `2` Yes/No, `0` обработка кадра, `#FF` завершение	При `#FF`: `Z` = OK/Yes, `NZ` = Cancel/No
`9/#09`	`ISTR`	Подкоманда в `A'`, остальные параметры ниже	Строковый редактор; состояние глобально, один редактор за раз
`10/#0A`	`NORK`	`HL` → место, `A'` = байт	Записывает две HEX-цифры, возвращает `HL` за ними
`11/#0B`	`TXTPR`	`IX`, `HL` → размеченный текст, `D/E=Y/X`	Возвращает `D` следующей строки, `E` без изменения
`12/#0C`	`MEZZ`	Как TXTPR, `A'` = номер сообщения	Выбирает абзац между двойными нулями
`13/#0D`	`DMAPL`	Подкоманда `A'`, параметры DMA	Настройка и запуск DMA, см. ниже
`14/#0E`	`TURBOPL`	`B=0` Z80, `B=1` AY, `B=#FF` восстановить; `C` = режим	Меняет частоту/кэш по политике WC
`15/#0F`	`GEDPL`	Нет	Восстанавливает TXT, палитру и offsets; уничтожает `#1000..#1447`

Подкоманды API 9 `ISTR`:

- `A'=#FF`: редактор типа 0; `HL` — экранная строка, `D=CURMAX`, `E=CURNOW`;
- `A'=0`: один кадр обработки клавиатуры;
- `A'=1`: завершить, стереть курсор, вернуть адрес строки в `HL`;
- `A'=#FD`: редактор типа 2; `HL` — буфер, `HL'` — его размер, `D/E` — Y/X, `B=CURMAX`, `C=CURNOW`;
- `A'=2`: вывести буфер типа 2, вызывается каждый кадр вместе с `0`;
- `A'=#FE`: инициализировать индикаторы INS/OVR, ENG/RUS, CAPS; `H` выбирает верхний/нижний заголовок, `L=#FF`;
- `A'=3`: обновить индикаторы;
- `A'=#FC`: опросить CAPS/INS/CTRL+SHIFT; нельзя в тот же кадр с подкомандой 0.

Подкоманды API 13 `DMAPL`:

A'	Операция
#00	Source= B:HL, destination= C:DE
#01/#02	Установить только source / destination
#03/#04	Source / destination из текущего окна: HL и B=0..3
#05	B → DMA_T: значение 0 означает один burst
#06	B → DMA_N: 255 соответствует 512 байтам
#07	Одновременно DMA_T= B, DMA_N= C
#08/#09	SourceAlign / DestinationAlign = C
#0A	AlignSize = C: 0 для 16c, 1 для 256c
#0B	E AlignSize, H SourceAlign, L DestinationAlign
#FC	RAM→RAM с ожиданием
#FB	BLT→RAM с ожиданием
#FA	FILL с ожиданием
#F9	RAM→CRAM с ожиданием
#F8	RAM→SFILE с ожиданием
#FD	RAM→RAM без ожидания, align=0
#FE	RAM→RAM с ожиданием, align=0
#FF	Ждать готовности DMA

Для #00..#02 bit 7 номера страницы выбирает страницу плагина, а сброшенный bit 7 — видеостраницу; bits 6..0 содержат номер.

6.2 Клавиатура: 16–47 и 85

API 16–45 возвращают NZ, когда соответствующее состояние обнаружено.

API dec/hex	Клавиша/функция
16/#10	Space
17/#11	Up
18/#12	Down
19/#13	Left
20/#14	Right
21/#15	Tab
22/#16	Enter
23/#17	Esc
24/#18	Backspace
25/#19	Page Up
26/#1A	Page Down

API dec/hex	Клавиша/функция
27/#1B	Home
28/#1C	End
29/#1D	F1
30/#1E	F2
31/#1F	F3
32/#20	F4
33/#21	F5
34/#22	F6
35/#23	F7
36/#24	F8
37/#25	F9
38/#26	F10
39/#27	Alt, без изменения авторепита
40/#28	Shift, без изменения авторепита
41/#29	Ctrl, без изменения авторепита
42/#2A	`KBSCN`: `A'=0` с раскладкой/modifiers, `A'=1` сырой основной стол; при `NZ` символ в `A`
43/#2B	Delete
44/#2C	Caps Lock
45/#2D	Любая клавиша
46/#2E	Ждать отпускания всех клавиш; клавиша, «державшаяся» ~1,5 с без повтора автоповтора, считается отпущенной — потерянное отпускание не вешает WC
47/#2F	Ждать нажатия любой клавиши
85/#55	Insert

`KBSCN` с режимом 0 вызывается не более одного раза за кадр. При `Z`, `A=0` означает неизвестную/отсутствующую клавишу; `A=#FF` — конец буфера.

Типовой цикл диалога:

```
.loop:
        EI
        HALT
        LD      A,6                 ; CURSOR
        CALL    WLD
        LD      A,23                ; Esc
        CALL    WLD
        JR      Z,.loop
```

6.3 Файлы, каталоги и потоки: 48–63, 72–77

Эти функции сами подключают рабочую файловую страницу. Плагину не следует вызывать `#40xx` или вручную менять `#600B/#600C`.

API dec/hex	Имя	Контракт
48/#30	LOAD512	HL = буфер, B = блоки по 512; возвращает новый HL, A=0/#0F/#FF, при ошибке CF=1
49/#31	SAVE512	Как LOAD512, но пишет существующую цепочку; не расширяет файл и не меняет размер
50/#32	GIPAGPL	Позиционирует поток на начало файла, переданного менеджером
51/#33	TENTRY	DE → 32-байтовый буфер; копирует текущую FAT ENTRY
52/#34	CHTOSEP	DE = начало массива LBA32, BC = исключающий конец
53/#35	reserved	Пустой RET
54/#36	TMRKFL	IX = панель, HL = номер 1..2047, DE = буфер имени #8000..#BFFF; HL=0 считает отмеченные в BC
55/#37	reserved	Пустой RET
56/#38	ADIR	A'=0 позиционировать начало активного каталога; A'=1 сбросить FindNext
57/#39	STREAM	Создание, клонирование и переключение потоков, см. ниже
58/#3A	NXTETY2	DE = выход, маска в A'; NZ = запись, Z = конец
59/#3B	FENTRY	HL → [тип,имя,0]; NZ найдено, размер в DE:HL; Z не найдено
60/#3C	LOAD256	Потоковое чтение через драйверный 256-байтовый вход; не каждый драйвер поддерживает. С 2026-10-10 буфер — от #4000, B — 1..64, последний байт HL+1024·(B−1)+767 — не за #FFFF; иначе A=#FF, CF=1 без обмена
61/#3D	LOADNONE	Линейно пропустить B секторов, параметры как LOAD512
62/#3E	GFILE	После успешного FENTRY выбрать найденный файл и поставить поток в начало. С 2026-10-10: пустой файл (кластер 0) — поток сразу в конце (A=#0F, NZ, CF=0), а не на корне; файл на кластере корня — отказ (CF=1, ABT=#FE)
63/#3F	GDIR	После FENTRY сделать найденный каталог активным; не использовать в исходном потоке панели
72/#48	MKFILE	HL → [attr,size_le32,name,0]; Z создан, NZ ошибка, код в A
73/#49	MKDIR	HL → [name,0]; Z создан, NZ ошибка
74/#4A	RENAME	HL → [тип,старое,0], DE → [новое,0]; NZ успех
75/#4B	DELFL	HL → [тип,имя,0]; исторически NZ означает удалено
76/#4C	APPEND	После FENTRY: HL = данные в #4000..#BFFF, BC=0..#4000; Z/A=0 успех
77/#4D	FILEX	HL → 32-байтовый блок в #8000..#BFFF; позиционный расширенный API

Формат имени для поиска/удаления/переименования:

```
FileName:
        DB      #00,"LOG.TXT",0       ; #00 файл
DirName:
        DB      #10,"GAMES",0         ; #10 каталог
```

Маска API 58:

- bit 7 — только короткие имена;
- bit 6 — в полях битов 4 и 3 время и дата изменения (как в панели WC и её сортировке по дате), а не создания. Ядро до 2026-09-26 бит не знает и отдаёт создание; раскладка вывода одна и та же;
- bit 4 — добавить время (создания или, с битом 6, изменения);

- bit 3 — добавить дату (создания или, с битом 6, изменения);
- bit 2 — добавить size;
- bits 1..0: 0 любой объект, 1 файл, 2 каталог.

При `NZ` API 58 последовательно пишет включённые поля (размер 4 байта, дата 2, время 2), затем байт типа записи, имя длиной 1..255 и ноль. Она использует `#0000..#1FFF` файловой страницы и теряет позицию после MKFILE/MKDIR/LOAD512/SAVE512/FENTRY. После таких операций перезапустите перечисление через API 56, подкоманду 1.

Штамп (время и дата) записи на носителе Evo, которого нет в календаре, API 58 отдаёт как 2026-01-01 00:00 — и дату, и время, даже если запрошено одно поле. Нет в календаре: месяц не 1..12, день 0 или больше длины месяца (февраль года, кратного 4, — 29 дней; 2100-й тоже считается високосным), часы больше 23, минуты или секунды больше 59. Год FAT (1980..2107) принимается любой. Сама запись на носителе не меняется, панель WC показывает её как есть. Так же нормализует штампы только FILEX GET_METADATA; API 51 TENTRY и прямые входы ядра NXTETY `#4057` и TENTRY `#4036` отдают запись как на носителе, а GENTRY `#4033` переносит шаблон вызывающего в ENTRY тоже без проверки.

Время записи каталога ставит WildDOS, плагину ничего делать не нужно:

- MKFILE и MKDIR пишут время создания, изменения и доступа из часов CMOS. Часы считаются сбитыми, если поле не в BCD, вне диапазона, такого дня нет в месяце (31.02, 29.02 невисокосного года) или год раньше 2026; тогда ставится 2026-01-01 00:00. Поля читаются двумя снимками подряд, пока снимки не совпадут (не больше четырёх пар; не совпали — часы тоже считаются сбитыми, 2026-01-01): шаг часов посреди чтения не смешает старую дату с новым временем.
- Копия получает время оригинала: штамп изменения и штамп создания (с долями секунды, если их не больше 199) — каждый, только если он есть в календаре (те же правила, что у API 58); штамп, которого в календаре нет, у копии текущий по часам. Дата доступа копии — текущая: копию только что записали (до 2026-09-26 она тоже бралась у оригинала). Копией считается объект, который создаётся в другом потоке (другой панели) из того же буфера имени, по которому только что нашли источник через FENTRY, того же типа и с тем же именем — имя сверяется целиком. Так копирует сам WC (F5, в том числе каталоги); плагин, который копирует так же, получает то же поведение.
- APPEND обновляет время и дату изменения и дату доступа. FILEX делает то же при записи внутри файла (WRITE_AT) и при усечении (SET_EOF32).
- SAVE512 запись каталога не трогает: перезапись существующей цепочки время изменения не меняет.
- RENAME переносит запись целиком, со всеми временами и атрибутами. Если новая запись создалась, а старую удалить не вышло, старая добивается по месту (до трёх записей, с 2026-10-10), и переименование доводится; не вышло и это — новая убирается: на одной цепочке кластеров двух записей не остаётся (если не откажет и сам откат — тогда Z и `A=#FF`, обе записи могут остаться). Атрибут новая запись получает сразу целиком (с 2026-10-10; до 2026-09-25 он терялся, затем вписывался отдельной записью сектора после переименования). `.` и `..` не переименовываются и не удаляются.
- MKFILE (API 72) записывает переданный атрибут целиком: «только чтение», «скрытый», «системный», «архив» (`#27`, с 2026-10-10); бит каталога отбрасывается — каталог создаёт API 73. Копия F5 получает атрибут 0: F5 передаёт MKFILE ноль, атрибуты оригинала на копию не переносятся (время переносится).
- APPEND (API 76) к файлу «только для чтения» отказывает кодом `#28`.

- Прямой вход ядра GENTRY (`#4033`) по-прежнему только переносит 32 байта записи. Время ставит обёртка MKFILE/MKDIR самого WC, поэтому плагин, который зовёт GENTRY напрямую, получает прежнее поведение.

Подкоманды API 57 `STREAM`:

D	Действие
`#FF`	Сделать root активным; допустимо только после клонирования
`#FE`	Клонировать поток панели в 0 и 1, выбрать поток 0
`#FD`	Выбрать служебный INI-поток и каталог `WC`
`0/1`	При `BC=#FFFF` выбрать существующий поток; иначе создать: `B` устройство, `C` раздел

Для создания потока `B`: 0 SD-ZC, 1 Nemo master, 2 Nemo slave, 4 SMUC master, 5 SMUC slave; поле раздела исторически не учитывается. Возврат `NZ` означает, что устройство/раздел не найден. Для `D=#FD` код `A=1` — устройство не найдено, `A=2` — нет каталога `WC`.

6.4 Видео: 64–71, 80–81

API dec/hex	Имя	Вход/действие
`64/#40`	`MNGV_PL`	`A'=0` основной TXT; `1..4` выбрать банк видеобуфера `#20/#30/#40/#50`
`65/#41`	`MNGCVPL`	`A'=0..63` отобразить видеостраницу в `#C000`
`66/#42`	`GVmod`	`A'` : bits 7..6 разрешение, bits 1..0 ZX/16c/256c/TXT
`67/#43`	`GYoff`	`HL=0..511` , вертикальное смещение
`68/#44`	`GXoff`	`HL=0..511` , горизонтальное смещение
`69/#45`	`GVtm`	`C=0..63` , страница TileMap
`70/#46`	`GVtl`	`B bit0` = плоскость 0/1, `C=0..63` , TileGraphics
`71/#47`	`GVsgp`	`C=0..63` , SpriteGraphics
`80/#50`	`MNG0VPL`	`A'=0..63` , видеостраница в `#0000`
`81/#51`	`MNG8VPL`	`A'=0..63` , видеостраница в `#8000`

Код режима API 66:

- bits 7..6: `00` 256×192, `01` 320×200, `10` 320×240, `11` 360×288;
- bit 5 `nogfx` , bit 4 `notsu` ;
- bits 1..0: `00` ZX, `01` 16 цветов, `10` 256 цветов, `11` TXT.

API 80 заменяет окно `#0000` , где файловая страница использует `#2000..#3FFF` . Пока оно переключено, нельзя передавать файловому API ни структуры, ни буферы из `#0000..#7FFF` . Публичные файловые обёртки временно переключают страницы, но адрес данных при этом не начинает указывать на прежнюю память.

6.5 Поздние служебные функции: 78–86

API dec/hex	Имя	Контракт
`78/#4E`	`MNG0_PL`	`A'=0..63`, страница плагина в `#0000`
`79/#4F`	`MNG8_PL`	`A'=0..63`, страница плагина в `#8000`
`80/#50`	`MNG0VPL`	Видеостраница в `#0000`
`81/#51`	`MNG8VPL`	Видеостраница в `#8000`
`82/#52`	reserved	Пустой `RET`
`83/#53`	`PRM_PL`	`A'=1..8`; `NZ` и `A=1..10` — выбранная опция, `Z` — параметра нет
`84/#54`	`SCRLWOW`	`IX`, `D/E=Y/X`, `B/C=height/width`, режим в `A'`
`85/#55`	`INS_KEY`	`NZ` при Insert
`86/#56`	`INT_PL`	Настроить подавление INT или обработчик плагина

API 84, bits `A'`:

- bit 7 — переносить атрибуты вместе с символами;
- bit 6 — очищать освободившийся край;
- bits 5..2 — шаг 1..16 символов;
- bits 1..0: 0 вниз, 1 вверх; вправо/влево в текущей реализации отключены.

API 86:

- `A'=0` — отключить все INT-функции WC на время работы плагина;
- `A'=1` — отключить обновление часов;
- `A'=2` — отключить опрос PS/2;
- `A'=#FF`, `HL=#8000..#BFFE` — зарегистрировать обработчик плагина.

Все настройки API 86 восстанавливаются при выходе из плагина. Обработчик должен находиться на странице плагина, оставаться отображаемым по контракту WC, быть коротким и не вызывать нереентерабельный файловый API.

7. Готовые рецепты

7.1 Найти и прочитать файл

```
        LD      HL,Query
        LD      A,59                    ; FENTRY
        CALL    WLD
        JR      Z,.not_found

        LD      A,62                    ; GFILE: поток в начало найденного файла
        CALL    WLD

        LD      HL,Buffer
        LD      B,1                     ; один сектор
        LD      A,48                    ; LOAD512
        CALL    WLD
        JR      C,.io_error
        ; A=#0F означает физический конец FAT-цепочки.
        ; Логический объём полезных байтов берётся из размера DE:HL FENTRY.
        RET

Query:  DB      #00,"DATA.BIN",0
Buffer: DS      512
```

Не читайте файл «до `#0F`» без учёта размера: последний кластер обычно содержит padding после логического EOF.

7.2 Перечислить каталог, не ломая поток панели

```
        LD      D,#FE                   ; клонировать поток панели
        LD      A,57
        CALL    WLD

        LD      A,1                     ; reset FindNext активного каталога
        EX      AF,AF'
        LD      A,56
        CALL    WLD

.next:
        LD      DE,EntryBuffer
        LD      A,%00000100             ; размер + любой тип
        EX      AF,AF'
        LD      A,58
        CALL    WLD
        JR      Z,.done
        ; Разобрать [size32][flag][name,0].
        JR      .next
.done:
```

7.3 Создать файл заданного размера

```
CreateBlock:
        DB      #00                     ; атрибуты
        DD      0                       ; пустой файл, cluster=0
        DB      "NEW.TXT",0

        LD      HL,CreateBlock
        LD      A,72
        CALL    WLD
        JR      NZ,.create_error
```

Для непустого файла API 72 заранее создаёт цепочку нужного размера, но не записывает пользовательские данные. API 49 записывает только уже существующую цепочку. Для безопасного роста существующего файла используйте API 76 или 77.

7.4 Дописать данные через APPEND

```
        LD      HL,Target
        LD      A,59                    ; обязательно свежий успешный FENTRY
        CALL    WLD
        JR      Z,.not_found

        LD      HL,Data
        LD      BC,DataEnd-Data         ; 0..#4000
        LD      A,76
        CALL    WLD
        JR      NZ,.append_error        ; A = код
        ; Z и A=0: данные и новый размер опубликованы.

Target: DB      #00,"LOG.TXT",0
Data:   DB      "line",13,10
DataEnd:
```

Весь `[HL,HL+BC)` должен лежать в `#4000..#C000`. Неуспешный FENTRY не очищает старый APPEND-контекст, поэтому игнорировать его `Z` опасно. `BC=0` — только пустая операция, не проверка контекста.

7.5 Позиционное чтение через FILEX

Сначала нужен успешный API 59 для файла. Затем:

```
        LD      HL,FilexBlock
        LD      A,77
        CALL    WLD
        JR      Z,.complete
        CP      #01                     ; EOF может содержать прочитанный префикс
        JR      Z,.partial_or_eof
        JR      .filex_error

FilexBlock:
        DB      32,1                    ; size, version
        DB      1,0                     ; READ_AT, flags
        DD      #00123456               ; offset LE32
        DW      Buffer
        DW      512
        DW      0,0                     ; aux, aux_length
        DD      0,0                     ; source_dir, dest_dir
        DD      0                       ; result_count
        DB      0,0                     ; status, result_flags
        DW      0                       ; reserved
Buffer: DS      512
```

Блок и все его буферы должны целиком лежать в `#8000..#BFFF`, не пересекаться между собой и не выходить за `#C000`. Перед каждым вызовом блок лучше полностью занулять, затем заполнять используемые поля. Полный контракт FILEX находится в руководстве DOS.

Последовательные вызовы `READ_AT` / `WRITE_AT` одного файла (с 2026-10-11) дешёвые на любой глубине: FILEX ищет кластер по смещению от последнего найденного, а сектор FAT читает один раз на 128 ссылок. Смещение раньше прежнего, запись FAT, монтирование или чтение каталога между вызовами — поиск снова от первого кластера. Прежде каждый вызов шёл по цепочке от начала файла и читал сектор FAT на каждую ссылку.

7.6 Попросить менеджер обновить панель

Если плагин изменил содержимое каталога и завершил все операции:

```
        LD      A,3
        RET
```

Не пытайтесь вручную править внутренний массив панели: перестроение перечитает каталог, пересортирует индекс и согласует курсор.

8. Параметры плагина в `wc.ini`

В `+260` лежит последовательность записей:

```
[record_length]
[default_option]
[parameter_name in lowercase]
[option_count]
[option_1,0][option_2,0]...[option_n,0]
```

Нулевой `record_length` завершает таблицу. Загрузчик распознаёт до восьми параметров плагина, имя передаётся с дефисом, например:

```
[PLUGINS]
MYPLUGIN.WMF -loading=fast -cache=off
```

API 83 получает выбранную опцию по индексу параметра `1..8`; номер опции начинается с 1. Если параметр не указан в `wc.ini`, возвращается default.

9. Ограничения, которые должен учитывать плагин

- API не реентерабелен. Не запускайте файловую операцию из INT-обработчика или внутри другой файловой операции.
- `IX` активной панели — входной объект менеджера. Не храните его как вечный указатель после возврата в WC и не меняйте внутренние поля без изменения самого менеджера.
- API 48/49 не должны получать блок, который переходит через `#FFFF→#0000`; разбивайте такой диапазон на два вызова.
- `SAVE512` не увеличивает файл. Для роста есть APPEND и FILEX.
- API 75 не защищает от удаления непустого каталога. Сначала перечислите его.
- `LOAD256` не реализован драйвером SD-NGS.
- `GEDPL` разрушает `#1000..#1447` подключённой страницы `#0000`.
- Структуры FILEX допускаются только в `#8000..#BFFF`.
- Со штатным `PLM.WMF` страницы нерезидентного плагина не переживают его вытеснение из пула: после него плагин читается с диска заново. Данные, нужные между запусками, храните в файле, а физический номер страницы не запоминайте (см. 3.4).
- Исторические `wc_api.h/wc_h.h` содержат неверные поздние номера и устаревшие подписи. Числа следует брать из таблицы `PLUGIN_API_TABLE` в `MD20.ASM`, а FILEX-константы — из `CORE32_FILEX_API.ASM`.

10. Где смотреть рабочие примеры

- `source/plugins/PLUG01.ASM`, `PLUG02.ASM`, `PLUG03.ASM` — простые окна и вызовы интерфейса; их старые заголовки полезны как каркас, но версия формата там историческая.
- `source/plugins/txt_editor/TXTEDIT.ASM` — строковый ввод и текстовый UI.
- `source/plugins/bmpview/PLUGBP.ASM` — видеостраницы и графика.
- `source/plugins/multigigascreen/MCGSV.ASM` — переключение окон видеопамяти.
- `source/plugins/filex/FILEX.ASM` — корректный 512-байтовый заголовок и одноразовый тип `#06`.
- `source/plugins/plm/PLM.ASM` — менеджер загрузки: перехваты в RAM, работа из собственной страницы в окне `#8000` и перенос страниц пула через DMA.
- `source/plugins/READ_ME.TXT` — исходное историческое описание ABI.

Драйверы накопителей: ABI и разработка

Драйвер WildDOS — это сменный низкоуровневый модуль блочного устройства. Он не знает имён файлов и FAT32: его задача — выбрать устройство, принять LBA и переслать один или несколько блоков. CORE32 выполняет всё остальное.

1. Место драйвера в стеке

```
плагин / панель
       │
       │ API #6006
       ▼
файловые обёртки WC
       │
       ▼
CORE32: FAT32, каталоги, цепочки, потоки
       │
       │ XPOZI + RDDSE/SDDSE
       ▼
driver runtime #3800
       │
       ├─ IDE Nemo
       ├─ SD Z-Controller
       ├─ SD через NGS
       └─ IDE SMUC
```

Плагин не вызывает `#3800` напрямую. Драйверный ABI предназначен для CORE32 и кода менеджера, который устанавливает драйвер через `DOS_SWP`.

2. Физическая модель

```
логическое окно Z80 #0000..#3FFF

#0000 ┌──────────────────────────────┐
      │ каталожные буферы            │
#2000 ├──────────────────────────────┤
      │ рабочая область CORE32       │
#3000 ├──────────────────────────────┤
      │ FAT + секторные буферы       │
#3800 ├──────────────────────────────┤
      │ JP-таблица драйвера          │  72 байта
#3848 ├──────────────────────────────┤
      │ общее состояние CORE32       │
      │ + код/данные драйвера        │
#4000 └──────────────────────────────┘
```

Для левой панели это окно обычно отображает физическую страницу `#F0`, для правой — `#F1`. Переключение панели тем самым переключает и драйвер, и весь его контекст.

Упакованные драйверы собраны внутри CORE32 (DR1..DR4, `#5176..#5AA9` образа), но с 2026-09-27 установщик переносит их в страницу `#E8` (`DRIVERS_E8` расширения), а их место в странице `#05` занимает резидентный блок расширения (docs/04, раздел 2). `DOS_SWP` распаковывает выбранный образ

оттуда (через шлюз, ID_UNPACK_DRIVER) по `#3800`; старый драйвер и его состояние полностью заменяются. Распакованный драйвер по `#3800` побайтно тот же, что прежде: тест `test_boot_resources.DriverSwap` сверяет все выборы A=0..6 с распаковкой прежних байт DRx из файла.

3. Таблица входов `#3800`

Таблица занимает 24 слота по три байта. Первые восемь определены, остальные 16 исторически были резервом. В SD-ZC часть резерва используется короткими шлюзами к CORE32_EXT, но внешние номера первых восьми не меняются.

Адрес	Offset	Имя	Вход	Результат
`#3800`	`+0`	`SELDEV`	`A` — вариант устройства	Выбирает master/slave или интерфейс; специфично для драйвера
`#3803`	`+3`	`DEVINI`	Нет общего параметра	`Z` — устройство готово, `NZ` — не найдено/ошибка
`#3806`	`+6`	`REINI`	Нет	Повторно применить выбор/состояние; может быть пустым `RET`
`#3809`	`+9`	`XPOZI`	`DE:HL` — LBA32	Сохранить общую и приватную позицию
`#380C`	`+12`	`PROZ`	`DE:HL` — LBA32	Обновить приватную позицию, не меняя общие `LTHL/LTDE`
`#380F`	`+15`	`RDDSE`	`HL` — буфер, `A` — число секторов 512 байт	Читает подряд, продвигает `HL`; ошибку записывает в `ABT`
`#3812`	`+18`	`SDDSE`	`HL` — буфер, `A` — число секторов 512 байт	Пишет подряд, продвигает `HL`; ошибку записывает в `ABT`
`#3815`	`+21`	`RDD256`	`HL` — буфер, `A` — число 256-байтовых блоков	Необязательный старый быстрый вход

`DE:HL` в LBA означает `DE` — старшее слово, `HL` — младшее. Секторный размер CORE32 фиксирован: 512 байт. Драйвер с иным физическим блоком обязан сам представить интерфейс 512-байтовых логических секторов.

4. Общая область состояния

После 72-байтовой таблицы по `#3848` начинается layout, который должен совпасть с `KERNEL.ASM`. Драйверный исходник повторяет эти объявления, потому что распаковка одновременно устанавливает начальные значения CORE32.

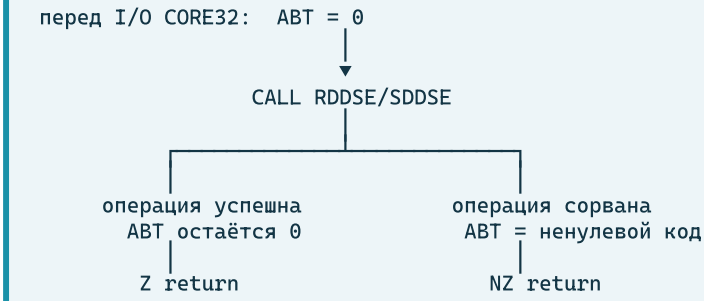
Адрес	Имя	Размер	Назначение
`#3848`	`NSDC`	1	Сектор внутри текущего кластера
`#3849`	`EOC`	1	Состояние конца FAT-цепочки
`#384A`	`ABT`	1	Единый код низкоуровневой ошибки
`#384B`	`BZN`	1	Остаток блоков текущего запроса
`#384C`	`NR0`	2	Размер текущей низкоуровневой порции

Адрес	Имя	Размер	Назначение
`#384E..`	—	—	Позиции потока и промежуточные значения
`#3868`	`ENTRY`	32	Текущая FAT32 directory entry
`#3888`	`FATFLAGS`	1	Младший байт BPB_ExtFlags
`#3889..`	—	—	BPB, FAT, root, FSInfo, текущие кластеры
до `RECCAT`	—	—	Текущий и сохранённый каталоги

Точные адреса вычисляются в `source/KERNEL.ASM`. Не вставляйте приватную переменную внутрь этого layout: весь код CORE32 обращается к фиксированным offset.

Размещайте приватные данные/код драйвера после общей области и завершайте образ до `#4000`. Полный runtime имеет не более `#0800` байт от `#3800` до границы окна.

5. Контракт ошибки `ABT`



Обёртки `SAFE_RDDSE/SAFE_SDDSE` очищают `ABT`, вызывают драйвер, затем делают `LD A,(ABT) / OR A`. Поэтому новый драйвер обязан сигнализировать отказ через `ABT`, а не только через `CF`, статусный регистр контроллера или произвольный ненулевой `A`.

Рекомендуемый эпилог:

```
.ok:
        XOR     A
        LD      (ABT),A
        RET

.io_error:
        LD      A,#01               ; внутренний ненулевой код драйвера
        LD      (ABT),A
        OR      A                   ; удобно и прямому вызывающему: NZ
        RET
```

Старые драйверы не всегда меняли `ABT`; обёртка трактует их возврат как успех. Боевой SD-ZC сохраняет этот старый контракт и используется без runtime-патча. Для нового драйвера не повторяйте историческую неопределённость.

Правила отказа:

- любой timeout, CRC/token error, ATA ERR/DF и недовыполненный сектор должны вернуть управление и установить `ABT`;
- не оставляйте chip select, DMA или контроллер в состоянии незавершённой транзакции;
- не продолжайте следующий сектор после первой ошибки;
- `HL` после ошибки может указывать за фактически переданной частью, но верхний API не сообщает частичное число; вызывающий должен считать весь запрос неуспешным;
- бесконечное ожидание статуса превращает ошибку носителя в зависание всего WC, поэтому каждый wait-loop нового драйвера должен иметь конечный бюджет.

6. Позиционирование

CORE32 использует две похожие операции:

```
XPOZI(DE:HL)
  ├─ сохраняет LBA в общих LTHL/LTDE
  └─ подготавливает приватный BLKNUM/регистры

PROZ(DE:HL)
  └─ только подготавливает приватный BLKNUM/регистры
```

`RDDSE/SDDSE` читают позицию, ранее установленную одной из них. После I/O CORE32 сам продвигает общую LBA потока; драйвер не должен тайно прибавлять к `LTHL/LTDE`. Контроллер может внутренне перейти к следующему сектору в рамках multi-block команды — это нормально.

Пример IDE-преобразования LBA28:

```
sector number  = LBA bits 7..0
cylinder low   = LBA bits 15..8
cylinder high  = LBA bits 23..16
drive/head     = master/slave prefix | LBA bits 27..24
```

SDHC принимает LBA как номер блока; SDSC/MMC требует байтовый адрес. Это различие определяется во время `DEVINI` и хранится в состоянии драйвера.

7. Многоблочный обмен и буфер

`A` задаёт число последовательных блоков, а `HL` — первый байт. Успешный драйвер должен обработать ровно запрос и вернуть `HL`, продвинутый на `512*A` для RDDSE/SDDSE или `256*A` для RDD256.

```
HL before ──► [ sector 0 ][ sector 1 ] ... [ sector A-1 ] ──► HL after
              512 bytes   512 bytes          512 bytes
```

Не разрешайте физическому обмену молча перейти через `#FFFF→#0000`. Внешняя обёртка плагинов требует делить такой запрос. Внутренний код также должен подавать порции, помещающиеся в видимый адресный диапазон.

DMA можно использовать, если адрес и контроллер удовлетворяют требованиям выравнивания. Обязательно оставьте PIO-путь для невыравненного буфера. Текущий Nemo и ZC выбирают DMA для подходящих адресов, иначе выполняют портовый цикл.

Особый случай: приватный CORE32_EXT временно отображается в `#C000`. Перед PIO-записью буфера из этого окна шлюз обязан вернуть исходную страницу, иначе драйвер прочитает код расширения вместо пользовательских данных. Новый драйвер не должен самостоятельно предполагать, какая физическая страница стоит в `#C000`; он работает с данным логическим `HL`.

8. Встроенные драйверы

Драйвер	Runtime	Интерфейс	`SELDEV`	`RDD256`	Важная особенность
`DIDENEMO.ASM`	959 байт	IDE Nemo	`A=0` master, иначе slave	Есть	ATA PIO/DMA, LBA28
`DIDESMUC.ASM`	903 байта	IDE SMUC	`A=0` master, иначе slave	Есть	Сначала включает SMUC через `#FFBA`
`DSDZC.ASM`	1164 байта	SD Z-Controller	`A=0` первый, иначе второй	Есть	CMD18/CMD25, DMA/PIO, шлюзы конечного ожидания
`DSDNGS.ASM`	1378 байт	NGS SD-service	выбор не делится на master/slave	Пустой `RET`	Загружает 1-КиБ службу в память NGS

Порты и протоколы:

Драйвер	Основные порты
Nemo	ATA-регистры `#10/#11`, `#30/#50/#70/#90/#B0/#D0/#F0`
SMUC	ATA-регистры `#D8BE/#F8BE..#FFBE`, config `#FFBA`
ZC	config `#77`, data `#57`
NGS	command `#BB`, data `#B3`

Шина SPI Z-Controller общая у SD-карты и FT812 платы VDAC2: `#77` выбирает устройство (`#03` — никто не выбран, `#07` — FT812), `#57` — данные. Аппаратный сброс TS-Config её не нормализует: если сброс пришёлся на обмен, защёлка `#77` остаётся с выбранным FT812 или SD. Поэтому установщик WildDOS (`WDOS_INSTALL_COPY`, сразу после INIT и до первого чтения карты в FINI) выполняет SpiBusIdle: `#77=#03`, 16 байт `#FF` в `#57` (128 тактов), снова `#77=#03` — проверенную на железе последовательность порта Zuma Deluxe для VDAC2/FT812 (2026-09-26, тест `test_spi_idle.py`). Код, работающий с FT812, по окончании обмена должен так же оставлять `#77=#03`.

SD-NGS не поддерживает API 60 `LOAD256`. Код верхнего уровня обязан либо запретить этот путь, либо использовать обычный API 48.

9. Выбор драйвера через `DOS_SWP`

Внутренний вход `#401B` принимает:

```
A=0  Nemo master
A=1  SD-ZC #1
A=2  SD-NGS
A=3  SMUC master
A=4  SMUC slave
A=5  Nemo slave
A=6  SD-ZC #2
```

```
CALL #401B
    │
    ├─ выбрать packed block DR1..DR4 (адрес в ядре)
    ├─ распаковать в #3800 (шлюз: тот же блок в DRIVERS_E8 страницы #E8)
    ├─ применить runtime-расширение, если требуется
    └─ CALL #3800 SELDEV
```

`DOS_SWP` сохраняет `IX`, но остальное следует считать внутренним ABI. После замены драйвера вызывают `#4003 DEVINI`, затем `#4009 HDD`, чтобы найти FAT32 и заполнить BPB. Просто распакованный драйвер ещё не означает смонтированный том.

Обычный плагин не должен вызывать `#401B`: устройство панели выбирается в `wc.ini`, а отдельный поток создаётся публичным API 57.

10. Каркас нового драйвера

```
        ORG     0
        DISP    #3800

DRIVER:
        JP      SELDEV
        JP      DEVINI
        JP      REINI
        JP      XPOZI
        JP      PROZ
        JP      RDDSE
        JP      SDDSE
        JP      RDD256

        ; 16 обязательных резервных слотов.
        DUP     16
        DS      3,0
        EDUP
        ASSERT  $ == #3848

; Ниже должна в точности следовать общая структура из KERNEL.ASM.
NSDC:   DS      1
EOC:    DS      1
ABT:    DS      1
        ; ... BZN, NR0, ENTRY, BPB/FAT, LSTCAT, RECCAT ...

; Приватные поля и код — только после последнего общего поля.
PRIVATE_LBA:    DS 4

SELDEV:
        ; Сохранить вариант устройства и перевести контроллер в idle.
        RET

DEVINI:
        ; Определить устройство, режим адресации и размер логического блока.
        XOR     A                       ; Z = готово
        RET

REINI:
        RET

XPOZI:
        LD      (LTHL),HL
        LD      (LTDE),DE
PROZ:
        LD      (PRIVATE_LBA),HL
        LD      (PRIVATE_LBA+2),DE
        RET

RDDSE:
        ; A секторов: дождаться с timeout, передать 512*A, продвинуть HL.
        ; При любом отказе записать ненулевой ABT и вернуться.
        RET

SDDSE:
        ; Симметричный путь записи.
        RET

RDD256:
        ; Реализовать либо явно документировать как неподдерживаемый.
        RET

DRIVER_END:
        ASSERT  DRIVER_END <= #4000
        ENT
```

На практике блок общих полей нельзя сокращать многоточием: скопируйте текущий layout из `KERNEL.ASM` или вынесите его в общий include одновременно для ядра и всех драйверов. Добавьте `ASSERT` хотя бы для `VALS=#3848`, `ENTRY=#3868`, `ABT=#384A` и конечной границы.

11. Подключение нового драйвера к WC

```
driver.asm
    │ assemble at logical #3800
    ▼
driver.bin
    │ Hrust pack + unpack byte comparison
    ▼
driver.CPD
    │ include in CORE32 + DOS_SWP mapping
    ▼
boot payload
    │
    ├─ добавить имя/номер в SDRV и wc.ini
    ├─ сохранить panel.device mapping
    └─ обновить проверки размера/символов
```

Последовательность изменений:

1. Реализовать таблицу и общий layout без изменения адресов.
2. Ассемблировать runtime с `DISP #3800`; убедиться, что конец ниже `#4000`.
3. Упаковать Hrust и распаковать обратно; байты должны совпасть с runtime.
4. Добавить packed block в конец CORE32 так, чтобы не вытеснить `DEHR` и фиксированные шлюзы. С 2026-09-27 блоки DR1..DR4 при установке переносятся в `DRIVERS_E8` страницы `#E8` одним куском `DR1..DEHR`: новый блок между ними перенесётся сам, но `DRIVERS_LENGTH` вырастет — проверить, что рабочие таблицы расширения (`BSS_END`) по-прежнему помещаются в `#E8`.
5. Расширить выбор в `DOS_SWP` либо заменить существующий вариант осознанно.
6. Добавить строку настройки панели и согласовать числовой device ID.
7. Проверить `DEVINI → HDD → RDD/SDD` на отдельном тестовом образе, затем уже файловые операции.

`build.ps1` сейчас заново собирает четыре исходника, проверяет точный размер и SHA-256 их runtime. Для нового драйвера добавьте аналогичный элемент в `DriverChecks`, чтобы исходник не превратился в необязательную копию.

12. Диагностика по слоям

Симптом	Первый уровень поиска
`DEVINI` сразу NZ	SELDEV, порты, reset/identify, выбранный master/slave
`DEVINI` Z, но `HDD` NZ	чтение LBA0, адресация SDSC/SDHC, размер 512, MBR/BPB
Чтение работает, запись портит соседний сектор	число блоков, продвижение HL, LBA, stop token/flush
Ошибка носителя превращается в зависание	wait-loop без timeout
Ошибка возвращается как успех	драйвер не записал `ABT`

Симптом	Первый уровень поиска
DMA работает только на части адресов	отсутствует PIO fallback или неверное выравнивание
Левая панель работает, правая нет	драйвер хранит состояние вне панельной страницы
API 60 ничего не читает на NGS	`RDD256` этого драйвера пустой

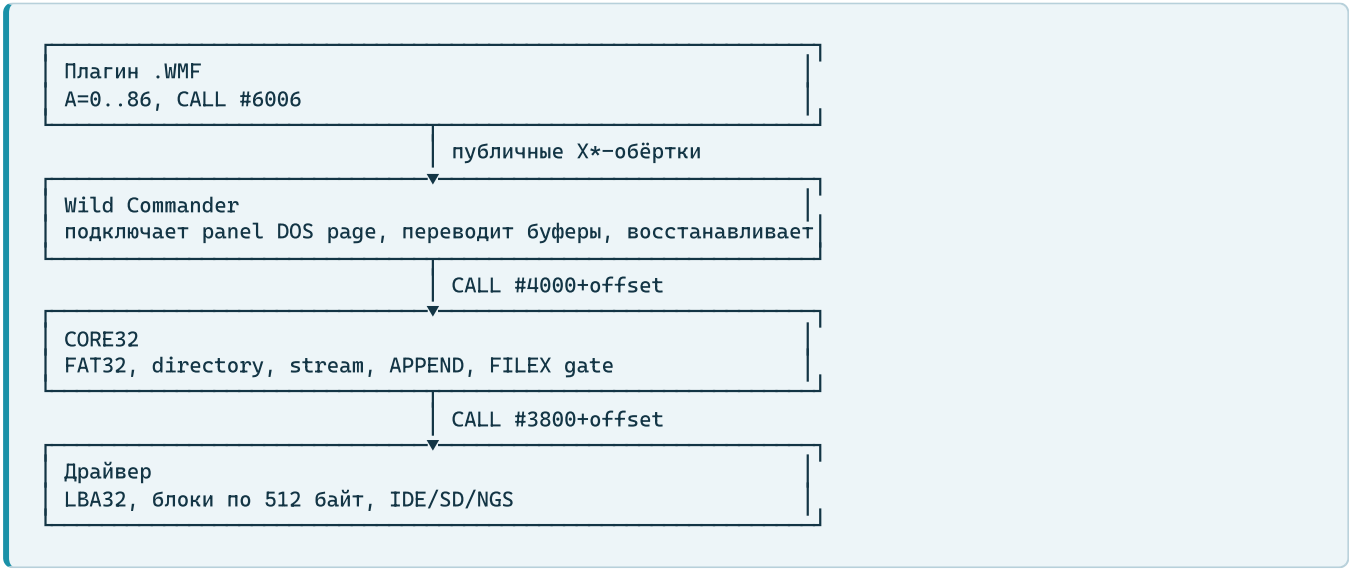
13. Файлы-источники

- `source/KERNEL.ASM` — фиксированная таблица и общий layout.
- `source/CORE32.ASM` — `RDD`, `SDD`, безопасные обёртки и `DOS_SWP`.
- `source/DIDENEMO.ASM` — полный пример ATA/Nemo.
- `source/DIDESMUC.ASM` — ATA через SMUC.
- `source/DSDZC.ASM` — SD multi-block, DMA и конечные шлюзы ожидания.
- `source/DSDNGS.ASM` — host/service-протокол NGS.
- `source/DEHR2.ASM` — распаковщик runtime.
- `build.ps1` — сборка, размеры, хэши и упаковка.

Уровень WildDOS/CORE32: файловое API и внутренняя архитектура

WildDOSFileCore, или CORE32, реализует FAT32, каталоги, потоки и операции изменения файлов. Этот документ показывает, как пользоваться им из плагина и как обращаться к внутреннему API из кода самого менеджера.

1. Четыре уровня вызова



Для `.WMF` допустим только верхний вход `#6006`. Внутренний `#4000` не подключает нужную страницу и не защищает память плагина.

2. Установка при старте

Физический payload содержит CORE32 около `#C000`. При `WC_ENTRY` установщик копирует рабочую часть в постоянный логический диапазон `#4000..#5BF1`:

```
boot payload                          runtime Z80

#C00C..#CA87 (DR1..DR4, DEHR) LDIR ─► #5176..#5BF1
#CA88..#D73B (код ядра, HR) ── DEHR ─► #4000..#5175
CORE32_EXT, часть 1 ── DEHR ─────────► page #E8:#C000..WDOS_EXT_PART2_START-1
CORE32_EXT, часть 2 (хвост файла
  #D73C..) ── LDIR на место части 1,
  DEHR ──────────────────────────────► page #E8:WDOS_EXT_PART2_START..
packed drivers #5176..#5AA9 ─ LDIR ─► page #E8:DRIVERS_E8 (распаковывает DOS_SWP в #3800)
резидентный блок расширения ─ LDIR ─► #5176.. (на место драйверов)
второй резидентный блок ──── LDIR ─► #56C6..#5AA9 (с 2026-10-10)
образ WMF #11 ──────────────── LDIR ─► #5300..#56C1 (WMF_RUNTIME, docs/05, 5.4)
место оверлея FILEX ← HALT (#76), затем FILEX.WMF при своей ранней установке
  копирует туда оверлей ──────────► page #E8:#D38B..#E656 (docs/05, 5.5)
```

Код `#4000..#5BF1` соседствует с менеджером `#6011..`; данные находятся в переключаемой странице `#0000..#3FFF`. Это даёт каждой панели собственное файловое состояние при одном общем коде.

С 2026-09-27 код ядра `#4000..#5175` лежит в файле HR-потоком (4470 байт → 3349), драйверы DR1..DR4 и DEHR — как есть: заглушка по `#C000` ставит LDIR в `#5176..#5BF1`, затем только что поставленный DEHR (`#5AAA`) разворачивает код. Расширение — две части (место перед `#BF60` кончилось, docs/05, 5.3); граница частей — по месту (tools/split_extension.py): поток части 1 влезает до `#BF60`, поток части 2 — в хвост до `#E000` и на место потока части 1, обе части распаковываются обратно (у отдельных длин MHMT даёт поток, который его же распаковщик не читает). Длина потока растёт с границей не строго, поэтому за двоичным поиском просматривается окно вокруг найденной границы и берётся первая сверху, проходящая все проверки; части — не короче 16 байт (меньше MHMT не пакует). Часть 2 лежит в хвосте файла за потоком кода ядра, при загрузке — в окне `#C000` загрузочной страницы. Весь `boot.$C` — не длиннее 32768 байт: BIOS TS-Conf читает файл целыми кластерами с `#6000`, и при кластере 32 КиБ (обычном у SDHC после SD Formatter) второй кластер — 64 КиБ с `#6000` — через `#FFFF` затирал `#0000..#5FFF`: экран, системные переменные и стек BIOS в странице `#05`; машина падала до старта WC (стенд combo64 с файлом 35345 байт, 2026-09-27). Последний сектор Hobeta не добивается: 128 целых секторов с заголовком — уже 32785 байт. Установщик до любой распаковки сверяет метку сборки в самом конце файла (`WDOS_EXTENSION2_MARK`; загрузчик, прочитавший файл не до конца, оставил бы чужие байты и на месте потока кода ядра, и на месте части 2 — тогда красный бордюр и остановка DI:HALT, WC не запускается), ставит ядро, распаковывает часть 1 в `#E8`, возвращает загрузочную страницу, переносит поток части 2 на место потока части 1 (там уже можно писать) и распаковывает его туда же, за часть 1. Затем `EXT_INSTALL_TAIL` расширения (строка таблицы ID_INSTALL_TAIL, 52; за ней — ALIAS_RESET, ALIAS_ADD, ALIAS_SCAN) переносит упакованные драйверы DR1..DR4 ядра (2356 байт, `#5176..#5AA9`) в `DRIVERS_E8` страницы `#E8` — `DOS_SWP` распаковывает их оттуда через шлюз (UNPACK_DRIVER прибавляет сдвиг), а на их место в странице `#05` кладёт резидентный блок расширения: `CORE_MKSG` и `CORE_SRHFCL` зовут MKSG и SRHFCL ядра со страницей и прерываниями вызывающего (OUTER_PAGE, OUTER_IFF — их запоминает шлюз при внешнем входе) и возвращают `#E8` и DI. Загрузчики: TS-BIOS (читает файл по цепочке до конца) и второй — `WCVW.RUNHOB` (Enter на `boot.$C` в панели) проверены на стенде; запуск из TR-DOS/Mounter — нет.

Шлюз расширения закрывает прерывания на всё время функции, а кадровое прерывание TS-Conf — короткий импульс: пришло при DI — потеряно. Поэтому на пути потокового чтения и записи шлюза нет (с 2026-10-01): подмена обработчика сектора для `LOAD256` / `LOADNON`, проверка ссылки FAT в `GIPAG` / `GIPP` и переход потока к следующему кластеру при попадании в кэш FAT лежат в том же резидентном блоке, ядро зовёт их напрямую по постоянным адресам (`STREAM_RESIDENT`, `CLASSIFY_RESIDENT`, `STREAM_LINK_RESIDENT` — `RESIDENT_BASE` +4/+7/+13, CORE32_EXT_API.ASM; с 2026-10-10 перед ними `OVERLAY_STATE` и вход `FILEX_REQUIRED`, между ними — `GIPAG_FILE`, а во втором блоке с `#56C6` — перевод имён и проверка LOAD256). Прежде каждый вызов `LOAD256` и каждый переход к кластеру шли через шлюз: плагин, который считает кадры по прерываниям во время чтения (VIDEO_PL без VDAC2), их терял — в выпусках 09-28 и 09-30 кадры ролика не сменялись вовсе. Через шлюз остались чтение сектора FAT при промахе кэша и учёт ошибок. Новую функцию расширения на этот путь ставить нельзя — только в резидентный блок.

С 2026-10-10 часть расширения — оверлей FILEX (docs/05, 5.5): код записи, APPEND, псевдонимов, часов и строки состояния хранится не в `boot.$C`, а в обязательном `FILEX.WMF`, который ставит его в `#E8`, сверив штамп сборки. Без FILEX или с чужим FILEX AX_RUN выводит сообщение и останавливает машину. Граница частей расширения теперь держит и то, что поток каждой части не длиннее её данных: DEHR кладёт поток в конец области назначения и распаковывает оттуда, и поток длиннее данных лёг бы ниже начала части — так часть 2 из 16 байт (поток 26) затёрла хвост второго резидентного блока (тест установщика, 2026-10-10).

Шлюз расширения возвращает вызывающему все регистры функции, IX тоже. Функция, которой IX не нужен для результата, обязана его сохранить: WC держит в IX описатель окна или панели (стенд 2026-09-27: `ALIAS_ADD`, оставлявший в IX указатель слота, давал F8 ложные отметки и зависание).

3. Монтирование тома

Внутренний путь панели:

```
DOS_SWP #401B
   │ распаковать драйвер и SELDEV
   ▼
DEVINI #4003
   │ обнаружить устройство
   ▼
HDD #4009
   ├─ прочитать LBA 0
   ├─ найти FAT-раздел записи MBR (#05/#0B/#0C/#0F), иначе – начало
   │  записи 1 любого типа (запись 2 пуста), иначе superfloppy BPB;
   │  логические тома в EBR не ищутся
   ├─ проверить FAT32 BPB, bytes/sector=512, SPC=power of two, резерв ≠ 0
   ├─ прочитать BPB_ExtFlags и FSInfo
   └─ вычислить FAT/data geometry; отказ: FATSz32 ≥ 2^24, ни одного кластера
      данных, граница номеров кластеров ≥ 2^28 (с 2026-10-10)
```

С 2026-10-10 (перенос из драйвера FAT32) LOAD_FREE_HINT отвергает и такие тома — монтирование кончается отказом HDD (NZ):

- конец тома за 2^32 (`ADDTOP` + число секторов > 2^32; ровно 2^32 можно) — прежде сложение теряло перенос, и последние сектора тома ложились в начало носителя;
- том длиннее своего раздела MBR. Длину раздела HDD берёт из той же записи, что и начало (`OKK` и запасной путь — запись 1), и держит на время монтирования в `DAHL:DADE` (временные ячейки CHTOSE и поиска места); у superfloppy границы нет (`#FFFFFFFF`). Раздел нулевой длины — пустой: отказ;
- граница номеров кластеров дальше `#0FFFFFF0` (точно, не только 2^28): ссылки `#0FFFFFF0..#0FFFFFF6` ядро считает зарезервированными, файл на таких номерах потом не открывался бы;
- FAT короче, чем нужно на все номера ниже границы (`ceil(граница/128)` секторов) — прежде номера без записи в FAT считались годными.

FSInfo читается (подсказка свободного места) и пишется (`RFRH`) только внутри резервной области: `0 < FSINF-ADDTOP < BREZS`. Вне её (испорченный `BPB_FSInfo`) FSInfo не трогается — подсказка холодная, `RFRH` отвечает успехом без записи; прежде RFRH переписывал там чужой сектор, если в нём находились подписи FSInfo.

Основные вычисленные поля:

Имя	Значение
`ADDTOP`	Абсолютный LBA начала выбранного раздела/тома
`BREZS`	BPB reserved sectors
`FSINF`	Относительный сектор FSInfo
`BFATS`	Число FAT

Имя	Значение
`BFTSZ`	Размер одной FAT в секторах, 32 бита
`BSECPC`	Секторов на кластер
`BROOTC`	Корневой кластер
`SFAT`	Относительное начало FAT, обычно reserved sectors
`SDFAT`	Относительное начало data area
`FSTFRC`	Подсказка первого свободного кластера

Преобразование кластера в LBA:

```
data_lba(cluster) = ADDTOP + SDFAT + (cluster - 2) * BSECPC
fat_lba(cluster)  = ADDTOP + SFAT  + floor(cluster * 4 / 512)
```

Кластер `0` в `GIPAG` означает root и заменяется `BROOTC`. Для файла это не так: GFILE (API 62) с 2026-10-10 идёт через `GIPAG_FILE` (7.20) — у файла кластер 0 значит «пустой файл».

4. Рабочая память CORE32

```
#0000 ┌──────────────────────────────┐
      │ GENBU/NXTBU: directory       │
#1000 │ две половины по 4 КиБ        │
#2000 ├──────────────────────────────┤
      │ внутренняя служебная область │
#3000 ├──────────────────────────────┤ SECBU: FAT sector
#3200 ├──────────────────────────────┤ LOBU: sector/data
#3400 ├──────────────────────────────┤ LOBU2: second sector
#3600 ├──────────────────────────────┤
#37F8 │ iterator state               │
#3800 ├──────────────────────────────┤ driver JP table
#3848 │ stream/BPB/ENTRY state       │
#4000 └──────────────────────────────┘
```

Важные следствия:

- перечисление каталога уничтожает `#0000..#1FFF` этой физической страницы;
- `SECBU/LOBU/LOBU2`, текущая ENTRY и поток глобальны внутри одной панели;
- файловые вызовы не реентерабельны;
- указатель в одном из этих буферов нельзя сохранять через другую операцию;
- внешний плагин должен держать свои структуры в своей странице и пользоваться обёртками `#6006`.

5. FAT32 ENTRY и имена

Текущая короткая запись имеет стандартные 32 байта FAT:

Offset	Поле
`+0..+10`	SFN 8.3
`+11`	Атрибуты
`+13`	Create tenth
`+14/+16`	Create time/date
`+18`	Access date
`+20`	Старшие 16 бит первого кластера
`+22/+24`	Write time/date
`+26`	Младшие 16 бит первого кластера
`+28`	Размер LE32

Публичные запросы имени используют форму:

```
[type][name bytes][0]

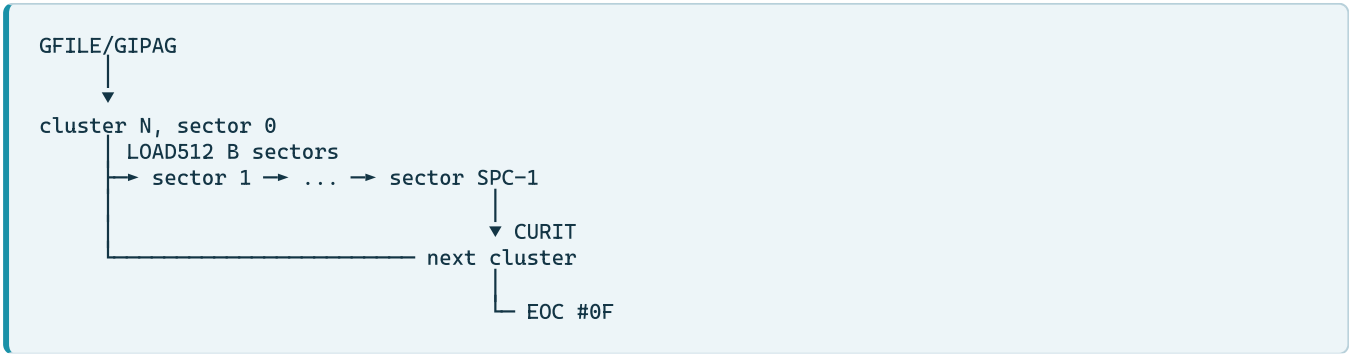
type #00 = file
type #10 = directory
```

Текущий валидатор принимает до 255 байт, отвергает управляющие символы и `"*/:<>?\|`. Конечные пробелы/точки при создании удаляются. Пробел внутри имени включает LFN, при поиске — и точка в конце (кроме `.` и `..`). Начальные и внутренние пробелы LFN значимы: « name» и «name» — разные имена (с 2026-10-10; прежде начальные пробелы пропускались с обеих сторон). Длинное имя, чьи записи занимают три сектора, находится.

Знаки имени переводятся CP866 ↔ UTF-16 по полной таблице (с 2026-10-10): кириллица, Ё/ё и, кроме того, псевдографика `#B0..#DF` и `#F2..#FE` (Є, є, Ї, ї, Ў, ў, °, ·, ·, √, №, ¤, ■) — при записи LFN (ORU), сравнении имён (UCS) и выводе имени в список (UCH, NXTETY2). Все три входа ядра ведут во второй резидентный блок (`OEM_UCS_RESIDENT`, `UCS_RESIDENT`, `UCH_RESIDENT`), длина и адреса ядра прежние. Прежде ORU писал такие знаки как «ё» («Папка №2» становилась «Папка ё2»), а UCS и UCH их не знали: имя не находилось, в списке было короткое имя. Под запретом в длинном имени остаётся `#FF` (неразрывный пробел, код 1): в сравнении имён `#FF` — конец имени на диске. Короткое 8.3 хранит любые знаки как есть. Регистр Є/є, Ї/ї, Ў/ў при сравнении различается (ACS их не сводит).

6. Потоковая модель

Поток хранит текущий кластер, LBA, сектор внутри кластера, EOC, обработчик I/O и активный каталог.



Результат нормального потока:

Состояние	A	CF	Действие вызывающего
Запрос выполнен, цепочка продолжается	`#00`	0	Можно читать следующий блок
Встречен FAT EOC	`#0F`	0	Штатный физический конец цепочки
I/O/FAT error	`#FF`	1	Немедленно прекратить

`#0F` — не логический EOF файла. Всегда ограничивайте полезные байты 32-битным размером ENTRY/FENTRY.

Переход к следующему кластеру читает сектор FAT только при смене сектора (один раз на 128 кластеров): `SECBU` страницы служит кэшем, а признак `FATTAG` (`#3858`) сверяется с общим поколением FAT. Поколение сбрасывают запись FAT, монтирование, смена и инициализация драйвера и начало чтения каталога; подробности — в разделе «Контракт потоков» `source/CORE32_API.md`.

`SAVE512` пишет только уже выделенную цепочку. Он не создаёт кластер и не изменяет размер. Для роста есть APPEND или FILEX.

7. Публичные файловые сценарии плагина

7.1 Найти → выбрать → прочитать

```
FENTRY 59  [#00,"FILE",0]
   | NZ, DE:HL=size
   v
GFILE 62
   |
   v
LOAD512 48  (HL=buffer, B=sectors)
```

```
        LD      HL,Query
        LD      A,59
        CALL    #6006
        JR      Z,.not_found
        LD      (SizeLo),HL
        LD      (SizeHi),DE

        LD      A,62
        CALL    #6006

        LD      HL,Buffer
        LD      B,1
        LD      A,48
        CALL    #6006
        JR      C,.io_error
```

7.2 Создать → записать существующую цепочку

```
MKFILE 72 [attr,size32,name,0]
   │ Z; поток поставлен в начало созданного файла
   ▼
SAVE512 49
```

Если размер не кратен 512, последний сектор всё равно существует целиком; код должен определить содержимое его хвоста. Для нового пустого файла создайте `size=0`, затем используйте APPEND/WRITE_AT вместо SAVE512.

7.3 Перечислить каталог

```
STREAM 57, D=#FE       clone panel stream
ADIR 56, A'=1          reset enumerator
NXTETY2 58             repeat until Z
```

Выход API 58 имеет переменный формат: включённые маской size/date/time, затем `entry_flag`, имя и ноль. Перед передачей записи в FENTRY/DELFL оставьте в флаге только bit 4 (`AND #10`).

7.4 Создать каталог и войти

```
MKDIR 73 [name,0]
  │ Z
  ▼
FENTRY 59 [#10,name,0]
  │ NZ
  ▼
GDIR 63
```

В исходном потоке панели GDIR запрещён: сначала клонируйте поток. Если цель — изменить видимую панель, завершите плагин с `A=3` и позвольте менеджеру самому перечитать каталоги обеих панелей, сохранив активную.

7.5 Удалить каталог

API 75 не проверяет, пуст ли каталог. Безопасный порядок:

1. FENTRY каталога.
2. Клонировать поток и выполнить GDIR.
3. NXTETY2 убедиться, что кроме `.` / `..` записей нет.
4. Вернуться в родительский поток.
5. DELFL с type `#10`.

7.6 Перед первым DOS-вызовом: памятка начинающему

Публичный DOS API вызывается тем же способом, что остальной API WC:

```
        ; 1. Заполнить все входные регистры и буферы.
        ; 2. Последним загрузить номер функции в A.
        LD      A,59                ; десятичный номер FENTRY
        CALL    #6006
        ; 3. Сразу проверить документированный флаг.
        JR      Z,.not_found
```

Основные обозначения:

```
DE:HL       32-битное число: DE — старшие 16 бит, HL — младшие
Z           zero flag установлен
NZ          zero flag сброшен
CF          carry flag; для потока CF=1 означает ошибку
[type,name,0]
             байт #00/#10, байты имени, завершающий ноль
buffer end  адрес сразу после буфера, а не адрес последнего байта
```

Если функции нужен входной `A`, его передают через `A'`, потому что основной `A` занят номером API:

```
        LD      A,1                 ; подкоманда ADIR: reset iterator
        EX      AF,AF'
        LD      A,56                ; номер API
        CALL    #6006
```

Самые безопасные места для данных обычного плагина — его текущая страница `#8000..#BFFF`. Не помещайте запрос или имя в `#C000`: многие вызовы сами переключают это окно. Не храните рабочие данные в `#0000..#3FFF`, если плагин переключает окно API 78/80. FILEX вообще принимает только `#8000..#BFFF`.

7.7 API 48 `LOAD512`: последовательное чтение

Назначение: прочитать один или несколько 512-байтовых секторов из уже выбранного файлового потока.

Перед вызовом поток должен указывать на файл:

- при запуске плагина по расширению WC уже позиционировал его;
- после API 59 `FENTRY` нужно вызвать API 62 `GFILE`;
- для исходного файла, переданного плагину, можно повторно вызвать API 50.

Параметр	Значение
`HL`	Начало выходного буфера
`B`	Число блоков по 512 байт
Выход `HL`	Адрес сразу за обработанными блоками
Выход `A=0, CF=0`	Запрос завершён, цепочка ещё не доказанно закончилась
Выход `A=#0F, CF=0`	Встречен физический EOC FAT
Выход `A=#FF, CF=1`	Ошибка чтения/FAT

```
        LD      HL,Buffer
        LD      B,2                     ; 1024 байта
        LD      A,48
        CALL    #6006
        JR      C,.read_error
        LD      (NextBuffer),HL
```

Нельзя считать, что все 1024 байта полезны: сначала сохраните размер из FENTRY и обрабатывайте только `min(request, bytes_left)`. Не задавайте блок, который пересекает `#FFFF→#0000`; разделите его. Вызов теряет состояние FindNext, поэтому перечисление каталога после него нужно начать заново.

7.8 API 49 `SAVE512`: последовательная запись без роста

Назначение: записать данные в уже существующие секторы выбранной FAT-цепочки.

Параметр	Значение
`HL`	Исходный буфер
`B`	Число блоков по 512 байт
Выход	Как LOAD512: новый HL, потоковый A/CF

```
        LD      HL,Buffer
        LD      B,1
        LD      A,49
        CALL    #6006
        JR      C,.write_error
```

Критически важно: SAVE512 не выделяет кластеры и не меняет поле размера. Его можно применять после MKFILE с заранее заданным ненулевым размером либо для перезаписи существующего файла в пределах его цепочки. Запись за логический EOF через SAVE512 создаёт несогласованные скрытые данные; для этого используйте APPEND или WRITE_AT.

Статус конца цепочки ограничивает только физически выделенные кластеры, а не логический EOF. Поэтому последний вызов должен передавать в `B` ровно `ceil(bytes_left/512)`, а не полный 32-секторный буфер. Иначе маленький файл на крупном кластере без необходимости запишет до 16 КиБ кластерного хвоста.

Штатный `WCFX.COPYF` следует этому правилу: сохраняет 32-битный размер из `FENTRY`, обрабатывает полные блоки по 16 КиБ и завершает копирование точным числом секторов последнего блока. Для файла 541 байт это два сектора независимо от того, выделила FAT под файл кластер 4, 32 или 64 КиБ.

7.9 API 50 `GIPAGPL`: вернуть исходный файл плагина

API 50 берёт кластер из кэша выбранного менеджером объекта `#7F28` и ставит поток на его первый сектор.

```
        LD      A,50
        CALL    #6006
```

Входных регистров нет. Используйте его, если другой вызов сдвинул поток файла, который WC передал плагину. Для произвольного файла, найденного вашим FENTRY, правильный вызов — API 62 GFILE, а не API 50.

API 50 — прежний GIPAG (кластер 0 — корень): WC сам ставит им поток и на каталог, в том числе на корень. Проверку пустого файла и кластера корня (2026-10-10) делает только GFILE (7.20).

7.10 API 51 `TENTRY`: получить 32 байта FAT ENTRY

Параметр	Значение
`DE`	Выходной буфер минимум 32 байта
Выход	32 байта скопированы, `DE` продвинут LDIR

```
        LD      DE,EntryCopy
        LD      A,51
        CALL    #6006
```

Из ENTRY можно прочитать атрибут, timestamps, first cluster и size. Копируйте её сразу: внутренний экземпляр общий и будет заменён следующим FENTRY/GETEN. Не меняйте полученную копию в надежде изменить диск — это только снимок.

Времена в ENTRY — как на носителе, без проверки календаря. Штампы для показа или передачи дальше (FTP, SMB) берите у FILEX GET_METADATA (10.3): там штамп, которого нет в календаре, уже заменён на 2026-01-01 00:00. API 51 копирует снимок ENTRY, сохранённый WC при FENTRY, и контекст APPEND/FILEX не трогает. Прямой вход ядра `#4036 TENTRY` (8.14) — другое: он заново запоминает положение записи по альтернативному `BC`.

7.11 API 52 `CHTOSEP`: разложить цепочку в LBA

Назначение: получить последовательность физических LBA32 всех секторов файла, переданного менеджером.

Параметр	Значение
`DE`	Начало массива четырёхбайтовых LBA
`BC`	Адрес сразу после массива
Источник цепочки	Сохранённый кластер объекта `#7F28`

```
        LD      DE,SectorList
        LD      BC,SectorListEnd
        LD      A,52
        CALL    #6006
```

Это исторический низкоуровневый вызов без удобного возвращаемого счётчика и с грубой проверкой границы по кластерам. Буфер должен быть рассчитан заранее из размера файла и `sectors_per_cluster`; оставляйте место как минимум для полного последнего кластера. Для обычного чтения используйте LOAD512, а для позиционного — READ_AT.

7.12 API 53 и 55: зарезервированы

Оба номера ведут в пустой `RET`:

```
        LD      A,53                ; ничего не делает
        CALL    #6006
```

Они не являются TENTRYN/TMRKNXT и не должны использоваться как feature probe. API 76/77 находятся на собственных номерах и не заменяют эти значения.

7.13 API 54 `TMRKFL`: отмеченные объекты панели

API работает с внутренним индексом панели и требует исходный `IX`, полученный при входе в плагин.

Режим подсчёта:

```
        LD      HL,0
        LD      A,54
        CALL    #6006
        JR      Z,.none_marked
        ; BC = число отмеченных объектов
```

Режим получения N-го отмеченного объекта:

```
        LD      HL,1                ; первый отмеченный
        LD      DE,NameBuffer       ; до #BFFF, место до 256 байт
        LD      A,54
        CALL    #6006
        JR      NZ,.not_found       ; для этого API Z = имя получено
```

При `HL=1..2047` bit 7 регистра H имеет специальное значение. Если он сброшен, успешный вызов также позиционирует файловый поток на начало объекта. Если нужен только заголовок без смены потока, установите bit 7 и передайте номер в остальных 15 битах. Каталоги и файлы определяются по внутреннему флагу записи.

7.14 API 56 `ADIR`: состояние активного каталога

Подкоманда передаётся через `A'`:

A'	Действие
0	Позиционировать активный каталог как поток с начала
1	Сбросить счётчики FindNext/NXTETY2 и восстановить активный каталог

```
        LD      A,1
        EX      AF,AF'
        LD      A,56
        CALL    #6006
```

Вызывайте подкоманду 1 перед первым API 58 и после любой операции, которая сбрасывает iterator: MKFILE, MKDIR, LOAD512, SAVE512 или FENTRY.

7.15 API 57 `STREAM`: создать и переключить поток

Поток — сохранённая копия драйвера, FAT geometry, каталога и текущей позиции. WC предоставляет два пользовательских потока, 0 и 1, на физических страницах `#F3/#F4`; исходный поток панели сначала нужно клонировать.

Специальные команды:

```
        LD      D,#FE                   ; clone panel stream → 0 и 1
        LD      A,57
        CALL    #6006

        LD      D,#FF                   ; root текущего клонированного потока
        LD      A,57
        CALL    #6006
```

Выбор существующего потока:

```
        LD      D,1
        LD      BC,#FFFF
        LD      A,57
        CALL    #6006                   ; флаги в этом режиме не определяют ошибку
```

Создание/пересоздание:

```
        LD      D,0                     ; номер потока
        LD      B,0                     ; SD-ZC
        LD      C,0                     ; раздел исторически не используется
        LD      A,57
        CALL    #6006
        JR      NZ,.mount_failed
```

Значения `B`: 0 SD-ZC, 1 Nemo master, 2 Nemo slave, 4 SMUC master, 5 SMUC slave. Команда `D=#FD` выбирает INI-поток и каталог WC; при ошибке A=1 означает устройство, A=2 каталог WC. Не вызывайте `D=#FF` или GDIR до клонирования.

7.16 API 58 `NXTETY2`: следующая запись каталога

Перед первым вызовом выполните ADIR с `A'=1`.

Вход	Значение
`DE`	Начало выходного буфера
`A' bit7`	Только SFN вместо LFN
`A' bit6`	В полях bit4/bit3 время и дата изменения, а не создания (с 2026-09-26)
`A' bit4`	Включить время, 2 байта: создания, с bit6 — изменения
`A' bit3`	Включить дату, 2 байта: создания, с bit6 — изменения
`A' bit2`	Включить size, 4 байта
`A' bits1..0`	0 всё, 1 файлы, 2 каталоги

Выход:

```
Z   конец каталога, буфер не содержит новой записи
NZ  [optional size][optional date][optional time][flag][name,0]
DE  адрес сразу после результата
```

`Z` бывает и при отказе чтения каталога: ядро ставит ноль в начало непрочитанного, и перечисление кончается на последней прочитанной записи. Отличает отказ `ABT` (`#384A` страницы, где шло перечисление) ≠0 сразу после вызова, до следующего обмена с носителем. Так делает обход дерева F5/F8 WC. Недопустимый первый кластер каталога (1, `#0FFFFFF0..#0FFFFFF7`) — тоже `Z` с `ABT=#FE` (с 2026-09-26: прежде NXTINI после отказа GIPAG всё же звал LOAD512, тот обнулял `ABT`, и каталог выглядел пустым). `ABT` не долговечен: однократный отказ при чтении наперёд, после которого запись уже прочитана, перечисление может выдать как `NZ`, а следующий обмен сбросит признак; вызывающему, которому нужна полнота, надёжно лишь `Z` с `ABT` ≠0.

Панель WC показывает и сортирует файлы по дате изменения, поэтому списку для человека нужен bit6. Старое ядро bit6 не знает и отдаёт создание в той же раскладке. Штамп, которого нет в календаре (месяц не 1..12, дня нет в месяце, часы больше 23, минуты или секунды больше 59), отдаётся как 2026-01-01 00:00: и дата, и время, даже если запрошено одно поле. Запись на носителе при этом не меняется.

Пример только файлов с размером:

```
.next:
        LD      DE,FindBuffer
        LD      A,%00000101          ; size + files
        EX      AF,AF'
        LD      A,58
        CALL    #6006
        JR      Z,.done
        ; FindBuffer+0..3 size, +4 type, +5 name
        JR      .next
```

Функция использует `#0000..#1FFF` файловой страницы. Результат копируйте до следующего файлового вызова. Для последующей функции имени нормализуйте flag: `AND #10`.

7.17 API 59 `FENTRY`: найти файл, каталог или путь

Это центральная поисковая операция. `HL` указывает на запрос:

```
Query:  DB      #00,"DIR\SUB\FILE.BIN",0
        ; slash и backslash распознаются как разделители пути
```

Выход	Значение
`Z`	Объект/компонент пути не найден
`NZ`	Объект найден
`DE:HL` при NZ	32-битный размер последнего объекта

Успешный FENTRY также:

- сохраняет кластер найденного объекта для GFILE/GDIR;
- копирует его ENTRY в менеджер;
- создаёт новый контекст APPEND/FILEX;

- при пути временно переходит через компоненты и затем восстанавливает исходный активный каталог.

Неуспешный FENTRY не очищает прежний APPEND-контекст. Поэтому всегда проверяйте Z перед API 76/77. FENTRY сбрасывает iterator API 58.

7.18 API 60 `LOAD256`: поток по 256 байт

Параметры аналогичны LOAD512:

Вход	Значение
`HL`	Буфер
`B`	Число секторов; каждый занимает 1024 байта адресов (`RDD256` делает `INC H` четырежды), буфер — `B×1024−256` байт и не должен переходить `#FFFF`

Вызов предназначался видеопроигрывателю и передаёт обмен драйверному RDD256. Nemo, SMUC и ZC его реализуют; SD-NGS возвращается без чтения. Для переносимого плагина используйте LOAD512 и самостоятельно разделяйте сектор.

С 2026-10-10 буфер проверяется до обмена (`LOAD256_CODE` резидентного блока): `HL` не ниже `#4000`, `B` — 1..64, последний байт `HL+1024·(B−1)+767` — не дальше `#FFFF` (SMUC пишет у сектора и вторую половину, по `HL+512`). Иначе обмена нет: `A=#FF`, CF=1, поток не сдвигается. Прежде DMA шёл и за `#FFFF` — в `#0000..` файловой страницы (NXTBU, GENBU), а при `HL` ниже `#4000` — в SECBU и LOBU, которые затем пишутся на носитель.

7.19 API 61 `LOADNONE`: пропустить поток

`HL/B` подаются как LOAD512, но данные не копируются. Значим прежде всего `B`; HL остаётся частью исторического ABI.

```
        LD      HL,0                    ; данные не требуются
        LD      B,8                     ; пропустить восемь секторов
        LD      A,61
        CALL    #6006
        JR      C,.stream_error
```

Это линейный seek: чтобы попасть на далёкий offset, ядро проходит FAT от текущей позиции. Для нового кода с 32-битным смещением используйте FILEX READ_AT/WRITE_AT.

7.20 API 62 `GFILE`: выбрать результат FENTRY как файл

```
        LD      HL,Query
        LD      A,59
        CALL    #6006
        JR      Z,.not_found

        LD      A,62
        CALL    #6006
        ; Следующий LOAD512 читает с начала найденного файла.
```

API использует сохранённый кластер последнего успешного FENTRY. Не вставляйте между FENTRY и GFILE другой поиск, который заменит этот кластер.

С 2026-10-10 поток ставит `GIPAG_FILE` (резидентный блок ядра). Кластер 0 — пустой файл: поток сразу в конце (`EOC=#0F`, NZ, CF=0 — как у GIPAG на метке конца), LOAD512 и SAVE512 отвечают `#0F` и ничего не читают и не пишут; прежде GIPAG понимал ноль как корень, и SAVE512 писал в корневой каталог. Первый кластер из цепочки корня — порча (файл на кластере корня): `ABT=#FE`, `EOC=#FF`, CF=1, поток не ставится. Переход потока по ссылке FAT на первый кластер корня (`BROOTC`) — тоже порча: цепочка файла или каталога ушла в корень, поток останавливается (`EOC=#FF`, CF=1), а не читает и не пишет корень.

7.21 API 63 `GDIR`: выбрать результат FENTRY как каталог

```
        LD      HL,DirQuery
        LD      A,59
        CALL    #6006
        JR      Z,.not_found
        LD      A,63
        CALL    #6006
```

После этого найденный cluster становится активным каталогом текущего потока. Нельзя выполнять GDIR в живом потоке панели: менеджер потеряет свой каталог. Сначала STREAM `D=#FE`, затем работайте в клоне.

7.22 API 72 `MKFILE`: создать файл

Входной блок:

```
+0      attr
+1..+4  requested size LE32
+5..    name,0
```

```
Create:
        DB      #00
        DD      0                       ; настоящий пустой файл
        DB      "NEW.DAT",0

        LD      HL,Create
        LD      A,72
        CALL    #6006
        JR      NZ,.create_error        ; A = код
```

При `size=0` создаётся cluster=0 без FAT-цепочки. При положительном размере MKFILE выделяет всю цепочку, но не заполняет пользовательские байты. После успеха поток указывает на начало нового файла, ENTRY доступна через API 51.

Исторические ошибки:

```
1 invalid long name       2 short-name generation failure
3 long name exists        4 short name exists
16 no space               255 unknown
```

7.23 API 73 `MKDIR`: создать каталог

Вход — только нуль-терминированное имя:

```
        LD      HL,DirName
        LD      A,73
        CALL    #6006
        JR      NZ,.mkdir_error

DirName: DB      "SAVES",0
```

CORE32 выделяет кластер, зануляет его и создаёт `.` / `..`. После успеха объект позиционирован «как файл», но не становится активным каталогом. Для входа выполните FENTRY с type `#10`, затем GDIR в клонированном потоке.

С 2026-09-25 тело каталога (сектор с `.` / `..` и обнулённый хвост кластера) пишется до записи в родителе; при отказе записи тела цепочка освобождается, и в родителе ничего не появляется (MKDIR_ENTRY расширения). Освобождение — DELCHA: код ошибки сохраняется, отказ самой записи FAT не проверяется, и тогда могут остаться потерянные кластеры (так при любом «цепочка освобождается» ниже, у MKDIR и MKFILE). Прежде запись публиковалась первой, и отказ записи тела оставлял каталог с мусором свободного кластера.

Отказ носителя при записи в родителе неоднозначен: сектор мог лечь, хоть драйвер и вернул ошибку. Тогда имя ищется заново — по NXTBU, то есть в том виде, в каком его записал SVHDFL (исходный текст «new » дал бы запись «NEW» и не нашёлся бы): запись есть и указывает на новый кластер — каталог создан, выход Z; поиск прошёл и записи нет — цепочка освобождается; поиск не удался или нашлась запись с чужим кластером — цепочка остаётся (потерянные кластеры), а не освобождается под живой записью. Отказ по имени или месту (ABT=0) — цепочка освобождается, как прежде. Код отказа носителя при записи каталога — `A=#FF` («UNKNOWN ERROR!!» в F7), прежде он совпадал с кодом недопустимого имени (1). Отказ записи FAT при выделении цепочки (MKSG) — тоже `#FF` (IO_RESULT, с 2026-09-26), и отказ чтения FAT при поиске свободного кластера — `#FF` (MKSG_CHECKED, прежде 16 — «Not Enough Space!»). Так же с 2026-09-25 поступает MKFILE (#4039, API 72): прежде `JP NZ,DELCHA` освобождал цепочку и под легшей записью.

7.24 API 74 `RENAME`: переименовать в том же каталоге

```
OldName: DB      #00,"OLD.TXT",0
NewName: DB      "NEW.TXT",0

        LD      HL,OldName
        LD      DE,NewName
        LD      A,74
        CALL    #6006
        JR      Z,.failed             ; not found или collision
        ; NZ = success
```

Первый запрос содержит type, второй — только новое имя. Этот исторический вызов работает в активном каталоге. Для переноса между каталогами и явного REPLACE используйте FILEX MOVE_RENAME.

Если прежнюю запись удалить не удалось, её короткая запись перечитывается по месту, запомненному при первом поиске (сектор и смещение, с 2026-09-25), и с 2026-10-10 добивается там же: до трёх записей `#E5` с перечитыванием — переименование доводится; откат — лишь если её не пометить.

Код в `A` при `Z` (его показывает окно F6 WC): `8` — источник не найден либо его поиск не прочитал каталог (тогда `CF=1`, как у исходного ядра); код SVHDFL (1–4 — имя, 16 — нет места) — новую запись не писали; `0` или `#0F` — отказ носителя, не переименовано, новой записи нет; `#FF` — исход неясен: откат не удался или новую запись не проверить, на цепочке могут остаться обе записи. `A=0` не значит «всё как было»: если удаление прежней легло на носитель, а её сектор не перечитался, откат оставляет цепочку без ссылок (потерянные кластеры, данные никому не отданы), а при частичном удалении у прежней записи уже стёрто длинное имя. Помечена удалённой (#E5) — уже была или помечена при доведении: новая остаётся, выход NZ, откат не снимает последнюю ссылку на цепочку. Не пометить и тремя записями, сектор не читается или место неизвестно — только что созданная запись удаляется, выход Z: потерянная цепочка лучше двух записей на одной. Поиск по прежнему имени для этого не годится: удаление пишет секторы с LFN раньше сектора SFN, и при отказе последнего длинное имя уже стёрто, а короткая запись жива. Атрибуты R/H/S/A переименованного объекта SVHDFL пишет в новую запись сразу (с 2026-10-10); прежде — отдельной записью сектора уже после удаления прежнего имени, и её отказ оставлял атрибут урезанным до бита каталога. `.` и `..` не переименовываются: `A=8`. Сектор прежней записи перечитывается до трёх раз; не удался и откат (новую удалить не вышло) — выход Z, `A=#FF`.

Запись нового имени (SVHDFL) отказала от носителя (ABT≠0) — она могла лечь, и тогда на цепочке уже две записи. Новое имя ищется по NXTBU: есть и смотрит на цепочку прежней — переименование доводится (прежняя удаляется); нет — «не переименовано»; поиск не удался или кластер чужой — «не переименовано», `A=#FF`. Отказ по имени (занято, недопустимо) носителя не касается: новую запись не писали.

С 2026-10-10 поиск новой записи после такого отказа (ENTRY_LANDED — его же зовут MKFILE и MKDIR) повторяется до трёх раз, если каталог не прочитался: одиночный отказ чтения сразу за легшей с ошибкой записью оставлял обе записи на одной цепочке и `A=#FF` (Codex, двойной отказ). Так же до трёх раз повторяется удаление новой записи при откате, если каталог не прочитался; отказ записи при откате не повторяется: удаление пишет LFN раньше SFN, и после частичной записи повторный поиск не нашёл бы имени при живой короткой записи — вышло бы «откатано» при двух записях; тут честнее `A=#FF`. Если SVHDFL отказал на испорченной цепочке каталога (`ABT=#FE` — это всегда до любой записи), ENTRY_LANDED сразу отвечает «записи нет»: прежде поиск по тому же каталогу отказывал, исход был «неизвестен», и цепочка, уже выделенная MKFILE или MKDIR, оставалась потерянной.

Короткое имя длинного файла (с 2026-09-25): расширение 8.3 кончается на конце имени и на недопустимом в 8.3 знаке. Прежде оно копировалось до трёх знаков без остановки: у «README.md» было «MD»+#00, у однобуквенного — ещё и байт буфера за концом имени (так в исходном WC; касалось всего, что создаёт записи: F5, F6, F7, MKFILE плагинов). Основа 8.3 (8 знаков подряд) у имени короче восьми тоже брала байты за его концом — остатки прежнего имени в буфере вызывающего («A» → «AZPRIV~1»): теперь копия имени в NXTBU после нормализации дополняется нулями, и недостающие знаки основы — случайные допустимые, как и прежде для недопустимых знаков.

Длинное имя при чтении каталога (NXTETY2) собирается не длиннее 255 знаков: повреждённый каталог с 20 записями LFN по 13 знаков без терминатора (260) прежде писал за 256-байтовое поле имени вызывающего.

7.25 API 75 `DELFL`: удалить файл или каталог

```
DeleteQuery:
        DB      #00,"TEMP.BIN",0

        LD      HL,DeleteQuery
        LD      A,75
        CALL    #6006
        JR      Z,.not_found_or_failed
        ; исторически NZ = удалено
```

Функция удаляет directory entry и освобождает цепочку. Она не запрещает удаление непустого каталога. Для каталога сначала отдельно докажите пустоту. `.` и `..` не удаляются (с 2026-10-10): `Z`, `A=0`, на диск ничего не пишется (прежде DELETE `.` освобождал цепочку самого каталога, `..` — родителя). `Z` — объект не найден либо поиск не прочитал каталог (A=8; во втором случае CF=1), не удалось записать каталог (A=0) или, с 2026-09-25, запись уже удалена, а цепочку освободить не удалось (A=0): прежде этот случай возвращал NZ «успех». После любой записи носителя не трактуйте один `Z` как полную транзакционную гарантию. Если плагин удалил каталог и вернул код 3, соседняя панель перечитывается с проверкой, не стояла ли она в удалённом каталоге.

7.26 API 76 и 77

API 76 APPEND и API 77 FILEX являются такими же публичными DOS-вызовами, как исторические функции выше. Их полные блоки, коды и примеры находятся в разделе APPEND и разделе FILEX.

8. Внутренняя таблица `#4000`

Таблица содержит 31 трёхбайтовый `JP`. Она нужна коду WC, а не обычному плагину.

Адрес	Offset	Имя	Контракт
`#4000`	`+#00`	reserved	Пустой RET; SELDEV находится в `#3800`
`#4003`	`+#03`	`DEVINI`	Инициализировать драйвер, обычно Z=успех
`#4006`	`+#06`	reserved	Пустой RET; REINI находится в `#3806`
`#4009`	`+#09`	`HDD`	Найти FAT32 и заполнить geometry
`#400C`	`+#0C`	`RDD`	`DE:HL=LBA`, `BC=buffer`, `A=sectors`; Z/NZ по ABT
`#400F`	`+#0F`	`SDD`	То же для записи
`#4012`	`+#12`	`GIPAG`	`HL` → cluster32, ноль=root
`#4015`	`+#15`	`LOAD512`	`HL=buffer`, `B=sectors`, потоковое чтение
`#4018`	`+#18`	`SAVE512`	Потоковая запись существующей цепочки
`#401B`	`+#1B`	`DOS_SWP`	`A=0..6`, выбрать/распаковать драйвер
`#401E`	`+#1E`	`GLSTCAT`	Копирует cluster32 из `HL` в `LSTCAT`
`#4021`	`+#21`	`TLSTCAT`	Копирует текущий `LSTCAT` в четыре байта по `DE`
`#4024`	`+#24`	`SRHFCL`	Найти свободный кластер; DE:HL/Z или CF=1
`#4027`	`+#27`	`APPEND`	Внутренний вход публичного API 76

Адрес	Offset	Имя	Контракт
`#402A`	`+#2A`	`MKSG`	`DE:HL>0` размер цепочки; нулевой размер запрещён
`#402D`	`+#2D`	`FILEX_GATE`	Шлюз API 77 к провайдеру FILEX.WMF
`#4030`	`+#30`	`RFRH`	Проверить/обновить FSInfo
`#4033`	`+#33`	`GENTRY`	Копирует 32 байта из `HL` во внутреннюю ENTRY
`#4036`	`+#36`	`TENTRY`	Копирует ENTRY в `DE`, фиксирует APPEND-контекст
`#4039`	`+#39`	`MKFILE`	`[attr,size32,name,0]`
`#403C`	`+#3C`	`MKDIR`	`[name,0]`
`#403F`	`+#3F`	`DELFL`	`[type,name,0]`
`#4042`	`+#42`	`RENAME`	старый запрос в HL, новое имя в DE
`#4045`	`+#45`	`DLSG`	`HL` → cluster32, освободить цепочку
`#4048`	`+#48`	`CHTOSE`	cluster32 → массив LBA32
`#404B`	`+#4B`	reserved	Пустой RET
`#404E`	`+#4E`	`SRHDRN`	Найти `[type,name,0]`, NZ=найдено
`#4051`	`+#51`	`LOAD256`	Поток через RDD256
`#4054`	`+#54`	`LOADNON`	Пройти `B` секторов без копирования
`#4057`	`+#57`	`NXTETY`	Следующая полная запись каталога
`#405A`	`+#5A`	`NXTETY2`	Следующая запись по маске в A

Исторические имена `GLSTCAT/TLSTCAT` и `GENTRY/TENTRY` легко прочитать наоборот. Ориентируйтесь на направление фактического `LDIR`, указанное в таблице.

8.1 Как пользоваться внутренними входами

Внутренний вызов допустим, только когда:

1. нужная panel/stream page подключена в `#0000`;
2. CORE32 установлен по `#4000`;
3. драйвер выбран и том смонтирован, если функция работает с носителем;
4. все указатели относятся к памяти, которая останется видимой во время вызова;
5. на каждом пути возврата страница `#0000` будет восстановлена.

В отличие от публичного API здесь основной `A` не занят номером функции: параметры передаются прямо в обычных регистрах, а адрес выбирается самим операндом `CALL #40xx`.

8.2 `#4000` и `#4006`: пустые слоты

Оба выполняют только RET. Они оставлены для сохранения трёхбайтовой нумерации. Настоящие драйверные SELDEV/REINI находятся по `#3800/#3806` уже внутри подключённой файловой страницы.

8.3 `#4003 DEVINI`

Инициализирует текущий распакованный драйвер.

```
        CALL    #4003
        JR      NZ,.device_not_ready
```

Параметров общего уровня нет. Обычно `Z` означает готовность. Конкретный драйвер может изменить большинство регистров, поэтому сохраняйте нужные сами. DEVINI не ищет FAT32 — после него нужен HDD.

8.4 `#4009 HDD`

Ищет MBR/superfloppy FAT32, читает BPB и заполняет все geometry variables.

```
        CALL    #4009
        JR      NZ,.fat32_not_found
```

Перед вызовом драйвер должен быть выбран и DEVINI должен завершиться успешно. HDD использует LOBU, сбрасывает current cluster/catalog/stream, FSTFRC и iterator. `Z` означает, что файловые операции можно начинать.

8.5 `#400C RDD` и `#400F SDD`: абсолютные сектора

Это прямой блочный доступ без FAT-потока.

```
        LD      DE,#0000
        LD      HL,#1234            ; LBA = #00001234
        LD      BC,Buffer           ; адрес буфера
        LD      A,1                 ; один сектор
        CALL    #400C               ; RDD
        JR      NZ,.media_error
```

SDD имеет те же параметры и пишет данные. Функции вызывают XPOZI, затем SAFE_RDDSE/SAFE_SDDSE. Возвращаемый `A` равен общему `ABT`: ноль/Z — успех, неноль/NZ — отказ. Это опасный raw I/O: он не знает границ раздела, FAT и файлов. Не используйте его в прикладном плагине.

Об отказе знает только драйвер, который ставит `ABT`. Штатные драйверы `boot.$C` (Nemo, SD-ZC, SD-NGS, SMUC) его не ставят никогда: SD-ZC при ошибке чтения ждёт без предела, а отвергнутую картой или запрещённую защитой запись молча считает выполненной. Поэтому всё описанное здесь про отказы носителя — откаты RENAME, MKDIR/MKFILE, F5, APPEND, FILEX, проверки ENTRY_LANDED — действует с драйвером, сообщающим ошибки; со штатными драйверами эти ветки не срабатывают, а отказ записи остаётся незамеченным. Штатная сборка такого драйвера не содержит: экспериментальный SD-ZC из `source/DSDZC.ASM` рассчитан на особую сборку расширения (номера шлюза в нём зашиты числами), `build.ps1` её не делает, и сообщает ли он ошибки на деле, не проверялось.

С 2026-10-11 SD-ZC сообщает об одном отказе — вынутой карте. После распаковки драйвера (DOS_SWP) входы его таблицы RDDSE, SDDSE и RDD256 (`#380F`, `#3812`, `#3815`) ведут в резидентный блок ядра (ZCG_*); байты кода драйвера прежние. Перед обменом — CMD16 (длина блока 512); перед командой, как CSL драйвера, — ожидание готовности (`#FF`, до 65536 опросов): RDD256 кончает CMD12 без WTBY, и занятая карта не должна сойти за вынутую; занята дольше предела — тоже «не отвечает». Карта не ответила за 10 байт — её нет или её вставили заново и она ещё не в режиме SPI:

- не загрузочная карта — отказ обмена: `ABT=1`, NZ, драйвер не зовётся;

- загрузочная — красное окно «Insert boot drive» и попытки SDINI, пока карта не ответит. Слот, серийный номер тома и сектор его BPB запоминаются при первом монтировании после старта (BOOT_DRIVE_NOTE; сектор — свой, а не текущий ADDTOP: при перемонтировании тот уже сброшен). Ответившая карта сверяется по серийному номеру BPB: чужая — «Wrong drive - insert boot drive», пока её не вынут; своя — кэш FAT сбрасывается, окно убирается, позиция драйвера (BLKNUM) возвращается прежняя. BPB читается в свою страницу Improved `#EA` (`WC_PHYS_MEDIA_PAGE`, окно `#8000`; прежняя страница окна возвращается), а не в буферы ядра: там мог ждать записи сектор FAT, и повтор обмена записал бы на его место BPB. Сверка — всегда: прежде BPB клался в пул фона окон WC, и при почти полном пуле её пропускали — чужая карта принималась своей. Дальше:
- до первого прохода главного цикла (старт WC: `wc.ini`, плагины) и на время перемонтирования в главном цикле прерванный обмен просто повторяется (тем же буфером и с той же позицией);
- потом — этот и все следующие обмены с ней до главного цикла отказывают (`MEDIA_FENCE`): прерванная операция и её откат не пишут по прежним номерам кластеров — карту могли менять на ПК. Главный цикл (MAIN, вход MEDIA_IDLE вместо CURSOR) перемонтирует каждую панель на загрузочной карте, как выбор диска (корень), и перерисовывает строку состояния второй панели. Перемонтирование только читает, поэтому вынутую ещё раз во время него карту забор не закрывает — иначе цикл выбора диска к главному циклу не вышел бы.

Загрузчик SPG при таком отказе программу не запускает: окно «SPG load failed - press reset» и останов (docs/05, 5.2).

Карту, вынутую посреди обмена (между проверкой и концом команды), драйвер по-прежнему ждёт без предела. В графическом режиме плагина окно рисуется в текстовой странице WC и на экране не видно; вставленная карта снимает ожидание так же.

8.6 `#4012 GIPAG`

`HL` указывает на четырёхбайтовый номер кластера:

```
Cluster:
        DD      5

        LD      HL,Cluster
        CALL    #4012
        JR      NZ,.bad_or_eoc
```

Нулевой cluster означает root. GIPAG сбрасывает `NSDC/EOC`, запоминает текущий кластер, вычисляет первый data LBA и вызывает драйверный XPOZI. На успехе Z. Передача EOC/reserved/bad cluster является ошибкой контекста.

8.7 `#4015 LOAD512` и `#4018 SAVE512`

Прямой потоковый ABI совпадает с публичными API 48/49: `HL=buffer`, `B=count`. Но прямой вход не выполняет плагинную трансляцию страниц. Буфер должен быть видим при текущей карте памяти.

LOAD512 временно использует `NW0=SAFE_RDDSE`. SAVE512 меняет его на SAFE_SDDSE, затем общий эпилог возвращает обработчик чтения. Из-за глобального NW0 оба входа нельзя вкладывать или вызывать параллельно.

8.8 `#401B DOS_SWP`

Принимает `A=0..6`, выбирает packed driver, распаковывает его по `#3800` и вызывает SELDEV. Подробная нумерация находится в руководстве драйверов.

```
        LD      A,1                     ; первый SD-ZC
        CALL    #401B
        CALL    #4003
        JR      NZ,.no_device
        CALL    #4009
        JR      NZ,.no_fat32
```

Распаковка заменяет общее состояние по `#3848`, поэтому старый поток после DOS_SWP недействителен.

8.9 `#401E GLSTCAT` и `#4021 TLSTCAT`

Несмотря на названия, направление такое:

```
        ; SET current catalog
        LD      HL,ClusterValue         ; HL указывает на 4 байта
        CALL    #401E

        ; GET current catalog
        LD      DE,ClusterCopy          ; DE указывает на 4 байта
        CALL    #4021
```

GLSTCAT копирует четыре байта в `LSTCAT` и сбрасывает iterator. TLSTCAT копирует `LSTCAT` наружу. Cluster 0 означает root при последующем GIPAG.

8.10 `#4024 SRHFCL`

Ищет свободный data cluster, начиная с `FSTFRC`, с одним циклическим обходом допустимого диапазона.

При монтировании диапазон `FSI_Nxt_Free` проверяется, затем единственным чтением FAT подтверждается, что указанный кластер действительно свободен. Неизвестное или недопустимое значение не запускает холодный поиск от кластера 2: read-only RAM-курсор устанавливается в точку 15/16 области данных. Занятое — с 2026-10-11 вторым чтением проверяется и точка 15/16: свободна — поиск от неё, занята и она — от самой подсказки, как велит спецификация FAT32 (свободное обычно сразу за последней записью). FSInfo пишет Windows, WC его на обычных путях не обновляет, поэтому после записей WC в прежнем сеансе подсказка указывает на его же файлы; прежде такой том искал место от 15/16 до конца и с возвратом через всю FAT от кластера 2 — у пользователя первая после загрузки запись `wc.ini` шла около минуты. Если свободного места нет и там, обычный обход доходит до конца и один раз возвращается к кластеру 2, поэтому допустимый диапазон всё равно проверяется полностью.

Записи одного сектора FAT с 2026-10-11 проверяет расширение одним вызовом шлюза (`SCAN_FREE_SECTOR`, ID 4): число записей до конца сектора, до границы данных и — после возврата к 2 — до начала поиска считается один раз, затем плотный цикл. Прежде шлюз (`CHECK_FREE_SCAN_LIMIT`) звался на каждую запись, ~2330 тактов на кластер; полный обход FAT в эмуляторе (≈1400 секторов) — 48 с, теперь около 4 с. Прерывания закрыты на время одного сектора (до 128 записей), а не на каждую запись.

```
success: CF=0, Z, DE:HL = cluster
failure: CF=1, свободного кластера или корректного FAT-доступа нет
```

Это building block аллокатора, а не функция резервирования: найденная FAT запись ещё не становится занятой сама по себе. Прикладному коду нужен MKFILE, MKDIR, APPEND или FILEX.

8.11 `#4027 APPEND` и `#402D FILEX_GATE`

Это внутренние цели публичных API 76/77. Они ожидают уже подключённую panel page. FILEX_GATE дополнительно подключает зарегистрированную страницу провайдера в `#C000` и восстанавливает её. Не вызывайте provider runtime `#C010` напрямую.

8.12 `#402A MKSG`

Создаёт FAT-цепочку под положительный размер `DE:HL`:

```
        LD      DE,0
        LD      HL,4096
        CALL    #402A
        JR      NZ,.allocation_failed
        ; первый cluster находится во внутреннем FCTS
```

Нулевой размер этому внутреннему входу запрещён. Он собирает список кластеров в GENBU, связывает FAT и обновляет FSTFRC. Успех — `Z` и `CF=1` (конец BUtoFAT). Если места не хватает, возвращает `NZ/A=16` и пытается освободить уже собранную часть. Тот же код 16 — при отказе чтения FAT во время поиска свободного кластера (после частичного выделения освобождение ещё и сбрасывает `ABT`). MKDIR и APPEND зовут MKSG через MKSG_CHECKED расширения и отдают в этом случае `#FF` (с 2026-09-26). MKFILE с 2026-09-27 зовёт MKSG сам, вне шлюза расширения, а итог разбирает ALLOCATE_FILE тем же правилом: внутри шлюза прерывания выключены, и выделение файла в гигабайт (~4900 тактов на кластер) шло при DI минуту и дольше. MKDIR, APPEND и рост каталога (DIR_GROW) с того же дня ищут место тоже при прерываниях и странице вызывающего — через резидентный блок `CORE_MKSG` / `CORE_SRHFCL` (раздел 2). Каждая запись FAT по-прежнему проверяется коротким вызовом шлюза (~2330 тактов); прерывания обслуживаются между ними. Обычно вызывайте MKFILE, который правильно обрабатывает size=0. После успешного выделения найденная позиция остаётся в FSTFRC текущего сеанса; MKFILE не выполняет отдельную запись FSInfo, поскольку этот дополнительный I/O зависал на реальной SD-ZC.

8.13 `#4030 RFRH`

Проверяет сигнатуры FSInfo и безопасно обновляет его. На успехе Z, при плохих сигнатурах или I/O — NZ, причём плохой сектор не перезаписывается. Это внутренняя часть commit, а не команда точного пересчёта всего свободного места.

8.14 `#4033 GENTRY` и `#4036 TENTRY`

```
        LD      HL,EntrySource
        CALL    #4033               ; 32 bytes: caller → internal ENTRY

        LD      DE,EntryDestination
        CALL    #4036               ; 32 bytes: internal ENTRY → caller
```

TENTRY дополнительно сохраняет APPEND context (SFN, directory sector/offset, size/cluster). Поэтому это не просто memcpy. Оба используют LDIR и изменяют регистры/обычные флаги пересылки; TENTRY отдельно восстанавливает заявленные флаги менеджера вокруг копии. Времена в записи оба переносят как есть, без проверки календаря (нормализует только API 58 и FILEX GET_METADATA).

8.15 `#4039 MKFILE`

Вход `HL` указывает на `[attr,size32,name,0]`. На успехе Z; для положительного размера `DE:HL` возвращает/делает доступным первый cluster, ENTRY заполнена. Нулевой файл создаётся без цепочки. Цепочку выделяет MKSG, с 2026-09-27 — прямо из MKFILE, при прерываниях вызывающего (8.12); хвост MKFILE (SVHDFL) стоит за MKDIR — вход MKDIR `#4D5B` закреплён. Внутренний вызывающий обязан сам передать актуальную ENTRY/current directory context, что публичная обёртка API 72 делает автоматически. Отказ носителя при записи в родителе (с 2026-09-25): запись ищется заново (ID_ENTRY_LANDED, запрос — нормализованный NXTBU); легла с нашим кластером — успех, нет — цепочка освобождается (`A=#FF`), неизвестно — цепочка остаётся (`A=#FF`), см. 7.23.

8.16 `#403C MKDIR`

`HL` указывает на `name,0`. Выделяет cluster, пишет `.` / `..` и зануляет хвост кластера, затем создаёт directory entry (тело — до публикации, с 2026-09-25; сама процедура — MKDIR_ENTRY расширения, вход прежний). Z — успех, NZ/A — ошибка (16 — нет места, 1–4 — имя, из них 2 — исчерпаны суффиксы ~N короткого имени; #FF — отказ носителя при записи каталога и при записи FAT во время выделения цепочки (MKSG): расширение отдаёт вместо сырого кода драйвера #FF, F7 пишет «UNKNOWN ERROR!!»; прежде код 1 давал «Long Name is not valid!»); при отказе записи FAT цепочка освобождается (попытка: при отказе записи FAT могут остаться потерянные кластеры), кроме отказа носителя, после которого не удалось выяснить, легла ли запись в родителе (см. 7.23). После вызова iterator и часть секторных буферов недействительны.

8.17 `#403F DELFL`

`HL` указывает на `[type,name,0]`. Находит SFN/LFN, освобождает цепочку и помечает записи удалёнными. Не проверяет пустоту каталога. Исторический ABI не даёт однозначного статуса для всех поздних ошибок записи, поэтому новый код не должен использовать его как атомарную операцию.

8.18 `#4042 RENAME`

`HL` → `[type,old,0]`, `DE` → `new,0`. Работает внутри активного каталога. Проверяет collision и создаёт новый набор LFN/SFN. Исторически NZ означает успех, Z — источник не найден либо имя уже существует. С 2026-09-25 Z и при отказе носителя: `A=0` или `#0F` — не переименовано, `#FF` — исход неясен; `A=8` с `CF=1` — поиск источника не прочитал каталог (коды и оговорки — в 7.24).

8.19 `#4045 DLSG`

`HL` указывает на cluster32. Проходит FAT-цепочку и обнуляет каждую запись, сбрасывает FSTFRC. У функции нет стабильного прикладного статуса; ошибки SDDSE исторически могут быть видны только через ABT. Используется как часть компенсации и удаления, не как публичная команда. Каждую ссылку проверяет ID_CLASSIFY_DATA расширения: недопустимая (1, `#0FFFFFF0..#0FFFFFF7`), за областью данных тома и, с 2026-09-27, кластер цепочки корня — остановка с FAT_LINK_ERROR (`ABT=#FE`), освобождённое до неё записано. Таблицу кластеров корня (до 768 — корень FAT32 при кластере от 4 КиБ целиком — с фильтром по младшим 11 битам номера, чтобы не просматривать таблицу на каждый кластер) DLSG

строит в самом начале (ID_NOTE_FREED_CHAIN → ROOT_BUILD): одно чтение сектора FAT на корень в один кластер; сектор FAT корня не читается — DLSG не освобождает ничего. Прежде запись файла на кластере корня (порча) F8 освобождала корень, на втором и следующих — хвост корня с его именами.

8.20 `#4048 CHTOSE`

`HL` → cluster32, `DE` → массив LBA32, `BC` → адрес конца. Для каждого кластера пишет `BSECPC` последовательных LBA, затем читает следующую FAT link. Буфер должен вместить целый последний cluster. Отдельного count результата нет.

8.21 `#404B`: пустой слот

Пустой RET, сохранён для размера таблицы. Не является скрытой файловой операцией.

8.22 `#404E SRHDRN`

Ищет имя/путь `HL → [type,name,0]` в текущем каталоге.

```
Z   не найдено
NZ  найдено; internal ENTRY заполнена, DE:HL содержит first cluster/результат
```

Это нижний механизм FENTRY. Прямой вход не выполняет менеджерское сохранение пути, кэша и APPEND context; для плагина нужен API 59.

8.23 `#4051 LOAD256` и `#4054 LOADNON`

Оба используют общий потоковый цикл с временным обработчиком:

- LOAD256: `DE=RDD256`, фактически читает 256-байтовые порции;
- LOADNON: `DE=Z0`, продвигает поток без копирования.

Вход `HL/B` и потоковые A/CF соответствуют историческим API 60/61. Эпилог восстанавливает NW0 даже при ошибке.

8.24 `#4057 NXTETY` и `#405A NXTETY2`

Обе функции используют глобальный iterator и `#0000..#1FFF`.

- NXTETY принимает `DE` и выдаёт полную нормализованную запись переменной длины, которую использует панельный менеджер; у SFN файлов и каталогов конечное пробельное дополнение трёхбайтового расширения удаляется одинаково;
- NXTETY2 принимает дополнительно маску в обычном `A` и выдаёт документированный набор size/date/time/flag/name (bit6 маски — время и дата изменения вместо создания; штамп, которого нет в календаре, — 2026-01-01 00:00, см. 7.16).

NXTETY отдаёт время и дату изменения как на носителе: по ним панель WC показывает строку информации и сортирует по дате, и замена на 2026-01-01 исказила бы и то и другое. Программе нужны API 58 или FILEX GET_METADATA.

`NZ` означает новую запись, `Z` — конец каталога либо отказ чтения (тогда `ABT` ≠0, см. 7.16). После GDIR/GLSTCAT iterator сброшен; после файлового I/O его также следует сбросить явно.

Каталог читается в `#0000..#1FFF` по 8 КиБ, дальше — по 4 КиБ (NXMR). С 2026-07-16 до 2026-09-26 NXMR в конце цепочки отдавал `NZ` без новой записи: полный каталог (записи до конца последнего кластера, нулевой нет) размером 8 КиБ, 12 КиБ и т. д. перечислялся без конца — панель набирала записи до переполнения таблицы. Теперь там `Z`, как у нулевой записи.

Перед прямым вызовом рабочая panel page должна стоять в окне `#0000`. В текущем менеджере это делает одноуровневая пара `CORE_PAGE_IN=#6B7A` и `CORE_PAGE_OUT=#6B8B`. Она хранит только одну прежнюю страницу, поэтому вложенный вход запрещён:

```
        CALL    #6B7A               ; current internal build only
        CALL    #4039               ; например, MKFILE
        PUSH    AF                  ; сохранить результат
        CALL    #6B8B
        POP     AF
```

Для нового внутреннего кода лучше вызывать именованные символы сборки, а не фиксировать эти два адреса. Для плагина этот пример запрещён — используйте API 72.

9. API 76 APPEND

APPEND дописывает `0..#4000` байт в файл, последним успешно найденный API 59.

9.1 Обязательная последовательность

```
FENTRY 59
    │
    ├─ Z ──► STOP: старый контекст всё ещё существует
    │
    └─ NZ
       ▼
APPEND 76 (HL=data, BC=length)
       │
       ├─ Z,A=0 ──► новый EOF опубликован
       └─ NZ,A=error ──► не повторять блок вслепую
```

`GFILE` перед APPEND не нужен. Успешный FENTRY сохраняет положение SFN и EOF. Последовательные APPEND используют кэш последнего кластера и его базового LBA.

9.2 Буфер и результат

Поле	Контракт
`HL`	Начало данных
`BC`	`0..#4000` байт
Диапазон	Весь `[HL,HL+BC)` внутри `#4000..#C000`
Успех	`Z`, `A=0`, HL сразу за данными при `BC>0`
Ошибка	`NZ`, код в A; фактическое число байт не возвращается

`BC=0` всегда успешно и не проверяет контекст. Это не probe.

9.3 Внутренний commit

```
проверить FENTRY context / вычислить EOF
          ├─ при необходимости заранее выделить всю новую цепочку
          ├─ записать данные (RMW неполных секторов)
          ├─ связать старый хвост с новой цепочкой
          ├─ обновить FSInfo
          └─ последней записать SFN cluster + size32
```

Новая цепочка до commit остаётся отсоединённой. При последующей ошибке код пытается вернуть прежний EOC, освободить новую цепочку и обновить FSInfo. Это best-effort компенсация, а не журналируемая транзакция.

9.4 Коды APPEND

Код	Значение
`#20`	Нет действительного контекста FENTRY
`#21`	Неверный диапазон буфера
`#22`	Длина больше `#4000`
`#23`	SFN ENTRY изменилась после FENTRY
`#24`	Переполнение размера `#FFFFFFFF`
`#25`	Недопустимая/непрочитанная FAT-цепочка
`#26`	Объект является каталогом
`#27`	Внутренне противоречивый commit
`#28`	Файл «только для чтения» (с 2026-10-10)
`#2A`	Компенсирующий rollback сам завершился ошибкой

`#25` — и цикл в цепочке, и кластер вне области данных тома или кластер корня: первый вызов после FENTRY проверяет кластеры до EOF и путь от кластера EOF до конца цепочки (не больше 255 ссылок). Неизвестный исход записи каталога или неудавшийся откат снимают контекст: следующий вызов — `#20`.

Также наружу могут пройти `#10`, `#01` и другие ненулевые коды. Любой `NZ` считается ошибкой. После неоднозначного сбоя заново выполните FENTRY и проверьте реальный размер/содержимое перед решением о повторе.

10. API 77 FILEX

FILEX — версионируемый 32-байтовый интерфейс для позиционных операций. Его провайдер `FILEX.WMF` должен быть первой активной строкой `[PLUGINS]`. Тип `#06` при старте публикует свою страницу в резидентном шлюзе. Без провайдера API возвращает `#15 NO_CONTEXT`.

10.1 Вызов

```
        LD      HL,FilexBlock
        LD      A,77
        CALL    #6006
        JR      Z,.ok                   ; только status #00
        ; A и block.status содержат один код.
```

Блок, имена, данные и выходные структуры должны целиком лежать в `#8000..#BFFF`, не пересекать друг друга и не выходить за `#C000`. Максимум одного READ_AT/WRITE_AT — `#4000` байт. AF/BC/DE/HL/IX разрушены, IY сохранён.

10.2 Блок версии 1

```
offset  size  field
  0      1    size = 32
  1      1    version = 1
  2      1    operation 0..7
  3      1    flags
  4      4    offset / new EOF, LE32
  8      2    buffer
 10      2    length
 12      2    aux
 14      2    aux_length
 16      4    source_dir cluster
 20      4    dest_dir cluster
 24      4    result_count
 28      1    status
 29      1    result_flags
 30      2    reserved = 0
```

Перед каждым вызовом занулите все 32 байта. Это автоматически выполняет требование к reserved и не оставляет result предыдущей операции.

10.3 Операции

op	Имя	Нужно FENTRY	Поля	Результат
0	`QUERY_CAPS`	Нет	flags=0	`result_count.low=#FF`, `result_count+1` bit0 = GET_METADATA, result_flags=ABI 1
1	`READ_AT`	Да	offset, buffer, length	result_count = реально прочитано
2	`WRITE_AT`	Да	offset, buffer, length	result_count = записано, файл при необходимости вырос
3	`SET_EOF32`	Да	offset = новый размер	result_count = итоговый EOF
4	`GET_FS_INFO`	Нет	buffer, length>=48	Геометрия/FSInfo/label
5	`MOVE_RENAME`	Нет	два имени, два parent clusters, optional REPLACE	Перенос/переименование
6	`SET_METADATA`	Да	buffer, length>=16	Атрибуты и FAT timestamps
7	`READ_FAT`	Нет	offset, buffer, length	Секторно выровненное окно используемой FAT
8	`GET_METADATA`	Да	buffer, length>=16	Атрибут и FAT timestamps в структуре SET_METADATA (с 2026-09-26)

READ_AT

- не прогоняет все предыдущие секторы через LOADNONE;
- вычисляет требуемый кластер/сектор напрямую;
- offset за EOF возвращает `EOF=#01`, `result_count=0`;
- частичное чтение до EOF возвращает `#01`, но result_count содержит префикс;
- нулевая длина возвращает OK.

WRITE_AT

- выполняет read-modify-write неполных секторов;
- если offset выше старого EOF, заполняет разрыв нулями;
- до первой изменяющей записи проверяет всё место для роста;
- новый размер публикуется после данных, FAT и FSInfo;
- перезаписанные байты внутри старого EOF при позднем I/O-сбое не откатываются.

SET_EOF32

```
new EOF < old EOF   truncate + zero sector tail + detach/free chain tail
new EOF = old EOF   no-op
new EOF > old EOF   preflight space + allocate + zero extension + publish
new EOF = 0         empty file, first cluster 0
```

При truncate новая ENTRY публикуется до освобождения лишнего хвоста, хвост последнего сектора зануляется после неё (с 2026-10-10). Если очистка после commit не завершилась, возвращается `#25 COMMITTED_CLEANUP`: новый размер уже действителен.

MOVE_RENAME

Оба имени имеют формат `[type,name,0]` и переданную длину `3..257`. Parent cluster `0` означает root. Для других каталогов вызывающий должен сохранить их кластер из ENTRY/навигации.

`flags bit0=REPLACE` разрешает заменить существующее назначение. Типы должны совпадать. Непустой каталог не заменяется; каталог нельзя перенести в своего потомка. При смене родителя обновляется запись `..`. Отказ записи перенаправленной SFN назначения (с 2026-09-26) откатывается: сектор перечитывается, и если новая запись легла, прежняя запись назначения возвращается; если не легла — возвращать нечего; статус `MEDIA`. Прежде легшая с ответом «ошибка» запись оставляла на цепочке источника две записи, а старая цепочка назначения терялась. С 2026-10-10 заменяемый каталог не может быть текущим (`INVALID_MOVE`), его цепочка должна быть конечной, а цепочки источника и назначения — не пересекаться (`FAT`).

Имя источника или назначения с пробелом или точкой в конце, в том числе `.` и `..` (с 2026-09-25), — `FILEX_STATUS_INVALID_NAME` до каких-либо изменений: SVHDFL записывал его усечённым («NEW » → NEW), а поиск и откат MOVE шли по исходному тексту и записи не находили — на объект оставались две ссылки даже без сбоя носителя.

Откат (с 2026-09-25): признак «`..` изменён» ставится до записи сектора — при отказе, после которого сектор всё же лёг, откат возвращает прежнего родителя (вернуть неизменённый `..` безвредно). Ссылка назначения удаляется (или запись назначения восстанавливается при REPLACE) только после того, как `..` вернулся; не вернулся (или не удалось удалить саму ссылку назначения) — статус `FILEX_STATUS_ROLLBACK`: откат не подтверждён. Обычно на объект остаются две ссылки, но если удаление источника легло, а его сектор не перечитался, — одна или ни одной; куда смотрит `..`, по

статусу тоже не узнать. Прежде результат возврата `..` отбрасывался, и каталог мог остаться в прежнем родителе с `..` на новый: подъём по `..` (рекурсивное удаление) искал бы имя в чужом каталоге. Отказ самой записи ссылки назначения тоже идёт через откат: ссылка могла лечь, и прежде на объект оставались обе ссылки; до MOVE имени назначения не было, поэтому найденная по нему запись — своя и удаляется.

Отказ удаления исходной ссылки тоже неоднозначен: запись могла лечь. Прежде FILEX искал источник по имени и, не найдя, фиксировал перенос (`COMMITTED_CLEANUP`). Но удаление пишет секторы LFN раньше сектора SFN: при отказе последнего длинное имя уже стёрто, а короткая запись жива, и поиск её не находит (как и при ошибке чтения) — на объект оставались две живые ссылки. С 2026-09-25 сектор SFN источника перечитывается по месту, запомненному при поиске (до трёх попыток): `#E5` — перенос зафиксирован (`COMMITTED_CLEANUP`); запись жива — с 2026-10-10 она помечается `#E5` по месту (до трёх записей с перечитыванием), и перенос тоже фиксируется; не пометить или сектор не читается — `MEDIA` и откат. Каждый шаг отката (возврат `..` , удаление ссылки назначения по имени и по месту, возврат прежней записи назначения) — до трёх попыток, и любой выход отката снимает контекст FENTRY/APPEND: он мог быть уже взят по новой ссылке. Если удаление всё же легло, а перечитать не удалось, после отката ссылок на объект не остаётся (потерянные кластеры) — так же, как у RENAME ядра: это лучше общей цепочки.

`flags bit1=CURRENT_DIR` означает, что оба имени находятся в уже активном каталоге WildDOS. В этом режиме `source_dir` и `dest_dir` игнорируются. Возможность объявляется битом `FILEX_CAP_MOVE_CURRENT_DIR` ; старый провайдер без этого бита продолжает принимать прежний явный cluster32 ABI.

SET_METADATA

16-байтовая структура:

Offset	Поле
0	size=16
1	attr_mask
2	attr_value
3	time_mask: bit0 create, bit1 access, bit2 write
4	create_tenth
5	create_time LE16
7	create_date LE16
9	access_date LE16
11	write_time LE16
13	write_date LE16
15	applied_attr, выход

Разрешённые атрибуты mask `#27` : read-only, hidden, system, archive. С read-only файла разрешено сначала снять сам read-only; остальные изменения до этого возвращают `#18` .

С 2026-09-26 штамп программы, которого нет в календаре (месяц не 1..12, дня нет в месяце, часы больше 23, минуты или секунды больше 59), на диск не пишется: вместо него пишется текущий штамп часов ядра (сбитые часы — 2026-01-01 00:00), как у копии с неверной датой источника. Доли секунды создания при

этом — 0; их больше 199 — тоже 0. Дата до 1980 года в FAT не кодируется: программе, у которой такая дата (например, у файла с ПК), стоит передать дату 0 — WildDOS поставит текущую. Структура программы не меняется (кроме applied_attr); что записано, покажет GET_METADATA.

GET_METADATA

Операция 8 (с 2026-09-26; есть, если в QUERY_CAPS `result_count+1` bit0 = `FILEX_CAP2_GET_METADATA`) заполняет ту же 16-байтовую структуру по записи последнего FENTRY и ничего не пишет: size=16, attr_mask= `#27`, attr_value и applied_attr — атрибут записи, time_mask=%111, времена — поля записи. Структуру можно без правки отдать в SET_METADATA. Штамп с носителя, которого нет в календаре, отдаётся как 2026-01-01 00:00 (доли секунды создания тогда — 0, их больше 199 — тоже 0); запись на носителе не меняется. `result_count` =16, `result_flags` — атрибут.

READ_FAT

Операция читает используемую копию FAT через тот же файловый контекст WildCommander. При зеркалировании читается FAT0, а при отключённом зеркалировании — копия, выбранная `BPB_ExtFlags`.

`offset` задаётся от начала FAT, а `length` и `offset` должны быть кратны размеру сектора. Запрос не может выходить за границу FAT; фактически прочитанное число байт возвращается в `result_count`.

10.4 GET_FS_INFO, 48 байт

Offset	Размер	Поле
0	1	size=48
1	1	version=1
2	1	flags
3	1	sectors_per_cluster
4	2	bytes_per_sector=512
8	4	total_clusters
12	4	free_clusters или `#FFFFFFFF`
16	4	serial
20	12	label 11 байт + ноль
32	4	total_sectors
36	4	next_free или `#FFFFFFFF`
40	1	FAT count
41	1	active FAT
42	1	BPB_ExtFlags low
43	1	media state
44	4	sectors per FAT

Flags:

```
bit0 MEDIA_PRESENT
bit1 WRITABLE
bit2 FREE_KNOWN
bit3 LABEL_VALID
bit4 WRITE_STATE_KNOWN
```

Если bit4 сброшен, состояние write-protect неизвестно; это не означает «носитель доступен для записи».

`free_clusters` и `next_free` берутся из FSInfo, только если его сектор лежит внутри резервной области тома, подписи и нули перед `#55AA` на месте, а `next_free` — кластер данных этого тома; иначе `#FFFFFFFF` (с 2026-10-10). Флаг вызова `flags bit0=REFRESH_FREE` пересчитывает свободные кластеры по FAT и ставит `FREE_KNOWN` — на большой карте это долго (прежде флаг принимался и ничего не менял).

10.5 Статусы FILEX

Код	Имя	Значение
`#00`	`OK`	Операция полностью завершена
`#01`	`EOF`	Конец/частичное чтение; проверять result_count
`#10`	`BAD_BLOCK`	size/reserved/общая структура неверны
`#11`	`BAD_VERSION`	Версия не поддерживается
`#12`	`BAD_OPERATION`	Неизвестный op
`#13`	`BAD_BUFFER`	Буфер вне окна, пересекается или переполняется
`#14`	`BAD_LENGTH`	Недопустимая длина
`#15`	`NO_CONTEXT`	Нет FILEX provider или FENTRY context
`#16`	`ENTRY_CHANGED`	SFN изменилась после выбора
`#17`	`IS_DIRECTORY`	Файловая операция дана каталогу
`#18`	`READ_ONLY`	Запрещено атрибутом
`#19`	`NOT_FOUND`	Источник отсутствует
`#1A`	`EXISTS`	Назначение уже существует
`#1B`	`NOT_EMPTY`	Каталог назначения не пуст
`#1C`	`INVALID_NAME`	Имя не прошло проверку
`#1D`	`INVALID_MOVE`	Цикл/несовместимый перенос
`#20`	`FAT`	Неверная цепочка или геометрия
`#21`	`MEDIA`	Низкоуровневый I/O отказ
`#22`	`NO_SPACE`	Недостаточно реальных data clusters/slots
`#23`	`SIZE_OVERFLOW`	Offset/length/new size вышли за LE32
`#24`	`ROLLBACK`	Компенсация сама не завершилась
`#25`	`COMMITTED_CLEANUP`	Результат опубликован, cleanup не закончен
`#26`	`INTERNAL`	Внутреннее противоречие

10.6 Шаблон блока в ASM

```
FILEX_OP       EQU FilexBlock+2
FILEX_FLAGS    EQU FilexBlock+3
FILEX_OFFSET   EQU FilexBlock+4
FILEX_BUFFER   EQU FilexBlock+8
FILEX_LENGTH   EQU FilexBlock+10
FILEX_RESULT   EQU FilexBlock+24
FILEX_STATUS   EQU FilexBlock+28

FilexBlock:
        DB      32,1,0,0
        DD      0
        DW      0,0,0,0
        DD      0,0,0
        DB      0,0
        DW      0
        ASSERT  $-FilexBlock == 32
```

Для реального плагина подключайте `source/CORE32_FILEX_API.ASM`, а не дублируйте offset вручную.

11. Согласованность FAT32

CORE32 Improved придерживается следующих правил:

- при mirror mode изменения идут во все FAT;
- при `BPB_ExtFlags.bit7=1` используется только выбранная active FAT;
- старший зарезервированный полубайт FAT32 сохраняется при записи;
- обычный cluster, EOC `#0FFFFFF8..#0FFFFFFF` и reserved/bad диапазон `#0FFFFFF0..#0FFFFFF7` различаются;
- свободный cluster ищется только внутри реального data-cluster range;
- нулевой файл имеет size=0, cluster=0 и не получает цепочку;
- `FSI_Nxt_Free` принимается при монтировании только после проверки диапазона и нулевой записи FAT; иначе используется RAM-курсор 15/16 (подсказка занята и 15/16 занята — от подсказки, с 2026-10-11) с полным циклическим обходом; MKFILE не записывает FSInfo автоматически, а найденный курсор остаётся в FSTFRC текущего сеанса;
- LFN удаляется только при правильной sequence/checksum, SFN — последней;
- при создании и переименовании SFN и LFN файлов/каталогов проверяются в общем пространстве имён без учёта регистра; кандидат SFN сравнивается и с LFN;
- суффиксы `~1` … `~999` сохраняют внутренние нули; исчерпание диапазона и ошибка чтения прекращают создание; ранее созданные имена не исправляются автоматически;
- с 2026-09-26 имя 8.3, которому длинное нужно только из-за заглавных букв (`boot.$C`, `ReadMe.txt`, `Файл.txt`), получает короткое без суффикса — само имя заглавными (cp866 тоже), если такого короткого в каталоге ещё нет (PLAIN_ALIAS расширения, вход LONG ядра; так делает и Windows). Имена длиннее 8.3, с пробелом, лишней точкой или знаками `+,;=[]` получают `~n`, как прежде. Причина: ПЗУ ZX Evolution ищет `boot.$c` только по короткому `BOOT $C`, а прежнее `BOOTx$~1` после замены файла из WC давало «boot.$c NOT FOUND!»;
- max LFN 255 байт может занимать три сектора каталога;
- полный каталог без маркера `#00` заканчивается по EOC, а не повторяет старый буфер.

12. Нереентерабельные глобальные объекты

```
одновременно существует только один:

• NW0 stream handler
• directory iterator
• current ENTRY
• APPEND context/cache
• FILEX request/provider call
• SECBU/LOBU/LOBU2
```

Нельзя:

- вызывать файловый API из plugin INT-handler;
- начать вторую операцию до возврата первой;
- держать два параллельных APPEND-контекста;
- считать, что FENTRY одного файла не влияет на FILEX/APPEND другого;
- использовать флаги результата после вызова UI, который изменил `F`.

Для нескольких логических задач сериализуйте операции и перед каждой заново выбирайте файл/поток.

13. Как расширять DOS API

Если функция помещается в существующий контракт, предпочтителен версионируемый operation API 77:

```
добавить op constant
   → расширить QUERY_CAPS
   → проверить block/version/flags/buffers
   → реализовать provider operation
   → определить стабильные status/result fields
   → сохранить старые op 0..6 без изменений
```

Не добавляйте ещё один случайный регистровый API для SEEK/TRUNCATE: FILEX уже решает передачу offset32, длины, двух буферов и расширение версий.

Новый прямой элемент `#4000` потребует сдвига стабильной таблицы и всех адресов — это худший вариант. Приватную тяжёлую реализацию можно разместить в CORE32_EXT на странице Improved и вызвать через существующий шлюз, но её ID не становится публичным для плагинов.

14. Исходники

Файл	Назначение
`source/KERNEL.ASM`	Layout данных и 31-entry таблица `#4000`
`source/CORE32.ASM`	Основной FAT32 и потоковый код
`source/CORE32_EXT.ASM`	Исправления, APPEND и тяжёлые операции ядра
`source/CORE32_FILEX_API.ASM`	Канонические константы FILEX
`source/plugins/filex/FILEX_RUNTIME.ASM`	Реализация операций FILEX

Файл	Назначение
`source/MD20.ASM`	Публичные X*-обёртки `#6006`
`source/CORE32_API.md`	Дополнительный аудит поведения и кодов

Wild Commander Improved: карта использования памяти

Этот документ отвечает на главный практический вопрос автора плагина или драйвера: **какую память можно использовать, кто уже владеет каждым диапазоном и что изменится во время вызова API**.

> Самое важное правило: адрес Z80 и номер физической страницы — разные вещи. Адрес `#8000` не называет конкретную RAM. Он означает окно, в которое в данный момент подключена одна из физических страниц по 16 КиБ.

1. Карта за тридцать секунд

```
Логическое адресное пространство Z80                  Обычное состояние WC

 #FFFF +-----------------------------------------+
       | окно #C000: постоянно переключается     |  плагин / экран /
       | API 0, драйверы, файловые буферы, FILEX |   временная страница
 #C000 +-----------------------------------------+
       | окно #8000: верх WC или страница плагина |  обычно phys #02,
       | код плагина запускается через CALL #8000 |   при запуске — plugin
 #8000 +-----------------------------------------+
       | окно #4000: резидентное ядро и менеджер  |  phys #05
       | CORE32, стек/IM2, API #6006, код WC      |
 #4000 +-----------------------------------------+
       | окно #0000: состояние текущего WildDOS   |  обычно phys #F0/#F1,
       | каталог, секторные буферы, драйвер #3800 |  иногда plugin state
 #0000 +-----------------------------------------+
```

Для обычного плагина безопасная стратегия выглядит так:

```
код и постоянные данные       -> страницы, заявленные в заголовке .WMF
рабочий буфер                 -> свободная область своей страницы #8000..#BFFF
ещё один буфер на 16 КиБ      -> дополнительная относительная страница плагина
файловые операции             -> только публичный CALL #6006
прямой OUT в порты страниц    -> не использовать
```

2. Термины: окно, страница и смещение

TS-Conf делит видимые Z80 64 КиБ на четыре окна по 16 КиБ. Физическая страница также имеет размер 16 КиБ, поэтому любое её смещение равно младшим 14 битам логического адреса.

Окно	Диапазон адресов	Порт страницы	Тень WC
0	`#0000..#3FFF`	`#10AF`	`#6000`
1	`#4000..#7FFF`	`#11AF`	`#6001`
2	`#8000..#BFFF`	`#12AF`	`#6002`
3	`#C000..#FFFF`	`#13AF`	`#6003`

Пример: физическая страница `#75`, подключённая к окну `#8000`, видна как `#8000..#BFFF`. Байт по физическому смещению `#0123` читается по адресу `#8123`.

```
physical page #75                        logical window #8000

 offset #0000 ------------------------> address #8000
 offset #0123 ------------------------> address #8123
 offset #3FFF ------------------------> address #BFFF
```

Тени `#6000..#6003` — часть контракта менеджера. Если выполнить сырой `OUT (#12AF),A`, аппаратура переключится, но WC продолжит считать, что в окне остаётся старое значение из `#6002`. Следующее восстановление страницы станет непредсказуемым. Плагин должен применять API 0, 65, 78–81 или межстраничный вызов `#6028`.

3. Обычная раскладка окон

После инициализации основные резидентные части выглядят так:

Окно	Типичная физическая страница	Владелец	Можно ли плагину писать
`#0000`	`#F0` или `#F1`	Состояние WildDOS левой/правой панели	Нет
`#4000`	`#05`	CORE32, стек, IM2, основной код WC	Нет
`#8000`	`#02`	Верхняя часть WC; при запуске заменяется страницей плагина	Только в своей подключённой странице
`#C000`	меняется	Плагины, экран, загрузчик, буфер или расширение	Только после осознанного подключения своей страницы

Физические `#05` и `#02` вместе содержат непрерывный логический образ Commander `#4000..#BFFF`. Поэтому переход границы `#7FFF/#8000` в исходнике WC не означает переход к другому модулю API — это лишь граница двух окон MMU.

3.1 Что видит запущенный плагин

Загрузчик берёт относительную страницу из поля заголовка `.WMF +35`, подключает её к `#8000` и выполняет `CALL #8000`.

```
до запуска                          во время CALL #8000

#8000 -> phys #02 (сам WC)          #8000 -> phys (#61 + base + entry_page)
#C000 -> временная страница         #C000 -> задаётся плагином через API 0
```

Плагин не должен вычислять `#61 + base` самостоятельно. Относительный номер `0..pages-1` передаётся API, а текущая база хранится менеджером.

3.2 Что происходит во время файлового вызова

Обёртки API 48–63 и 72–77 временно подключают рабочую страницу файлового контекста к окну `#0000`. Предыдущая страница сохраняется в `#600C`, файловая берётся из `#600B`, после вызова прежняя раскладка восстанавливается.

```
плагин                диспетчер #6006                WildDOS

CALL #6006  ------->  save page0 in #600C  ------->  code #4000..
                      map #600B at #0000               data #0000..
                      call #40xx                       driver #3800
RET         <-------  restore page0       <-------    flags/result
```

Это одна из причин не вызывать `#40xx` из обычного плагина напрямую: одна и та же процедура кода CORE32 получит совершенно другие данные, если в `#0000` подключена не та страница.

3.3 Что происходит в API 77 FILEX

FILEX специально разводит участников по трём окнам:

```
#0000..#3FFF  текущий контекст WildDOS: каталог, FAT, драйвер
#4000..#7FFF  резидентный CORE32 и шлюз #402D
#8000..#BFFF  блок запроса FILEX и пользовательские буферы
#C000..#FFFF  код провайдера FILEX из его физической страницы
```

Поэтому блок запроса и все передаваемые FILEX буферы обязаны целиком лежать в `#8000..#BFFF`. Адрес `#C000` для них недопустим: во время операции там уже исполняется провайдер.

4. Детальная карта логических 64 КиБ

4.1 Окно `#0000..#3FFF`: изменяемое состояние WildDOS

CORE32 разделяет код и данные. Код постоянно находится по `#4000`, а данные берутся из текущей физической страницы окна `#0000`.

Диапазон	Назначение в странице WildDOS
`#0000..#0FFF`	Нижняя половина буфера перечисления каталога
`#1000..#1FFF`	Верхняя половина буфера перечисления каталога
`#2000..#2FFF`	Не считать свободной: часть рабочего состояния/буферов вызывающего кода
`#3000..#31FF`	Один сектор FAT (`SECBU`)
`#3200..#33FF`	Секторный буфер `LOBU`
`#3400..#35FF`	Второй секторный буфер `LOBU2`
`#3600..#37F7`	Внутренняя рабочая область
`#37F8..#37FF`	Состояние итератора каталога `CGDE/CGHL/CGPZ/CGFL`
`#3800..#3847`	24 трёхбайтовых входа текущего драйвера
`#3848..#3867`	Общие переменные драйвера и потока
`#3868..#3887`	Текущая 32-байтовая FAT directory entry
`#3888..#38CC+`	Геометрия FAT32, кластеры, позиции и состояние операций
остаток до `#3FFF`	Не является публичным буфером

Две основные страницы состояния:

Физическая страница	Назначение
`#F0`	Контекст WildDOS левой панели
`#F1`	Контекст WildDOS правой панели

API может временно использовать и страницу плагина как файловый контекст. Признаком выбранного контекста служит `#600B`; полагаться только на `#F0/#F1` внутри плагина нельзя.

> API `GEDPL` использует `#1000..#1447` текущей страницы окна `#0000`. Содержимое этого диапазона после переключения в основной текстовый режим сохранять нельзя.

4.2 Окно `#4000..#7FFF`: ядро и нижняя часть WC

```
#4000 +-----------------------------------------------------+
      | CORE32: таблица #4000 и весь код WildDOS            |
#5BF2 +-----------------------------------------------------+
      | #5BF2..#5BF9: обновление панелей после плагина      |
#5BFA +-----------------------------------------------------+
      | резерв, IM2, рабочие буферы и стек; не использовать |
#6000 +-----------------------------------------------------+
      | фиксированные тени страниц и публичный API          |
#6011 +-----------------------------------------------------+
      | MD20: запуск, панели, диспетчер API                  |
#7143 +-----------------------------------------------------+
      | PS2P: клавиатура и строковый ввод                    |
#7828 +-----------------------------------------------------+
      | WCIF: окна, текст, интерфейс                         |
#7EA0 +-----------------------------------------------------+
      | WCVLS: панели и общие структуры (продолжение выше)   |
#8000 +-----------------------------------------------------+
```

Точные важные границы:

Диапазон/адрес	Назначение
`#4000..#5BF1`	Рабочий образ CORE32
`#4000..#405C`	31 строка внутренней JP-таблицы CORE32
`#5176..#52FD`	Резидентный блок расширения: `#5176` — OVERLAY_STATE (оверлей FILEX, 5.5), с `#5177` — постоянные входы FILEX_REQUIRED, STREAM, CLASSIFY, GIPAG_FILE и переход потока; CORE_MKSG, CORE_SRHFCL и выход из `#E8`; с 2026-10-11 — проверка карты SD-ZC (ZCG_PROBE) и перемонтирование панелей после её возврата (MEDIA_IDLE_CODE). Кладёт установщик на место упакованных драйверов, те переехали в `#E8`
`#52FE..#52FF`	Не используется (2 байта резидентного места)
`#5300..#56C1`	Регистрация, поиск и пропуск метаданных WMF `#11`, с 2026-10-11 — отбор плагинов для меню Shift+F3 (MENU_FIT, docs/02, 2.2); внутренние входы WC/PLM, не публичный API
`#56C2..#56C5`	Не используется (4 байта)
`#56C6..#5AA9`	Второй резидентный блок (с 2026-10-10): входы перевода имён CP866 ↔ UTF-16 с полной таблицей (ORU, UCS, UCH ядра ведут сюда), проверка буфера LOAD256, выбор файла GIPAG_FILE, сообщение и останов без FILEX; с 2026-10-11 — входы чтения и записи SD-ZC через проверку вынутой карты (ZCG_*), окно «Insert boot drive», слот, серийный номер и сектор BPB загрузочной карты (BOOT_CS, BOOT_SERIAL, BOOT_LBA), MEDIA_FENCE и MAIN_SEEN (docs/04, 8.5); отказ загрузчика SPG — окно «SPG load failed - press reset» и останов (SPG_FAIL)

Диапазон/адрес	Назначение
`#5AAA`	Распаковщик драйверов DEHR
`#5BF2..#5BF9`	Резидентный обработчик возврата плагина с кодом 3, физическая страница #05
`#5BFA`	Резерв перед нижним слотом IM2 #5BFB
`#5BFF..#5C02`	Четыре байта таблицы IM2
`SP=#5FFE`	Начальный стек WC; стек растёт к меньшим адресам
`#6000..#6010`	Фиксированное состояние менеджера и API
`#6006`	Публичный диспетчер API плагинов
`#6011..#7142`	MD20
`#7143..#7827`	PS2P
`#7828..#7E9F`	WCIF
`#7EA0..#7FFF`	Начало WCVLS

Диапазон `#5BF2..#6010` нельзя использовать как «дыру». Здесь находятся обработчик обновления панелей, рабочий стек, IM2 и фиксированные переменные непосредственно перед кодом WC. Установщик отдельно копирует восемь байт обработчика сразу за CORE32; ASSERT проверяет точное соседство и отсутствие пересечения с IM2. Сам размер CORE32 и адреса его API не увеличены.

4.3 Фиксированные ячейки `#6000..#6010`

Адрес	Размер	Назначение
`#6000`	1	Тень страницы окна `#0000`
`#6001`	1	Тень страницы окна `#4000`
`#6002`	1	Тень страницы окна `#8000`
`#6003`	1	Тень страницы окна `#C000`
`#6004`	1	Флаг Esc плагинного менеджера
`#6005`	1	Флаг Enter
`#6006..#6008`	3	`JP` в публичный диспетчер API
`#6009..#600A`	2	Кадровый таймер
`#600B`	1	Активная рабочая страница файлового контекста
`#600C`	1	Временно сохранённая страница окна `#0000`
`#600D`	1	Конфигурация/тип видео
`#600E`	1	Высота текста: 25, 30 или 36 строк
`#600F`	1	Режим текстового экрана
`#6010`	1	Флаг обработки `wc.ini`

Читать документированные значения можно. Записывать в них напрямую нельзя, если конкретный API явно не требует обратного: эти ячейки одновременно используются менеджером, обработчиком прерываний и файловыми обёртками.

4.4 Окно `#8000..#BFFF`: верх WC или рабочая страница плагина

В штатном состоянии здесь подключена физическая страница `#02` и продолжается резидентный образ. Одна и та же страница проходит две жизни: при запуске в ней лежит загрузочный образ `boot.$C`, после запуска — рабочая часть WC и буферы. Адреса ниже — сборки `boot.$C` `f0d666f2…` (2026-09-28); всё за WCINI сдвигается вместе с его концом, и код обращается туда только по меткам (прежние числа `#AB9F`, `#B39F`, `#B4B6`, `#B59F` — исторические).

Рабочая карта (после запуска):

Диапазон	Модуль/ресурс
`#8000..#8530`	Продолжение WCVLS, начиная со структуры `F5WND=#8000`
`#8531..#9148`	WCFX: файловые команды менеджера
`#9149..#9B7B`	WCVW: просмотрщики и загрузчик плагинов
`#9B7C..#AB4E`	WCINI: конфигурация, пути, сортировки, обновление панелей; с `#A6EE` — окно хода F5/F8 (`CP_*`, со счётом файлов операции `CP_COUNT` / `CT_TREE`) и проверки обхода и удаления (`DL_*`, `TREE_*`, `CP_SOURCE`), 1121 байт
`#AB4F..#AFFF`	Одноразовые данные запуска (см. ниже); после запуска не используются
`#B000..#BFFF`	LOBU — рабочий буфер: просмотрщики и файловые операции его затирают

Загрузочный образ (что лежит в этой же странице, пока идёт запуск):

Диапазон	Длина	Содержимое	Когда нужно
`#AB4F..#AF24`	982	Шрифт FONT32L3, HR-поток (`#800` байт после распаковки)	VDAC: DEHR распаковывает его в страницы `#01` и `#09` с `#C000`
`#AF25..#B074`	336	MLZ50F (`#E9`) и DEHR1M (`#117`), один HR-поток	VDAC: одним вызовом DEHR в страницу `#01` с `#FE00` (DEHR1M — с `#FEE9`)
`#B075..#B101`	141	Установщик Improved `WDOS_INSTALL_COPY`: метка конца файла, ядро (LDIR драйверов и DEHR, DEHR кода), шина SPI в покой (SpiBusIdle, 24 байта), копия обработчика, распаковка обеих частей расширения, вызов EXT_INSTALL_TAIL	Из заглушки `#C000` (INSTALL_WILDDOS), до FINI
`#B102..#BEB1`	3504	Часть 1 расширения CORE32_EXT, HR-поток (7202 байта после распаковки; с 2026-10-10 — почти всё загрузочное расширение)	Установщик: DEHR распаковывает её в страницу `#E8` с `#C000`
`#BEB2..#BF5F`	174	Свободно: с оверлеем FILEX расширение помещается в часть 1 с запасом, границу частей держит поток части 2 не длиннее её данных	—
`#BF60..#BF67`	8	Шаблон обработчика возврата плагина с кодом 3	Копируется в `#5BF2..#5BF9` страницы `#05`
`#BF69..#BFD3`	107	AUTOEXEC (`AX_PATH`) — фиксированный адрес	Поздняя стадия запуска
`#BFD4..#BFFD`	42	Перенесённый INIT — фиксированный адрес	Самая ранняя стадия запуска

За `#BFFF` загрузочный образ продолжается в окне `#C000` загрузочной страницы (с 2026-09-27): заглушка установщика `#C000..#C00B` (INSTALL_WILDDOS), драйверы DR1..DR4 и DEHR ядра как есть `#C00C..#CA87` (2684 байт; LDIR — в `#5176..#5BF1`), код ядра `#4000..#5175` HR-потоком `#CA88..#D733` (3244 байта; 4470 после распаковки), часть 2 расширения HR-потоком `#D734..#D787` (84 байта; 84 после распаковки — граница частей держит поток каждой части не длиннее её данных, docs/04, раздел 2), за ним 2 байта метки сборки (`WDOS_EXTENSION2_MARK`: по ней установщик видит, что файл дочитан; нет её — красный бордюр и остановка). Логический конец — `#D78A`, файл — 30602 байта (запас до предела — 2166 байт: с 2026-10-10 холодная часть расширения — оверлей FILEX, 5.5); последний сектор Hobeta не добивается. Весь файл — не длиннее 32768 байт (конец не дальше `#E000`, ASSERT в BOOT.ASM и проверка build.ps1): BIOS TS-Conf читает его целыми кластерами с `#6000`, и при кластере 32 КиБ второй кластер через `#FFFF` затирал `#0000..#5FFF` (docs/04, раздел 2). Прежде образ CORE32 лежал здесь как есть (`#C00C..#DBFD`), за ним — копия первых байт окна `#DC00..#DC10` (её исполнение не доказано; ушла вместе с образом).

До 2026-09-26 шрифт и оба распаковщика лежали здесь несжатыми (2560 байт) и копировались LDIR; упаковка дала 1249 байт запаса перед `#BF60` (см. 5.3). К 27 сентября их почти съели короткие имена без «~n» (PLAIN_ALIAS расширения), приведение шины SPI в покой в установщике, одно окно хода F5/F8 (WCINI.CP_*, несжатых — код рисует и потому резидентный) и проверки цепочек перед обходом; оставалось 10 байт. Поэтому 27 сентября расширение разделено на две части: часть 2 лежит в хвосте файла, а граница частей с того же дня выбирается по месту (build.ps1): поток части 1 занимает всё место до `#BF60`, в хвост идёт остаток. Файл при этом вырос было до 138 секторов (35345 байт) — и с карты с кластером 32 КиБ WC перестал запускаться (docs/04, раздел 2); поэтому код ядра теперь хранится HR-потоком, а весь `boot.$C` — не длиннее 32768 байт. Установщик переносит поток части 2 на место потока части 1 (`#B102..`), поэтому и поток части 2 должен помещаться до `#BF60`. Шрифт и распаковщики распаковывает тот же DEHR ядра (`#5AAA`), что и расширение: к вызову VDAC ядро уже установлено (WC_ENTRY: INIT → INSTALL_WILDDOS → IM2 → FINI → VDAC). Байты в страницах `#01` и `#09` те же, что прежде.

Граница `#BF60` жёсткая: упакованные данные должны кончаться не дальше неё (ASSERT в BOOT.ASM), а шаблон, AUTOEXEC и перенесённый INIT имеют внешние фиксированные адреса. До загрузки плагинов установщик копирует шаблон в постоянный диапазон `#5BF2..#5BF9` страницы #05. Оба пути возврата вызывают эту копию; она перечитывает активную и противоположную панели штатными процедурами.

После запуска `#B000..#BFFF` является рабочим буфером (LOBU): его затирают просмотрщики и файловые операции, а прежде и сохранение настроек по F9 копировало сюда 4096 байт. Поэтому размещать в этом диапазоне постоянно вызываемый код нельзя: прежний вызов обработчика по #BF60 приводил к выполнению повреждённых байтов после выхода из редактора с сохранением.

При запуске `.WMF` всё окно заменяется выбранной страницей плагина. Поэтому плагин может хранить здесь код, структуры SOW и данные, но только в границах собственной 16-КиБ страницы.

Полезные общие структуры WC в резидентной странице `#02`:

Адрес	Назначение
`#7EA0`	36-байтовая левая панель `WINDOW`
`#7EC4`	36-байтовая правая панель `WINDO2`
`#7F14..#7F1B`	Состояние загрузчика плагинов, база и указатели заголовков
`#7F28..#7F3B`	Кластеры/размеры текущей файловой операции
`#7F4C..#7F4E`	Флаги вызовов из IM2
`#7F6B..#7F8A`	Текущая 32-байтовая directory entry менеджера

Адрес	Назначение
`#7FB7`	Окно ошибки `ERWND`
`#7FE2`	Окно прогресса `PBWND`
`#7FF4`	Диалог имени `MDWND`
`#8000`	Диалог копирования `F5WND`

Это внутренняя карта WC, а не свободный ABI-буфер. Плагин получает нужные значения через регистры запуска и публичные вызовы.

4.5 Окно `#C000..#FFFF`: общий переключаемый ресурс

В этом окне по очереди появляются:

- относительные страницы плагина через API 0;
- физические видеостраницы через API 65;
- временные секторные и загрузочные страницы WC;
- заголовки `.WMF` на физической странице `#60`;
- страница провайдера FILEX;
- исходный физический образ CORE32 только во время установки.

API 0 меняет отображение и не выдаёт отдельный «указатель». После него адрес `#C000` уже относится к новой странице:

```
WLD     EQU #6006

        ; Подключить относительную страницу 1 нашего плагина к #C000.
        LD      A,1
        EX      AF,AF'
        XOR     A                       ; API 0 MNGC_PL
        CALL    WLD

        LD      HL,#C000                ; начало именно этой страницы
```

Если вызываемый код ожидает прежнюю страницу `#C000`, сохраните относительный контекст в собственной логике или используйте `#6028`, который предназначен для межстраничного вызова с восстановлением.

5. Карта физических страниц `#00..#FF`

```
#00      основной текстовый экран
#01      шрифт/системный графический ресурс
#02      верхняя резидентная часть WC (#8000..#BFFF)
#03–#04  внутренние/непубличные
#05      CORE32 и нижняя часть WC (#4000..#7FFF)
#06–#0A  внутренние/непубличные
#0B–#13  девять страниц каталога левой панели
#14–#1C  девять страниц каталога правой панели
#1D      внутреннее/непубличное
#1E      backing page левой панели
#1F      backing page правой панели
#20–#5F  64 видеостраницы API 65/80/81
#60      цепочка 512-байтовых заголовков загруженных .WMF
#61–#9E  реально доступный пул загруженных плагинов (до 62 страниц)
#9F–#A0  граница/непубличный резерв; самовольно не занимать
#A1–#BF  временные страницы TAP/SNA/Z80
#C0–#E7  40 страниц RAM-диска TR-DOS Mounter
#E8–#EF  приватные расширения Wild Commander Improved
#F0–#FF  Commander и системные страницы
```

5.1 Верхние диапазоны — обязательный контракт

Диапазон	Размер	Владелец
`#A1..#BF`	31 страница	TAP/SNA Mounter
`#C0..#E7`	40 страниц	TR-DOS Mounter
`#E8..#EF`	8 страниц	Improved
`#F0..#FF`	16 страниц	Commander/система

CORE32_EXT установлен на странице `#E8`. Страница `#E9` — общий пул переменных списков расширений WMF `#11` (16 КиБ, выделение секторами по 512 байт). Страница `#EA` (с 2026-10-11) — сектор BPB вернувшейся загрузочной SD-карты на время сверки её серийного номера. Его содержимое живёт до удаления заголовка плагина и не зависит от выгрузки страниц кода менеджером PLM. Остальные страницы диапазона Improved зарезервированы для совместимых расширений, но не становятся автоматически свободным пулом плагина.

Для нового системного владельца верхней памяти сначала меняют единый контракт `source/PHYSICAL_MEMORY_MAP.ASM`, затем проверяют все пересечения. Нельзя просто выбрать «похожую на свободную» страницу между `#A1` и `#FF`.

5.2 Системные страницы, назначение которых известно

Страница	Назначение
`#F0`	WildDOS и состояние левой панели
`#F1`	WildDOS и состояние правой панели
`#F3..#F6`	Внутренние DMA/загрузочные рабочие страницы WC
`#F7`	Сохранённые пути и позиции панелей
`#F8..#FF`	Системный диапазон; не выделять плагинам

Загрузчик SPG 1.0 (`WCVW.SPG1`) не пишет блоки программы прямо в страницы, от которых сам зависит: блоки страниц `#02` , `#05` и `#E8` (CORE32_EXT, через него идёт чтение файла) грузятся во временные `#F6` , `#F5` и `#F3` и переносятся на места после чтения последнего блока, при закрытых прерываниях. `#F4` — буфер распаковки блоков. Программа, занимающая `#E8` , без этого затирала расширение посреди загрузки и не запускалась.

Блок, который не прочитался (LOAD512 с CF=1: отказ носителя, вынутая загрузочная карта — после её возврата обмены до главного цикла отказывают), с 2026-10-11 останавливает загрузку: программа не запускается, красное окно «SPG load failed - press reset» и останов (`WDOS_EXT.SPG_FAIL` , второй резидентный блок). Вернуться в WC нельзя — его страницы уже заняты блоками программы, загрузчик стоит в окне `#0000` . Прежде ошибка не проверялась, и программа запускалась без блока. Если программа уже заняла текстовую страницу WC (у TMNT — `#00..#0F`), окна «Insert boot drive» и отказа рисуются, но экран чёрный. Места в WCVW до `RUNSPG` (`#957B`) больше нет: вызов `CPG` с `DE` (источник, назначение) дал два байта, заполнитель — ещё четыре.

На странице `#F7` логические смещения `#1000` и `#2000` используются для состояния левой и правой панели. Это не те же данные, что каталожные страницы `#0B..#1C` .

5.3 Где расти улучшенному ядру (2026-09-27)

Строки сводки о хвосте `boot.$C` , странице `#E8` , резидентном месте `#05` и страницах Improved — по сборке с WMF `#11` (2026-10-09, раздел 5.4); остальные числа — снимок 2026-09-27.

Тесно не в страницах, а в хранении кода внутри `boot.$C` . Сводка поиска места:

Место	Сейчас	Что это значит
Упакованные данные перед `#BF60` (4.4)	заняты потоком части 1	Граница частей — по месту: рост резидентного кода WC (WCINI и соседи) и расширения уходит в хвост файла
Хвост `boot.$C` до `#E000` (4.4)	**2166 байт запаса** (файл 30602 байта, с 2026-10-11)	Предел — 32768 байт всего файла (BIOS TS-Conf, кластер 32 КиБ). Холодная часть расширения (4905 байт кода) — оверлей FILEX, в `boot.$C` её нет (5.5)
Страница `#E8` после распаковки	загрузочная часть до `#D3BD` , оверлей FILEX `#D3BE..#E6E6` , рабочие таблицы без начальных значений до `#FFFE` (корень, проверка F8); свободен 1 байт (с 2026-10-11: поиск места по сектору и выбор подсказки FSInfo)	Оверлей занимает ту же RAM, что прежде этот код, — свободной RAM в `#E8` не прибавилось. Рост кода сдвигает таблицы — ASSERT `BSS_END <= #10000`
Резидентное место в `#05` (4.2)	2 байта `#52FE..#52FF` , 4 байта `#56C2..#56C5`	Бывшие упакованные драйверы; там резидентный блок расширения (392 байта, `#5176..#52FD`), код WMF `#11` (962 байта, `#5300..#56C1`) и второй резидентный блок (996 байт, `#56C6..#5AA9` , вплотную к DEHR)

Место	Сейчас	Что это значит
CORE32, страница `#05`	4 байта `#4DE1..#4DE4`, 1 байт `#4D5A` перед MKDIR, по 1 байту перед GENTRY, FELD и RENAME (2 байта перед NXTETY с 2026-09-26 заняты исправлением NXMR, 2 перед CURIT с 2026-09-27 — переходом потока через STREAM_LINK; с 2026-09-27 за MKDIR стоит хвост MKFILE — MKSG вне шлюза, — а обёртка COPY_SAVE ушла в WCFX)	Входы API на фиксированных адресах (ASSERT), сдвигать нельзя
MD20 / WCFX	1–3 байта / 3 байта `#87D2..#87D4` и 1 байт `#8B47` (с 2026-09-27 `#87C4` — цикл порций FC_LOOP, 8 пустых байт после CP_DONE — COPY_SAVE)	Точные модули на фиксированных границах
Страница `#E9`	пул метаданных WMF `#11`, 16 КиБ	Списки расширений; не доступна плагинам как рабочая память
Страница `#EA`	сверка вернувшейся загрузочной SD-карты (с 2026-10-11)	В `#8000` на время чтения её BPB (ZCG_RECOVER, docs/04, 8.5); кода и постоянных данных нет
Страницы `#EB..#EF`	свободны	Зарезервированы за Improved, но без схемы доставки кода места в файле не дают

Резервы на будущее, по убыванию готовности:

1. **Сделано 2026-09-26:** шрифт и распаковщики MLZ50F+DEHR1M хранятся HR-потоками — +1249 байт перед `#BF60`, без новых страниц и без изменения размера `boot.$C` (4.4).
2. **Сделано 2026-09-27: упакованные драйверы CORE32** (Nemo, SD-ZC, SD-NGS, SMUC; 2356 байт `#5176..#5AA9`) установщик переносит в `DRIVERS_E8` страницы `#E8`, DOS_SWP распаковывает их оттуда через шлюз (тест всех выборов A=0..6 — `test_boot_resources.DriverSwap`). На их месте в `#05` — резидентный блок расширения; остальное место свободно под резидентный код (вызов по CALL — 27 тактов против ~1500 у шлюза).
3. **Сделано 2026-09-27: хвост `boot.$C` за прежним концом** — там часть 2 расширения (4.4). Предел — 32768 байт всего файла, а не 159 секторов, как считалось сначала: BIOS TS-Conf читает файл целыми кластерами с `#6000`, и при кластере 32 КиБ второй кластер затирал `#0000..#5FFF` (стенд combo64, опыт с меткой в остатке кластера). Кластер 64 КиБ не годится ни для какого `boot.$C`. Второй загрузчик `WCVW.RUNHOB` читает до конца цепочки; ПЗУ пользователя на железе и запуск из TR-DOS/Mounter не проверены. Хвост при загрузке лежит в окне `#C000` загрузочной страницы: установщик сначала распаковывает часть 1 в `#E8`, возвращает загрузочную страницу и лишь затем переносит поток части 2 в нижнюю память.
4. **Сделано 2026-09-27: код ядра CORE32 в файле — HR-потоком** (4470 байт → 3349); драйверы и DEHR — как есть, их ставит LDIR, код разворачивает этот DEHR. Так файл вернулся в 32768 байт с запасом ~790 байт (с 2026-09-28 — 774); байты ядра build.ps1 берёт из символического прохода и сверяет с основным (CORE32.ASM там собирается за логическим концом ради символов WDOS.*).

Чего не делать:

- не заводить новых обязательных (неотключаемых) `.WMF`: работа ядра не должна зависеть от ещё одного системного плагина;
- не занимать чужие страницы: RAM-диск Mounter `#C0..#E7`, TAP/SNA/Z80 `#A1..#BF`, видеостраницы `#20..#5F`, пул плагинов `#61..#9E` (с PLM его свободный остаток — не постоянный резерв: нерезидентные плагины получают страницы на время запуска);

- не класть постоянный код в LOBU `#B000..#BFFF`;
- новую страницу Improved (`#E9` и дальше) брать только через `source/PHYSICAL_MEMORY_MAP.ASM` с ASSERT на диапазон `#E8..#EF`.

Ядро и страница `#E8` ставятся заново при каждом запуске `boot.$C`. Программа SNA/Z80 или любая другая, которая сама распоряжается всей памятью TS-Conf, может затереть и приватные страницы Improved: на их сохранность за время её работы рассчитывать нельзя.

5.4 Размещение WMF #11 (2026-10-09)

Регистрация, поиск расширений, пропуск метаданных и отбор плагинов для меню Shift+F3 (с 2026-10-11) занимают 962 байта `#5300..#56C1` страницы `#05`, ниже неизменного распаковщика `#5AAA`. `WMF_FORMAT.ASM` задаёт внутренние точки входа; они не добавляют номеров публичного API. ASSERT проверяют границы, буфер имени, таблицу заголовков и прежние входы WCVW/PLM.

Установочный образ находится в общем сжатом потоке CORE32_EXT: временно `#D38B..#D70A` страницы `#E8` (с 2026-10-10 там же за ним исходники обоих резидентных блоков). `EXT_INSTALL_TAIL` копирует его в `#5300`, затем ставит на это место HALT до оверлея FILEX (5.5). Машинный тест исполняет настоящий установщик и сверяет копию побайтно.

Пул списков занимает всю физическую страницу `#E9`, отдельно от кода плагинов. Распределитель строит карту занятости по живым заголовкам. Поэтому удаление или сдвиг заголовка не оставляет занятого навсегда участка; перемещение страниц кода PLM не меняет указателя на список. Старые WMF этот пул не занимают. Предел размера `boot.$C` 32768 байт и прежние ASSERT сохранены. С 2026-10-11 файл занимает 30602 байта, запас — 2166 байт (5.5).

5.5 Оверлей FILEX: холодная часть расширения в FILEX.WMF (2026-10-10)

`boot.$C` упёрся в 32768 байт (запас был 5 байт), а свободной RAM хватает: тесно было в хранении кода. Поэтому код расширения, который нужен только после загрузки FILEX, в `boot.$C` больше не хранится:

- создание, переименование и удаление записей, MKDIR, ENTRY_LANDED, DIR_ALIVE;
- APPEND целиком;
- псевдонимы `~N` (ALIAS_*, PLAIN_ALIAS);
- выделение места и запись FAT (SAVE_FAT_SECTOR, поиск свободного кластера, ALLOCATE_FILE, SAVE_NEXT_FREE_HINT, NOTE_FREED_CHAIN);
- обход дерева F5/F8, FILE_CHAIN, DIR_GROW;
- часы штампов и строка состояния.

Чтение каталогов и файлов (в том числе проверка первого кластера у GFILE — FILE_START_CHECK, и штампы времени списка — CHECK_STAMP) остаётся в загрузочной части: до установки FILEX оно работает.

Это 4812 байт. Они собраны вместе с расширением, в том же модуле, по адресам `#D38B..#E656` (OVERLAY_START = RUNTIME_END), но сборка (`tools/split_overlay.py`) отрезает их от загрузочной части:

Файл сборки	Что это
`Build/CORE32_EXT_BOOT.bin`	Загрузочная часть страницы `#E8` (её пакует `boot.$C`)
`Build/CORE32_EXT_OVL.bin`	Оверлей; хранится в `FILEX.WMF`

Файл сборки	Что это
`Build/CORE32_EXT.bin`	Страница после установки оверлея (для машинных тестов)
`Build/CORE32_EXT_OVERLAY.INC`	Адрес и длина оверлея, адрес и байты штампа — для `FILEX.ASM`

Штамп `EXT_STAMP` (4 байта в загрузочной части) — первые байты SHA-256 загрузочной части и оверлея при нулевом штампе: сборка, где поменялся хоть байт кода расширения, получает другой.

Порядок при старте:

1. Установщик `boot.$C` распаковывает загрузочную часть в `#E8`; `EXT_INSTALL_TAIL` переносит образ WMF `#11` и резидентные блоки в `#05` и ставит на место оверлея HALT (`#76`): вход туда до FILEX (ошибка старта) останавливает машину, а не исполняет чужие байты.
2. Монтирование, чтение `wc.ini` и загрузка `FILEX.WMF` идут только загрузочной частью.
3. FILEX стоит первой строкой `[PLUGINS]` и заявляет раннюю установку (`+33` bit 0): ядро зовёт его с `A=#07` сразу после загрузки, до остальных плагинов. Установщик FILEX (исполняется по `#8000`) подключает `#E8` к окну `#C000` при закрытых прерываниях, сверяет `EXT_STAMP` со своим и копирует оверлей на место; `OVERLAY_STATE` (`#5176`) = 1. Штамп не совпал — оверлей не ставится, `OVERLAY_STATE` = 2. Окно `#C000` и IFF вызывающего возвращаются; второй вызов (AX_RUN, `A=#06`) видит 1 и выходит.
4. AX_RUN первым делом зовёт FILEX_REQUIRED (`#5177`): `OVERLAY_STATE` ≠ 1 — окно в манере сообщений WCINI и `DI:HALT` до первого плагина `#06`:

```
Core extension plugin: FILEX.WMF
was not found in folder /WC.          (FILEX нет или он старый)
in folder /WC doesn't match boot.$C.  (FILEX от другой сборки)
Commander cannot work. HALT.
```

Оверлей исполняется в `#E8`, как прежде этот код, с теми же данными и без лишних шлюзов — разница только в том, откуда он берётся. Свободной RAM в `#E8` от этого не прибавилось; место прибавилось в `boot.$C`. `FILEX.WMF` вырос до 13130 байт и по-прежнему занимает одну страницу (код и оверлей — `#C000..#F149`).

Следствие: WC без `FILEX.WMF` в папке `/WC` (или со старым, или от другой сборки) не работает — это правило проекта (2026-10-10). Запуск `boot.$C` с носителя без папки `/WC` тоже кончается этим сообщением.

6. Пул плагинов

6.1 Как вычисляется физическая страница

Заголовок `.WMF +34` задаёт число страниц. Загрузчик выдаёт каждому плагину непрерывный блок и хранит его относительную базу. Для понимания отладки можно использовать формулу:

```
physical_page = #61 + plugin_base + relative_page
```

Но в рабочем плагине формулу применять нельзя: передавайте только `relative_page` в API 0, 78 или 79. Тогда конфигурация `[PLUGINS]` может менять порядок модулей без пересборки кода.

Номинальная граница загрузчика — `#A0`, однако проверка требует, чтобы сумма страниц была **меньше** `#3F`. Поэтому максимум равен 62 страницам, то есть фактически используются `#61..#9E`; последняя номинальная `#9F` текущим алгоритмом не выдаётся.

6.2 Текущее распределение из `exe/WC/wc.ini`

Строка с `CLOCK.WMF;PLASMATW.WMF;SCRSAVER.WMF` загружает только CLOCK: точка с запятой начинает комментарий.

Плагин	Страниц	Физический диапазон
FILEX	1	`#61` (с 2026-10-10 — вместе с оверлеем ядра, 5.5)
MOUNTER	4	`#62..#65`
SETIME	1	`#66`
RE	1	`#67`
TXTEDIT	2	`#68..#69`
FILE_CR	2	`#6A..#6B`
TRDisp	1	`#6C`
TRDUMP	1	`#6D`
ROM_PROG	1	`#6E`
WPLAYER	3	`#6F..#71`
GSPLAYER	2	`#72..#73`
VIDEO_PL	1	`#74`
BMPV	1	`#75`
ILBMVIEW	2	`#76..#77`
FTVIEW	1	`#78`
GIFVIEW	1	`#79`
MCGSV	1	`#7A`
TAPM	4	`#7B..#7E`
COMPILER	2	`#7F..#80`
CLOCK	6	`#81..#86`
Итого	**38**	**`#61..#86`**

После этой конфигурации следующая база — `#87`, а до лимита загрузчика остаётся 24 страницы, или 384 КиБ. Это свободная ёмкость **пула загрузчика**, а не разрешение обращаться к страницам `#87..#9E` напрямую.

Так выглядит пул без менеджера загрузки. Со штатным `PLM.WMF` (см. раздел 3.4 в Плагинах и публичном API) эта таблица показывает не занятые страницы, а резерв, который плагин получит на время запуска: постоянно в пуле сидят только резиденты `#06`, а базы нерезидентов меняются при каждом уплотнении. Тем более нельзя считать физический номер страницы по этой таблице — только относительный через API 0/78/79.

6.3 Как добавить плагину постоянную память

Если нужны код плюс отдельный буфер 16 КиБ, заявите две страницы:

```
        ; Заголовок .WMF
        DB      2,0                 ; +34 pages=2, +35 entry_page=0
        DB      0,CodeSectors       ; относительная 0: код
        DB      1,0                 ; относительная 1: пустой постоянный буфер
        DS      4*2,0
```

Затем подключайте буфер относительным номером:

```
        LD      A,1
        EX      AF,AF'
        LD      A,79                ; API 79: page 1 -> #8000
        CALL    #6006
        ; Теперь #8000..#BFFF — наш буфер, но код текущей страницы исчез.
```

Последняя строка важна: после замены окна `#8000` нельзя продолжать выполнять код, который находился в этом же окне. Для безопасного вызова процедуры на другой странице используйте резидентный шлюз `#6028` либо исполняйте переключатель из окна `#4000/#C000` по продуманному протоколу.

7. Где размещать буфер: таблица решений

Задача	Рекомендуемая память	Почему
Структура SOW и короткие строки	Свободный хвост entry-page плагина в `#8000..#BFFF`	Постоянно видны во время обычного вызова
Большой постоянный буфер	Отдельная страница, заявленная в `.WMF +34`	Не пересекает другие плагины
`LOAD512/SAVE512`	Буфер своей подключённой страницы; не пересекать `#FFFF`	API работает секторами по 512 байт
`APPEND`	Обычно своя область `#8000..#BFFF`, длина `0..#4000`	Источник должен целиком лежать ниже `#C000`
Блок и буферы FILEX	Только `#8000..#BFFF`	`#C000` занят провайдером, `#0000` — DOS
Список каталога плагина	Собственный буфер; вызывать API 58	Не занимать внутренние `#0000..#1FFF` напрямую
Данные между запусками плагина	Зарезервированная страница плагина	Выделение живёт, пока плагин загружен
Новый системный RAM-диск	Только после изменения общей physical map	Верхние страницы имеют владельцев

Юридически допустимый адрес API ещё не означает, что память принадлежит плагину. Например, APPEND принимает источник начиная с `#4000`, но область `#4000..#7FFF` занята резидентным кодом WC. Её можно читать как источник лишь если вы намеренно передаёте уже существующие неизменяемые байты, а записывать туда свой буфер нельзя.

8. Границы и арифметика без переполнения

Для любого полуинтервала `[address,address+length)` проверяйте:

1. `length` разрешена конкретным API;
2. 16-битное сложение не переполнилось;
3. конечный адрес не вышел за разрешённое окно;
4. последний байт равен `end-1`, а сам `end` уже не принадлежит буферу.

8.1 Проверка буфера FILEX

```
; Вход: HL=адрес, BC=длина.
; Выход: CF=1 — диапазон не помещается в #8000..#BFFF.
CheckFilexBuffer:
        LD      A,H
        CP      #80
        JR      C,.bad
        CP      #C0
        JR      NC,.bad
        PUSH    HL
        ADD     HL,BC
        JR      C,.overflow
        LD      A,H
        CP      #C0
        JR      C,.ok
        JR      NZ,.overflow
        LD      A,L
        OR      A                       ; end может быть ровно #C000
        JR      NZ,.overflow
.ok:
        POP     HL
        OR      A                       ; CF=0 — диапазон допустим
        RET
.overflow:
        POP     HL
.bad:
        SCF
        RET
```

В реальном коде проще хранить блок FILEX и его буфер статически в одной заявленной странице и закрепить границы `ASSERT` при сборке.

8.2 Проверка статической раскладки плагина

```
        DISP    #8000
PluginStart:
        ; ... код и данные ...

FilexBlock:
        DS      32
IoBuffer:
        DS      4096
PluginEnd:

        ASSERT  FilexBlock >= #8000
        ASSERT  PluginEnd <= #C000
        ASSERT  IoBuffer+4096 <= #C000
        ENT
```

Такие проверки лучше ручного подсчёта: ошибка становится ошибкой сборки, а не повреждением памяти на компьютере пользователя.

9. Стек и прерывания

WC устанавливает `SP=#5FFE`; стек Z80 растёт вниз. Рядом находятся конец CORE32 и таблица IM2 `#5BFF..#5C02`. Следствия:

- не размещайте данные в предполагаемом «свободном хвосте» перед `#6000`;
- не оставляйте несбалансированные `PUSH`;
- при временном собственном стеке запрещайте прерывания на время смены `SP` либо обеспечьте обработчику отдельный корректный стек;
- перед `RET` всегда восстановите исходный `SP` и режим прерываний;
- обработчик IM2 может вызывать резидентные плагины, часы и PS/2, поэтому самоподстановочный и файловый код не является автоматически повторно-входимым.

Минимальный безопасный шаблон локального стека внутри своей страницы:

```
SavedSP:
        DW      0

UseLocalStack:
        DI
        LD      (SavedSP),SP
        LD      SP,LocalStackTop
        EI
        ; ... работа без переключения страницы, содержащей стек ...
        DI
        LD      SP,(SavedSP)
        EI
        RET

        DS      128
LocalStackTop:
```

Если во время работы переключить страницу, в которой находится `SavedSP` или сам локальный стек, возврат разрушится. Поэтому для большинства плагинов лучше не менять стек вообще.

10. Частые ошибки памяти

Ошибка 1. Физический номер зашит в код

```
        LD      A,#75
        OUT     (#12AF),A            ; плохо
```

Порядок `[PLUGINS]` изменится — страница `#75` начнёт принадлежать другому модулю. Кроме того, тень `#6002` останется старой.

Правильно: относительный номер плюс API 79.

Ошибка 2. Код переключил окно, из которого сам исполняется

```
        ; Этот код работает по #8100.
        LD      A,1
        ; API 79 прямо сейчас заменит страницу с продолжением кода.
```

Используйте `CALL #6028` или маленький переключатель в другом неизменяемом окне.

Ошибка 3. FILEX блок лежит по `#C000`

Шлюз подключит туда код провайдера и затрёт видимость блока до того, как тот будет обработан. Блок и буферы должны быть в `#8000..#BFFF`.

Ошибка 4. `LOAD512` переходит через `#FFFF`

Исторический поток не превращает адрес `#10000` в следующую физическую страницу. Разбейте операцию так, чтобы каждый вызов целиком помещался в одном видимом диапазоне.

Ошибка 5. Плагин использует `#0000..#3FFF` как вечную память

Файловая обёртка меняет страницу окна `#0000`; указатель остаётся тем же, а физические байты уже другие. Для постоянных данных резервируйте страницу WMF.

Ошибка 6. «Свободная» верхняя страница занята Mounter

Диапазоны `#A1..#FF` разделены между TAP/SNA, Mounter, Improved и системой. Новый владелец обязан пройти через общую карту, иначе повреждение проявится только при запуске другого плагина.

11. Контрольный список автора плагина

- [ ] В `.WMF +34` заявлены все страницы кода и постоянных данных.
- [ ] Код использует относительные страницы, а не физические `#61..#9E`.
- [ ] Переключение выполняется через API или `#6028`, тени `#6000..#6003` остаются согласованными.
- [ ] Код не продолжает исполняться из только что переключённого окна.
- [ ] Все буферы имеют сборочные `ASSERT` на нижнюю и верхнюю границы.
- [ ] LOAD/SAVE не пересекают `#FFFF -> #0000`.
- [ ] APPEND получает источник ниже `#C000` и длину не больше `#4000`.
- [ ] FILEX блок и все его указатели лежат в `#8000..#BFFF`.
- [ ] Нет прямых записей во внутренние `#7Fxx`, `#38xx` и системные страницы.
- [ ] Стек сбалансирован; временная смена `SP` учитывает IM2 и paging.
- [ ] Для новой системной страницы изменён и проверен `PHYSICAL_MEMORY_MAP.ASM`.

12. Источники в проекте

- `source/BOOT.ASM` — границы резидентных модулей и установка CORE32/CORE32_EXT.

- `source/MD20.ASM` — тени страниц, API переключения и файловые обёртки.
- `source/KERNEL.ASM` — карта данных WildDOS по `#0000..#38xx` и таблица `#4000`.
- `source/CORE32.ASM` — фактический диапазон ядра, драйверы и потоковые буферы.
- `source/WCVLS.ASM` — панели и общие структуры WC.
- `source/WCVW.ASM` — загрузчик `.WMF`, границы пула и межстраничные операции.
- `source/WCINI.ASM` — начальные страницы `#05/#02`, состояния панелей и страница путей `#F7`.
- `source/PHYSICAL_MEMORY_MAP.ASM` — обязательная карта верхних физических страниц.
- `exe/WC/wc.ini` — фактический порядок текущих плагинов.

Смежные руководства: публичный API плагинов, панельный менеджер, драйверы и WildDOS/CORE32.